

PPIL Serveur

Généré par Doxygen 1.16.1

1 Hiérarchie de répertoires	1
1.1 Répertoires	1
2 Index hiérarchique	3
2.1 Hiérarchie des classes	3
3 Index des classes	5
3.1 Liste des classes	5
4 Index des fichiers	7
4.1 Liste des fichiers	7
5 Documentation des répertoires	9
5.1 Répertoire de référence de include	9
5.2 Répertoire de référence de src	10
6 Documentation des classes	11
6.1 Référence de la classe Cercle	11
6.1.1 Description détaillée	14
6.1.2 Documentation des constructeurs et destructeur	14
6.1.2.1 Cercle() [1/2]	14
6.1.2.2 Cercle() [2/2]	15
6.1.2.3 ~Cercle()	15
6.1.3 Documentation des fonctions membres	15
6.1.3.1 accept()	15
6.1.3.2 clone()	16
6.1.3.3 getAire()	16
6.1.3.4 getc()	16
6.1.3.5 getPoints()	16
6.1.3.6 getr()	17
6.1.3.7 setc() [1/2]	17
6.1.3.8 setc() [2/2]	17
6.1.3.9 setPoints()	18
6.1.3.10 setr()	18
6.1.3.11 toString()	18
6.1.3.12 verificationRayon()	19
6.1.4 Documentation des données membres	19
6.1.4.1 c	19
6.1.4.2 r	19
6.2 Référence de la classe ExceptionArrayIsTooShort	19
6.2.1 Description détaillée	20
6.2.2 Documentation des fonctions membres	20
6.2.2.1 getMessage()	20
6.2.3 Documentation des données membres	21

6.2.3.1 message	21
6.3 Référence de la classe <code>ExceptionArrayOutOfBounds</code>	21
6.3.1 Description détaillée	22
6.3.2 Documentation des fonctions membres	22
6.3.2.1 <code>getMessage()</code>	22
6.3.3 Documentation des données membres	22
6.3.3.1 message	22
6.4 Référence de la classe <code>ExceptionArrayOutOfBoundsFC</code>	22
6.4.1 Description détaillée	23
6.4.2 Documentation des fonctions membres	23
6.4.2.1 <code>getMessage()</code>	23
6.4.3 Documentation des données membres	24
6.4.3.1 message	24
6.5 Référence de la classe <code>ExceptionCercle</code>	24
6.5.1 Description détaillée	25
6.5.2 Documentation des fonctions membres	25
6.5.2.1 <code>getMessage()</code>	25
6.5.3 Documentation des données membres	25
6.5.3.1 message	25
6.6 Référence de la classe <code>ExceptionConnexionImpossible</code>	25
6.6.1 Description détaillée	26
6.6.2 Documentation des fonctions membres	26
6.6.2.1 <code>getMessage()</code>	26
6.6.3 Documentation des données membres	27
6.6.3.1 message	27
6.7 Référence de la classe <code>ExceptionCouleurInconnue</code>	27
6.7.1 Description détaillée	28
6.7.2 Documentation des fonctions membres	28
6.7.2.1 <code>getMessage()</code>	28
6.7.3 Documentation des données membres	28
6.7.3.1 message	28
6.8 Référence de la classe <code>ExceptionCreationSocket</code>	28
6.8.1 Description détaillée	29
6.8.2 Documentation des fonctions membres	29
6.8.2.1 <code>getMessage()</code>	29
6.8.3 Documentation des données membres	30
6.8.3.1 message	30
6.9 Référence de la classe <code>ExceptionDessin</code>	30
6.9.1 Description détaillée	31
6.9.2 Documentation des fonctions membres	31
6.9.2.1 <code>getMessage()</code>	31
6.9.3 Documentation des données membres	31

6.9.3.1 message	31
6.10 Référence de la classe ExceptionEnvoiImpossible	31
6.10.1 Description détaillée	32
6.10.2 Documentation des fonctions membres	32
6.10.2.1 getMessage()	32
6.10.3 Documentation des données membres	33
6.10.3.1 message	33
6.11 Référence de la classe ExceptionFileCantBeOpen	33
6.11.1 Description détaillée	34
6.11.2 Documentation des fonctions membres	34
6.11.2.1 getMessage()	34
6.11.3 Documentation des données membres	34
6.11.3.1 message	34
6.12 Référence de la classe ExceptionFileCantBeRead	34
6.12.1 Description détaillée	35
6.12.2 Documentation des fonctions membres	35
6.12.2.1 getMessage()	35
6.12.3 Documentation des données membres	36
6.12.3.1 message	36
6.13 Référence de la classe ExceptionFileIsEmpty	36
6.13.1 Description détaillée	37
6.13.2 Documentation des fonctions membres	37
6.13.2.1 getMessage()	37
6.13.3 Documentation des données membres	37
6.13.3.1 message	37
6.14 Référence de la classe ExceptionForme	37
6.14.1 Description détaillée	38
6.14.2 Documentation des fonctions membres	38
6.14.2.1 getMessage()	38
6.14.2.2 what()	39
6.14.3 Documentation des fonctions amies et associées	39
6.14.3.1 operator<<	39
6.14.4 Documentation des données membres	39
6.14.4.1 message	39
6.15 Référence de la classe ExceptionFormeCharger	39
6.15.1 Description détaillée	40
6.15.2 Documentation des fonctions membres	40
6.15.2.1 getMessage()	40
6.15.3 Documentation des données membres	40
6.15.3.1 message	40
6.16 Référence de la classe ExceptionFormeComposee	41
6.16.1 Description détaillée	41

6.16.2 Documentation des fonctions membres	41
6.16.2.1 getMessage()	41
6.16.3 Documentation des données membres	42
6.16.3.1 message	42
6.17 Référence de la classe ExceptionFormeSimple	42
6.17.1 Description détaillée	43
6.17.2 Documentation des fonctions membres	43
6.17.2.1 getMessage()	43
6.17.3 Documentation des données membres	43
6.17.3.1 message	43
6.18 Référence de la classe ExceptionFormeVisiteur	43
6.18.1 Description détaillée	44
6.18.2 Documentation des fonctions membres	44
6.18.2.1 getMessage()	44
6.18.3 Documentation des données membres	44
6.18.3.1 message	44
6.19 Référence de la classe ExceptionKIsNull	45
6.19.1 Description détaillée	45
6.19.2 Documentation des fonctions membres	45
6.19.2.1 getMessage()	45
6.19.3 Documentation des données membres	46
6.19.3.1 message	46
6.20 Référence de la classe ExceptionParametragePaquet	46
6.20.1 Description détaillée	47
6.20.2 Documentation des fonctions membres	47
6.20.2.1 getMessage()	47
6.20.3 Documentation des données membres	47
6.20.3.1 message	47
6.21 Référence de la classe ExceptionPointEqualsAnotherPoint	47
6.21.1 Description détaillée	48
6.21.2 Documentation des fonctions membres	48
6.21.2.1 getMessage()	48
6.21.3 Documentation des données membres	48
6.21.3.1 message	48
6.22 Référence de la classe ExceptionRayonNegatif	49
6.22.1 Description détaillée	49
6.22.2 Documentation des fonctions membres	50
6.22.2.1 getMessage()	50
6.22.3 Documentation des données membres	50
6.22.3.1 message	50
6.23 Référence de la classe ExceptionSauvegarder	50
6.23.1 Description détaillée	51

6.23.2 Documentation des fonctions membres	51
6.23.2.1 getMessage()	51
6.23.3 Documentation des données membres	51
6.23.3.1 message	51
6.24 Référence de la classe ExceptionStringIsNotRecognizable	52
6.24.1 Description détaillée	52
6.24.2 Documentation des fonctions membres	52
6.24.2.1 getMessage()	52
6.24.3 Documentation des données membres	53
6.24.3.1 message	53
6.25 Référence de la classe ExceptionTransformationForme	53
6.25.1 Description détaillée	54
6.25.2 Documentation des fonctions membres	54
6.25.2.1 getMessage()	54
6.25.3 Documentation des données membres	54
6.25.3.1 message	54
6.26 Référence de la classe Forme	54
6.26.1 Description détaillée	56
6.26.2 Documentation des constructeurs et destructeur	56
6.26.2.1 Forme()	56
6.26.2.2 ~Forme()	57
6.26.3 Documentation des fonctions membres	57
6.26.3.1 accept()	57
6.26.3.2 charger()	57
6.26.3.3 clone()	57
6.26.3.4 getAire()	58
6.26.3.5 getCouleur()	58
6.26.3.6 getPoints()	58
6.26.3.7 homothetie()	58
6.26.3.8 operator string()	59
6.26.3.9 rotation()	59
6.26.3.10 setCouleur()	59
6.26.3.11 toString()	59
6.26.3.12 transformationEcran()	60
6.26.3.13 translation()	60
6.26.3.14 verificationKIsNotNull()	60
6.26.4 Documentation des fonctions amies et associées	61
6.26.4.1 operator<<	61
6.26.5 Documentation des données membres	61
6.26.5.1 couleur	61
6.27 Référence de la classe FormeCharger	61
6.27.1 Description détaillée	62

6.27.2 Documentation des constructeurs et destructeur	62
6.27.2.1 FormeCharger()	62
6.27.2.2 ~FormeCharger()	63
6.27.3 Documentation des fonctions membres	63
6.27.3.1 charger()	63
6.27.3.2 chargerDepuisFichier()	63
6.27.3.3 chargerDepuisFlux()	64
6.27.3.4 chargerDepuisFluxStart()	64
6.27.3.5 getOrigine()	64
6.27.4 Documentation des données membres	64
6.27.4.1 origine	64
6.27.4.2 suivant	65
6.28 Référence de la classe FormeChargerCercle	65
6.28.1 Description détaillée	66
6.28.2 Documentation des constructeurs et destructeur	66
6.28.2.1 FormeChargerCercle()	66
6.28.2.2 ~FormeChargerCercle()	66
6.28.3 Documentation des fonctions membres	67
6.28.3.1 charger()	67
6.29 Référence de la classe FormeChargerFormeComposee	67
6.29.1 Description détaillée	68
6.29.2 Documentation des constructeurs et destructeur	68
6.29.2.1 FormeChargerFormeComposee()	68
6.29.2.2 ~FormeChargerFormeComposee()	69
6.29.3 Documentation des fonctions membres	69
6.29.3.1 charger()	69
6.30 Référence de la classe FormeChargerPolygone	69
6.30.1 Description détaillée	70
6.30.2 Documentation des constructeurs et destructeur	71
6.30.2.1 FormeChargerPolygone()	71
6.30.2.2 ~FormeChargerPolygone()	71
6.30.3 Documentation des fonctions membres	71
6.30.3.1 charger()	71
6.31 Référence de la classe FormeChargerSegment	72
6.31.1 Description détaillée	73
6.31.2 Documentation des constructeurs et destructeur	73
6.31.2.1 FormeChargerSegment()	73
6.31.2.2 ~FormeChargerSegment()	73
6.31.3 Documentation des fonctions membres	73
6.31.3.1 charger()	73
6.32 Référence de la classe FormeChargerTriangle	74
6.32.1 Description détaillée	75

6.32.2 Documentation des constructeurs et destructeur	75
6.32.2.1 FormeChargerTriangle()	75
6.32.2.2 ~FormeChargerTriangle()	75
6.32.3 Documentation des fonctions membres	76
6.32.3.1 charger()	76
6.33 Référence de la classe FormeComposee	76
6.33.1 Description détaillée	79
6.33.2 Documentation des constructeurs et destructeur	80
6.33.2.1 FormeComposee() [1/3]	80
6.33.2.2 FormeComposee() [2/3]	80
6.33.2.3 FormeComposee() [3/3]	80
6.33.2.4 ~FormeComposee()	81
6.33.3 Documentation des fonctions membres	81
6.33.3.1 accept()	81
6.33.3.2 addForme()	81
6.33.3.3 addFormeAt()	82
6.33.3.4 clone()	82
6.33.3.5 copy()	82
6.33.3.6 destroy()	83
6.33.3.7 getAire()	83
6.33.3.8 getFormeAt()	83
6.33.3.9 getFormes()	83
6.33.3.10 getPoints()	84
6.33.3.11 homothetie()	84
6.33.3.12 operator+()	84
6.33.3.13 operator-()	84
6.33.3.14 operator=()	85
6.33.3.15 operator[]()	85
6.33.3.16 removeFormeAt()	85
6.33.3.17 rotation()	86
6.33.3.18 setFormes()	86
6.33.3.19 toString()	86
6.33.3.20 transformationEcran()	87
6.33.3.21 translation()	87
6.33.3.22 verificationAccesVector()	87
6.33.3.23 verificationAccesVectorAddForme()	88
6.33.4 Documentation des données membres	88
6.33.4.1 v	88
6.34 Référence de la classe FormeDessiner	88
6.34.1 Description détaillée	90
6.34.2 Documentation des constructeurs et destructeur	90
6.34.2.1 FormeDessiner()	90

6.34.2.2 ~FormeDessiner()	90
6.34.3 Documentation des fonctions membres	90
6.34.3.1 envoyerRequete()	90
6.34.3.2 preTransformationPassageEcran()	91
6.34.3.3 traiterFormePourRequete()	91
6.34.3.4 visit() [1/5]	91
6.34.3.5 visit() [2/5]	92
6.34.3.6 visit() [3/5]	92
6.34.3.7 visit() [4/5]	92
6.34.3.8 visit() [5/5]	93
6.34.4 Documentation des données membres	93
6.34.4.1 adress	93
6.34.4.2 port	93
6.34.4.3 sock	93
6.34.4.4 sockaddr	93
6.35 Référence de la classe FormeSauvegarder	94
6.35.1 Description détaillée	95
6.35.2 Documentation des constructeurs et destructeur	95
6.35.2.1 FormeSauvegarder()	95
6.35.2.2 ~FormeSauvegarder()	95
6.35.3 Documentation des fonctions membres	95
6.35.3.1 sauvegarderForme()	95
6.35.3.2 visit() [1/5]	96
6.35.3.3 visit() [2/5]	96
6.35.3.4 visit() [3/5]	96
6.35.3.5 visit() [4/5]	97
6.35.3.6 visit() [5/5]	97
6.35.4 Documentation des données membres	97
6.35.4.1 path	97
6.36 Référence de la classe FormeSimple	98
6.36.1 Description détaillée	100
6.36.2 Documentation des constructeurs et destructeur	100
6.36.2.1 FormeSimple()	100
6.36.2.2 ~FormeSimple()	100
6.36.3 Documentation des fonctions membres	101
6.36.3.1 accept()	101
6.36.3.2 clone()	101
6.36.3.3 getAire()	101
6.36.3.4 homothetie()	102
6.36.3.5 rotation()	102
6.36.3.6 setPoints()	102
6.36.3.7 transformationEcran()	103

6.36.3.8 translation()	103
6.36.3.9 verificationAccesVector()	103
6.36.3.10 verificationAccesVectorAddPoint()	104
6.36.3.11 verificationNonEgaliteVecteur2D()	104
6.36.3.12 verificationTailleVector()	104
6.37 Référence de la classe <code>FormeVisiteur</code>	105
6.37.1 Description détaillée	105
6.37.2 Documentation des constructeurs et destructeur	105
6.37.2.1 <code>~FormeVisiteur()</code>	105
6.37.3 Documentation des fonctions membres	106
6.37.3.1 <code>visit()</code> [1/5]	106
6.37.3.2 <code>visit()</code> [2/5]	106
6.37.3.3 <code>visit()</code> [3/5]	106
6.37.3.4 <code>visit()</code> [4/5]	107
6.37.3.5 <code>visit()</code> [5/5]	107
6.38 Référence de la classe <code>Matrice22</code>	107
6.38.1 Description détaillée	108
6.38.2 Documentation des constructeurs et destructeur	108
6.38.2.1 <code>Matrice22()</code> [1/2]	108
6.38.2.2 <code>Matrice22()</code> [2/2]	109
6.38.3 Documentation des fonctions membres	109
6.38.3.1 <code>creeRotation()</code>	109
6.38.3.2 <code>operator string()</code>	110
6.38.3.3 <code>operator*()</code>	110
6.38.4 Documentation des données membres	110
6.38.4.1 <code>ligneBas</code>	110
6.38.4.2 <code>ligneHaut</code>	110
6.39 Référence de la classe <code>MaWinsock</code>	110
6.39.1 Description détaillée	111
6.39.2 Documentation des constructeurs et destructeur	111
6.39.2.1 <code>MaWinsock()</code>	111
6.39.2.2 <code>~MaWinsock()</code>	111
6.39.3 Documentation des fonctions membres	112
6.39.3.1 <code>getInstance()</code>	112
6.39.4 Documentation des données membres	112
6.39.4.1 <code>instanceUnique</code>	112
6.39.4.2 <code>wsaData</code>	112
6.40 Référence de la classe <code>Polygone</code>	112
6.40.1 Description détaillée	116
6.40.2 Documentation des constructeurs et destructeur	116
6.40.2.1 <code>Polygone()</code> [1/2]	116
6.40.2.2 <code>Polygone()</code> [2/2]	116

6.40.2.3 ~Polygone()	117
6.40.3 Documentation des fonctions membres	117
6.40.3.1 accept()	117
6.40.3.2 addPoint()	117
6.40.3.3 addPointAt()	118
6.40.3.4 clone()	118
6.40.3.5 getAire()	118
6.40.3.6 getPointAt()	119
6.40.3.7 getPoints()	119
6.40.3.8 modifyPoint()	119
6.40.3.9 modifyPointAt()	120
6.40.3.10 operator+()	120
6.40.3.11 operator-()	120
6.40.3.12 operator[]()	121
6.40.3.13 removePoint()	121
6.40.3.14 removePointAt()	121
6.40.3.15 setPoints()	122
6.40.3.16 switchPoint()	122
6.40.3.17 switchPointAt()	122
6.40.3.18 toString()	123
6.40.3.19 verificationParametreVector()	123
6.40.3.20 verificationVectorEtVecteur2D()	123
6.40.4 Documentation des données membres	124
6.40.4.1 v	124
6.41 Référence de la classe Segment	124
6.41.1 Description détaillée	127
6.41.2 Documentation des constructeurs et destructeur	127
6.41.2.1 Segment() [1/2]	127
6.41.2.2 Segment() [2/2]	128
6.41.2.3 ~Segment()	128
6.41.3 Documentation des fonctions membres	128
6.41.3.1 accept()	128
6.41.3.2 clone()	129
6.41.3.3 getAire()	129
6.41.3.4 getp1()	129
6.41.3.5 getp2()	129
6.41.3.6 getPoints()	129
6.41.3.7 setp1() [1/2]	130
6.41.3.8 setp1() [2/2]	130
6.41.3.9 setp2() [1/2]	130
6.41.3.10 setp2() [2/2]	131
6.41.3.11 setPoints()	131

6.41.3.12 toString()	131
6.41.4 Documentation des données membres	132
6.41.4.1 p	132
6.42 Référence de la classe Triangle	132
6.42.1 Description détaillée	135
6.42.2 Documentation des constructeurs et destructeur	135
6.42.2.1 Triangle() [1/2]	135
6.42.2.2 Triangle() [2/2]	136
6.42.2.3 ~Triangle()	136
6.42.3 Documentation des fonctions membres	136
6.42.3.1 accept()	136
6.42.3.2 clone()	137
6.42.3.3 getAire()	137
6.42.3.4 getp1()	137
6.42.3.5 getp2()	137
6.42.3.6 getp3()	137
6.42.3.7 getPoints()	138
6.42.3.8 setp1() [1/2]	138
6.42.3.9 setp1() [2/2]	138
6.42.3.10 setp2() [1/2]	139
6.42.3.11 setp2() [2/2]	139
6.42.3.12 setp3() [1/2]	139
6.42.3.13 setp3() [2/2]	140
6.42.3.14 setPoints()	140
6.42.3.15 toString()	140
6.42.4 Documentation des données membres	141
6.42.4.1 p	141
6.43 Référence de la classe Vecteur2D	141
6.43.1 Description détaillée	142
6.43.2 Documentation des constructeurs et destructeur	142
6.43.2.1 Vecteur2D()	142
6.43.3 Documentation des fonctions membres	143
6.43.3.1 norme()	143
6.43.3.2 norme2()	143
6.43.3.3 operator string()	143
6.43.3.4 operator*() [1/2]	143
6.43.3.5 operator*() [2/2]	144
6.43.3.6 operator+()	144
6.43.3.7 operator+=()	144
6.43.3.8 operator-() [1/2]	145
6.43.3.9 operator-() [2/2]	145
6.43.3.10 operator==()	145

6.43.3.11 operator" ()	145
6.43.4 Documentation des données membres	146
6.43.4.1 x	146
6.43.4.2 y	146
7 Documentation des fichiers	147
7.1 Référence du fichier Cercle.h	147
7.1.1 Documentation des macros	147
7.1.1.1 M_PI	147
7.2 Cercle.h	148
7.3 Référence du fichier Forme.h	148
7.3.1 Documentation du type de l'énumération	149
7.3.1.1 Couleur	149
7.3.2 Documentation des fonctions	150
7.3.2.1 couleurToString()	150
7.3.2.2 stringToCouleur()	150
7.4 Forme.h	150
7.5 Référence du fichier FormeCharger.h	152
7.6 FormeCharger.h	152
7.7 Référence du fichier FormeChargerCercle.h	153
7.8 FormeChargerCercle.h	153
7.9 Référence du fichier FormeChargerFormeComposee.h	153
7.10 FormeChargerFormeComposee.h	154
7.11 Référence du fichier FormeChargerPolygone.h	154
7.12 FormeChargerPolygone.h	154
7.13 Référence du fichier FormeChargerSegment.h	154
7.14 FormeChargerSegment.h	155
7.15 Référence du fichier FormeChargerTriangle.h	155
7.16 FormeChargerTriangle.h	155
7.17 Référence du fichier FormeComposee.h	155
7.18 FormeComposee.h	156
7.19 Référence du fichier FormeDessiner.h	157
7.20 FormeDessiner.h	157
7.21 Référence du fichier FormeSauvegarder.h	158
7.22 FormeSauvegarder.h	159
7.23 Référence du fichier FormeSimple.h	159
7.24 FormeSimple.h	160
7.25 Référence du fichier FormeVisiteur.h	160
7.26 FormeVisiteur.h	161
7.27 Référence du fichier Matrice22.h	161
7.27.1 Documentation des fonctions	162
7.27.1.1 operator<<()	162

7.28 Matrice22.h	162
7.29 Référence du fichier MaWinsock.h	162
7.30 MaWinsock.h	163
7.31 Référence du fichier Polygone.h	163
7.32 Polygone.h	163
7.33 Référence du fichier Segment.h	164
7.34 Segment.h	164
7.35 Référence du fichier Triangle.h	164
7.36 Triangle.h	165
7.37 Référence du fichier Vecteur2D.h	165
7.37.1 Documentation des fonctions	166
7.37.1.1 operator<<()	166
7.38 Vecteur2D.h	166
7.39 Référence du fichier Cercle.cpp	167
7.40 Référence du fichier Forme.cpp	167
7.40.1 Documentation des fonctions	167
7.40.1.1 operator<<()	167
7.41 Référence du fichier FormeCharger.cpp	167
7.42 Référence du fichier FormeChargerCercle.cpp	167
7.43 Référence du fichier FormeChargerFormeComposee.cpp	168
7.44 Référence du fichier FormeChargerPolygone.cpp	168
7.45 Référence du fichier FormeChargerSegment.cpp	168
7.46 Référence du fichier FormeChargerTriangle.cpp	168
7.47 Référence du fichier FormeComposee.cpp	168
7.48 Référence du fichier FormeDessiner.cpp	168
7.49 Référence du fichier FormeSauvegarder.cpp	168
7.50 Référence du fichier FormeSimple.cpp	169
7.51 Référence du fichier FormeVisiteur.cpp	169
7.52 Référence du fichier Main.cpp	169
7.52.1 Documentation des macros	169
7.52.1.1 M_PI_2	169
7.52.1.2 M_PI_4	169
7.52.2 Documentation des fonctions	170
7.52.2.1 main()	170
7.53 Référence du fichier Matrice22.cpp	170
7.54 Référence du fichier MaWinsock.cpp	170
7.55 Référence du fichier Polygone.cpp	170
7.56 Référence du fichier Segment.cpp	170
7.57 Référence du fichier Triangle.cpp	170
7.58 Référence du fichier Vecteur2D.cpp	170

Chapitre 1

Hiérarchie de répertoires

1.1 Répertoires

include	9
Cercle.h	147
Forme.h	148
FormeCharger.h	152
FormeChargerCercle.h	153
FormeChargerFormeComposee.h	153
FormeChargerPolygone.h	154
FormeChargerSegment.h	154
FormeChargerTriangle.h	155
FormeComposee.h	155
FormeDessiner.h	157
FormeSauvegarder.h	158
FormeSimple.h	159
FormeVisiteur.h	160
Matrice22.h	161
MaWinsock.h	162
Polygone.h	163
Segment.h	164
Triangle.h	164
Vecteur2D.h	165
src	10
Cercle.cpp	167
Forme.cpp	167
FormeCharger.cpp	167
FormeChargerCercle.cpp	167
FormeChargerFormeComposee.cpp	168
FormeChargerPolygone.cpp	168
FormeChargerSegment.cpp	168
FormeChargerTriangle.cpp	168
FormeComposee.cpp	168
FormeDessiner.cpp	168
FormeSauvegarder.cpp	168
FormeSimple.cpp	169
FormeVisiteur.cpp	169
Main.cpp	169
Matrice22.cpp	170
MaWinsock.cpp	170
Polygone.cpp	170

Segment.cpp	170
Triangle.cpp	170
Vecteur2D.cpp	170

Chapitre 2

Index hiérarchique

2.1 Hiérarchie des classes

Cette liste d'héritage est classée approximativement par ordre alphabétique :

exception	
ExceptionForme	37
ExceptionCouleurInconnue	27
ExceptionFormeCharger	39
ExceptionFileCantBeRead	34
ExceptionFileIsEmpty	36
ExceptionStringIsNotRecognizable	52
ExceptionFormeComposee	41
ExceptionArrayOutOfBoundsFC	22
ExceptionFormeSimple	42
ExceptionArrayIsTooShort	19
ExceptionArrayOutOfBounds	21
ExceptionCercle	24
ExceptionRayonNegatif	49
ExceptionPointEqualsAnotherPoint	47
ExceptionFormeVisiteur	43
ExceptionDessin	30
ExceptionConnexionImpossible	25
ExceptionCreationSocket	28
ExceptionEnvoiImpossible	31
ExceptionParametragePaquet	46
ExceptionTransformationForme	53
ExceptionSauvegarder	50
ExceptionFileCantBeOpen	33
ExceptionKIsNull	45
Forme	54
FormeComposee	76
FormeSimple	98
Cercle	11
Polygone	112
Segment	124
Triangle	132
FormeCharger	61

FormeChargerCercle	65
FormeChargerFormeComposee	67
FormeChargerPolygone	69
FormeChargerSegment	72
FormeChargerTriangle	74
FormeVisiteur	105
FormeDessiner	88
FormeSauvegarder	94
Matrice22	107
MaWinsock	110
Vecteur2D	141

Chapitre 3

Index des classes

3.1 Liste des classes

Liste des classes, structures, unions et interfaces avec une brève description :

Cercle	Permet de gérer un cercle	11
ExceptionArrayIsTooShort	Permet de gérer l'exception lorsqu'on essaye d'appliquer un ensemble de points pour une forme tandis que cet ensemble n'est pas de bonne taille	19
ExceptionArrayOutOfBounds	Permet de gérer l'exception en cas de dépassement de case d'un tableau	21
ExceptionArrayOutOfBoundsFC	Permet de gérer l'exception en cas de dépassement de case d'un tableau	22
ExceptionCercle	Permet de gérer l'ensemble des exceptions liées aux cercles	24
ExceptionConnexionImpossible	Permet de gérer l'exception lors de l'impossibilité de se connecter au serveur	25
ExceptionCouleurInconnue	Permet de gérer l'erreur lors de la non-reconnaissance d'une couleur	27
ExceptionCreationSocket	Permet de gérer l'exception lors de l'impossibilité de créer le socket	28
ExceptionDessin	Permet de gérer l'ensemble des exceptions liées aux visiteurs de forme dessin	30
ExceptionEnvoiImpossible	Permet de gérer l'exception lors de l'impossibilité d'envoyer un message	31
ExceptionFileCantBeOpen	Permet de gérer l'exception liée à l'impossibilité de créer un fichier	33
ExceptionFileCantBeRead	Permet de gérer l'exception liée à l'impossible de lire un fichier	34
ExceptionFileIsEmpty	Permet de gérer l'exception liée à un fichier vide	36
ExceptionForme	Permet de gérer l'ensemble des exceptions liées aux formes	37
ExceptionFormeCharger	Permet de gérer l'ensemble des exceptions liées au chargement d'une forme	39
ExceptionFormeComposee	Permet de gérer l'ensemble des exceptions liées aux formes composées	41
ExceptionFormeSimple	Permet de gérer l'ensemble des exceptions liées aux formes simples	42

ExceptionFormeVisiteur	Permet de gérer l'ensemble des exceptions liées aux visiteurs de formes	43
ExceptionKIsNull	Permet de gérer l'erreur de l'utilisation d'un k nulle pour la rotation	45
ExceptionParametragePaquet	Permet de gérer l'exception lors de l'impossibilité de paramétrer l'envoi des messages	46
ExceptionPointEqualsAnotherPoint	Permet de gérer l'exception lorsque deux points sont identiques dans une forme simple	47
ExceptionRayonNegatif	Permet de gérer l'exception lorsque le rayon est negatif	49
ExceptionSauvegarder	Permet de gérer l'ensemble des exceptions liées aux visiteurs de formes sauvegarde	50
ExceptionStringIsNotRecognizable	Permet de gérer l'exception dans le cas d'une forme non reconnue	52
ExceptionTransformationForme	Permet de gérer l'exception lors de l'impossibilité de transformer la forme	53
Forme	Permet de créer une forme	54
FormeCharger	Permet de gérer le chargement d'une forme	61
FormeChargerCercle	Permet le chargement d'un cercle	65
FormeChargerFormeComposee	Permet le chargement d'une forme composée	67
FormeChargerPolygone	Permet le chargement d'un polygone	69
FormeChargerSegment	Permet le chargement d'un segment	72
FormeChargerTriangle	Permet le chargement d'un triangle	74
FormeComposee	Permet de gérer une forme composée	76
FormeDessiner	Permet l'implémentation du pattern design visiteur pour les formes dans le cadre d'un dessin	88
FormeSauvegarder	Permet l'implémentation du pattern design visiteur pour les formes dans le cadre d'une sauvegarde	94
FormeSimple	Permet de gérer une forme simple	98
FormeVisiteur	Permet l'implémentation du pattern design visiteur pour les formes	105
Matrice22	Permet de gérer des matrices de taille 2 * 2	107
MaWinsock	Permet la gestion de la socket serveur	110
Polygone	Permet de gérer du polygone	112
Segment	Permet de créer un segment	124
Triangle	Permet de gérer un triangle	132
Vecteur2D	Permet de gérer un vecteur	141

Chapitre 4

Index des fichiers

4.1 Liste des fichiers

Liste de tous les fichiers avec une brève description :

Cercle.h	147
Forme.h	148
FormeCharger.h	152
FormeChargerCercle.h	153
FormeChargerFormeComposee.h	153
FormeChargerPolygone.h	154
FormeChargerSegment.h	154
FormeChargerTriangle.h	155
FormeComposee.h	155
FormeDessiner.h	157
FormeSauvegarder.h	158
FormeSimple.h	159
FormeVisiteur.h	160
Matrice2D.h	161
MaWinsock.h	162
Polygone.h	163
Segment.h	164
Triangle.h	164
Vecteur2D.h	165
Cercle.cpp	167
Forme.cpp	167
FormeCharger.cpp	167
FormeChargerCercle.cpp	167
FormeChargerFormeComposee.cpp	168
FormeChargerPolygone.cpp	168
FormeChargerSegment.cpp	168
FormeChargerTriangle.cpp	168
FormeComposee.cpp	168
FormeDessiner.cpp	168
FormeSauvegarder.cpp	168
FormeSimple.cpp	169
FormeVisiteur.cpp	169
Main.cpp	169
Matrice2D.cpp	170
MaWinsock.cpp	170

Polygone.cpp	170
Segment.cpp	170
Triangle.cpp	170
Vecteur2D.cpp	170

Chapitre 5

Documentation des répertoires

5.1 Répertoire de référence de include

Fichiers

- fichier [Cercle.h](#)
- fichier [Forme.h](#)
- fichier [FormeCharger.h](#)
- fichier [FormeChargerCercle.h](#)
- fichier [FormeChargerFormeComposee.h](#)
- fichier [FormeChargerPolygone.h](#)
- fichier [FormeChargerSegment.h](#)
- fichier [FormeChargerTriangle.h](#)
- fichier [FormeComposee.h](#)
- fichier [FormeDessiner.h](#)
- fichier [FormeSauvegarder.h](#)
- fichier [FormeSimple.h](#)
- fichier [FormeVisiteur.h](#)
- fichier [Matrice22.h](#)
- fichier [MaWinsock.h](#)
- fichier [Polygone.h](#)
- fichier [Segment.h](#)
- fichier [Triangle.h](#)
- fichier [Vecteur2D.h](#)

5.2 Répertoire de référence de src

Fichiers

- fichier [Cercle.cpp](#)
- fichier [Forme.cpp](#)
- fichier [FormeCharger.cpp](#)
- fichier [FormeChargerCercle.cpp](#)
- fichier [FormeChargerFormeComposee.cpp](#)
- fichier [FormeChargerPolygone.cpp](#)
- fichier [FormeChargerSegment.cpp](#)
- fichier [FormeChargerTriangle.cpp](#)
- fichier [FormeComposee.cpp](#)
- fichier [FormeDessiner.cpp](#)
- fichier [FormeSauvegarder.cpp](#)
- fichier [FormeSimple.cpp](#)
- fichier [FormeVisiteur.cpp](#)
- fichier [Main.cpp](#)
- fichier [Matrice22.cpp](#)
- fichier [MaWinsock.cpp](#)
- fichier [Polygone.cpp](#)
- fichier [Segment.cpp](#)
- fichier [Triangle.cpp](#)
- fichier [Vecteur2D.cpp](#)

Chapitre 6

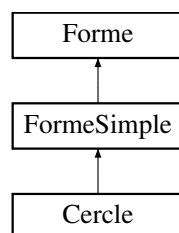
Documentation des classes

6.1 Référence de la classe Cercle

Permet de gérer un cercle.

```
#include <Cercle.h>
```

Graphe d'héritage de Cercle:



Fonctions membres publiques

- `Cercle` (const `Vecteur2D` &c, const double &r=1, const `Couleur` &couleur=`Couleur::BLACK`)

Construit un cercle à partir d'une couleur, d'un centre c et d'un rayon r .

- `Cercle` (const double &xc=0, const double &yc=0, const double &r=1, const `Couleur` &couleur=`Couleur::BLACK`)

Construit un cercle à partir d'une couleur, de coordonnées x_c et y_c désignant le centre et r le rayon.

- `Cercle * clone` () const override

Retourne un clone du cercle.

- `~Cercle` () override

Permet la destruction du cercle.

- `Vecteur2D getc` () const

Retourne le centre du cercle.

- double `getr ()` const
Retourne le rayon du cercle.
- `Cercle & setc` (const `Vecteur2D &c`)
Change le centre du cercle par un autre centre c .
- `Cercle & setc` (const double `&xc=0`, const double `&yc=0`)
Change le centre du cercle par les coordonnées x_c et y_c .
- `Cercle & setr` (const double `&r`)
Change le rayon du cercle par un autre rayon r .
- `vector< Vecteur2D > getPoints ()` const override
Permet d'obtenir l'ensemble des points du cercle.
- `Cercle & setPoints` (const `vector< Vecteur2D > &v`) override
Permet de modifier l'ensemble des points du cercle.
- double `getAire ()` const override
Retourne l'aire du cercle.
- string `toString ()` const override
Permet d'obtenir une chaîne de caractère indiquant les spécificités du cercle.
- `Cercle & accept` (`FormeVisiteur &visiteur`) override
Permet l'introduction de méthode via le design pattern visitor.

Fonctions membres publiques héritées de `FormeSimple`

- `FormeSimple` (const `Couleur &couleur=Couleur::BLACK`)
Construit une forme simple à partir d'une couleur.
- `~FormeSimple ()` override
Permet la destruction de la forme simple.
- `FormeSimple & translation` (const `Vecteur2D &p`) override
Permet de réaliser une translation de la forme simple.
- `FormeSimple & homothetie` (const `Vecteur2D &p`, const double `&k`) override
Permet de réaliser une homothétie de la forme simple.
- `FormeSimple & rotation` (const `Vecteur2D &p`, const double `&r`) override
Permet de réaliser une rotation de la forme simple.
- `FormeSimple & transformationEcran` (const `Matrice22 &m`, const `Vecteur2D &ab`) override

Permet de réaliser une transformation de la forme simple pour un affichage sur écran.

Fonctions membres publiques hérités de **Forme**

- **Forme** (const **Couleur** &couleur=**Couleur::BLACK**)

Construit une forme à partir d'une couleur.

- virtual ~**Forme** ()

Permet la destruction de la forme.

- **Couleur** get**Couleur** () const

Retourne la couleur de la forme courante.

- **Forme** & set**Couleur** (const **Couleur** &couleur)

Permet de changer la couleur de la forme.

- operator string () const

Permet d'obtenir une chaîne de caractère indiquant les spécificités de la forme.

Fonctions membres privées statiques

- static void verification**Rayon** (const double &r)

Verifie si le rayon est bien supérieur à 0.

Attributs privés

- **Vecteur2D** c

Le centre du cercle.

- double r

Le rayon du cercle.

Membres hérités additionnels

Fonctions membres publiques statiques hérités de **Forme**

- static **Forme** * charger (const string &path)

Permet de charger une forme à partir d'un fichier local.

Fonctions membres protégées statiques hérités de **FormeSimple**

- static void **verificationNonEgaliteVecteur2D** (const **Vecteur2D** &a, const **Vecteur2D** &b)

Verifie si un vecteur a n'est pas égal à un vecteur b .

- static void **verificationTailleVector** (const vector< **Vecteur2D** > &v, int i)

Verifie si un entier i correspond à la taille d'un vector v .

- static void **verificationAccesVector** (const vector< **Vecteur2D** > &v, int i)

Verifie si un entier i permet l'accès à un élément d'un vecteur v .

- static void **verificationAccesVectorAddPoint** (const vector< **Vecteur2D** > &v, int i)

Verifie si un entier i permet l'accès à un élément d'un vecteur v lors de l'ajout d'un point.

Fonctions membres protégées statiques hérités de **Forme**

- static void **verificationKIsNotNull** (const double &k)

Verifie si la constante k pour l'homothétie n'est pas nulle.

6.1.1 Description détaillée

Permet de gérer un cercle.

6.1.2 Documentation des constructeurs et destructeur

6.1.2.1 Cercle() [1/2]

```
Cercle::Cercle (
    const Vecteur2D & c,
    const double & r = 1,
    const Couleur & couleur = Couleur::BLACK) [explicit]
```

Construit un cercle à partir d'une couleur, d'un centre c et d'un rayon r .

Paramètres

in	c	Le centre du cercle
in	r	Le rayon du cercle
in	couleur	La couleur souhaitée (à défaut noir)

Renvoie

Un cercle

r Le rayon du cercle

6.1.2.2 Cercle() [2/2]

```
Cercle::Cercle (
    const double & xc = 0,
    const double & yc = 0,
    const double & r = 1,
    const Couleur & couleur = Couleur::BLACK) [explicit]
```

Construit un cercle à partir d'une couleur, de coordonnées `xc` et `yc` désignant le centre et `r` le rayon.

Paramètres

in	<code>xc</code>	Coordonnée x du centre du cercle (à défaut 0)
in	<code>yc</code>	Coordonnée y du centre du cercle (à défaut 0)
in	<code>r</code>	Le rayon du cercle (à défaut 1)
in	<code>couleur</code>	La couleur souhaitée (à défaut noir)

Renvoie

Un cercle

`r` doit être supérieur à 0

6.1.2.3 ~Cercle()

```
Cercle::~~Cercle () [override], [default]
```

Permet la destruction du cercle.

6.1.3 Documentation des fonctions membres

6.1.3.1 accept()

```
Cercle & Cercle::accept (
    FormeVisiteur & visiteur) [override], [virtual]
```

Permet l'introduction de méthode via le design pattern visitor.

Paramètres

in	<code>visiteur</code>	Un objet répondant au pattern visitor
----	-----------------------	---------------------------------------

Renvoie

(*this) L'objet implicite

Implémente [FormeSimple](#).

6.1.3.2 clone()

```
Cercle * Cercle::clone () const [override], [virtual]
```

Retourne un clone du cercle.

Renvoie

Un clone du cercle

Implémente [FormeSimple](#).

6.1.3.3 getAire()

```
double Cercle::getAire () const [override], [virtual]
```

Retourne l'aire du cercle.

Renvoie

L'aire du cercle

Implémente [FormeSimple](#).

6.1.3.4 getc()

```
Vecteur2D Cercle::getc () const
```

Retourne le centre du cercle.

Renvoie

c Le centre du cercle

6.1.3.5 getPoints()

```
vector< Vecteur2D > Cercle::getPoints () const [override], [virtual]
```

Permet d'obtenir l'ensemble des points du cercle.

Renvoie

L'ensemble des points

Implémente [Forme](#).

6.1.3.6 getr()

```
double Cercle::getr () const
```

Retourne le rayon du cercle.

Renvoie

`r` Le rayon du cercle

6.1.3.7 setc() [1/2]

```
Cercle & Cercle::setc (
    const double & xc = 0,
    const double & yc = 0)
```

Change le centre du cercle par les coordonnées `xc` et `yc`.

Paramètres

in	<code>xc</code>	Coordonnée x du nouveau centre du cercle (à défaut 0)
in	<code>yc</code>	Coordonnée y du nouveau centre du cercle (à défaut 0)

Renvoie

`*this` L'objet implicite

`xc` et `xr` ne peuvent pas être égaux en même temps que `yc` et `yr`

6.1.3.8 setc() [2/2]

```
Cercle & Cercle::setc (
    const Vecteur2D & c)
```

Change le centre du cercle par un autre centre `c`.

Paramètres

in	<code>c</code>	Le nouveau centre du cercle
----	----------------	-----------------------------

Renvoie

`*this` L'objet implicite

`r` doit être supérieur à 0

6.1.3.9 setPoints()

```
Cercle & Cercle::setPoints (
    const vector< Vecteur2D > & v)  [override], [virtual]
```

Permet de modifier l'ensemble des points du cercle.

Paramètres

in	v	Un vecteur de points
----	---	----------------------

Renvoie

(*this) L'objet implicite

Implémente [FormeSimple](#).

6.1.3.10 setr()

```
Cercle & Cercle::setr (
    const double & r)
```

Change le rayon du cercle par un autre rayon r.

Paramètres

in	r	Le nouveau rayon du cercle
----	---	----------------------------

Renvoie

*this L'objet implicite

r doit être supérieur à 0

6.1.3.11 toString()

```
string Cercle::toString () const  [override], [virtual]
```

Permet d'obtenir une chaîne de caractère indiquant les spécificités du cercle.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [Forme](#).

6.1.3.12 verificationRayon()

```
void Cercle::verificationRayon (
    const double & r) [static], [private]
```

Verifie si le rayon est bien supérieur à 0.

Renvoie

void

6.1.4 Documentation des données membres

6.1.4.1 c

```
Vecteur2D Cercle::c [private]
```

Le centre du cercle.

6.1.4.2 r

```
double Cercle::r [private]
```

Le rayon du cercle.

La documentation de cette classe a été générée à partir des fichiers suivants :

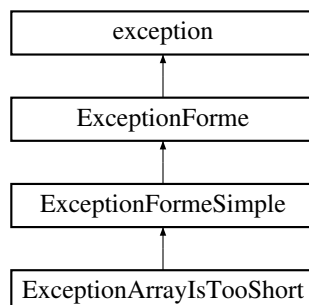
- [Cercle.h](#)
- [Cercle.cpp](#)

6.2 Référence de la classe ExceptionArrayIsTooShort

Permet de gérer l'exception lorsqu'on essaye d'appliquer un ensemble de points pour une forme tandis que cet ensemble n'est pas de bonne taille.

```
#include <FormeSimple.h>
```

Grappe d'héritage de ExceptionArrayIsTooShort:



Fonctions membres publiques

- string `getMessage ()` const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de `ExceptionFormeSimple`

- string `getMessage ()` const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de `ExceptionForme`

- const char * `what ()` const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

- string `message` = " Le tableau ne possede pas suffisamment de points pour couvrir la forme"

Le message lié à l'exception.

6.2.1 Description détaillée

Permet de gérer l'exception lorsqu'on essaye d'appliquer un ensemble de points pour une forme tandis que cet ensemble n'est pas de bonne taille.

6.2.2 Documentation des fonctions membres

6.2.2.1 `getMessage()`

```
string ExceptionArrayIsTooShort::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de `ExceptionForme`.

6.2.3 Documentation des données membres

6.2.3.1 message

```
string ExceptionArrayIsTooShort::message = " Le tableau ne possede pas suffisamment de points  
pour couvrir la forme" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

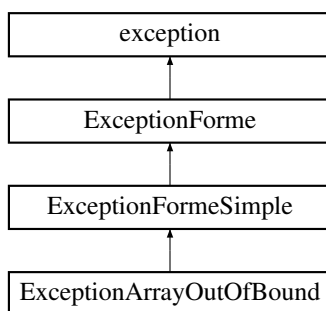
— [FormeSimple.h](#)

6.3 Référence de la classe `ExceptionArrayOutOfBounds`

Permet de gérer l'exception en cas de dépassement de case d'un tableau.

```
#include <FormeSimple.h>
```

Grappe d'héritage de `ExceptionArrayOutOfBounds`:



Fonctions membres publiques

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionFormeSimple](#)

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionForme](#)

— const char * [what](#) () const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— `string message = " La case du tableau n'existe pas"`

Le message lié à l'exception.

6.3.1 Description détaillée

Permet de gérer l'exception en cas de dépassement de case d'un tableau.

6.3.2 Documentation des fonctions membres

6.3.2.1 getMessage()

```
string ExceptionArrayOutOfBound::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionForme](#).

6.3.3 Documentation des données membres

6.3.3.1 message

```
string ExceptionArrayOutOfBound::message = " La case du tableau n'existe pas" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

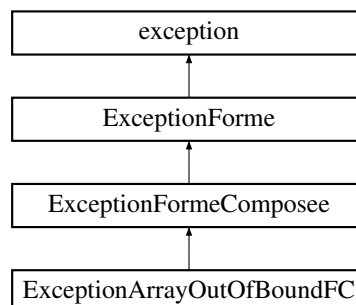
— [FormeSimple.h](#)

6.4 Référence de la classe ExceptionArrayOutOfBoundFC

Permet de gérer l'exception en cas de dépassement de case d'un tableau.

```
#include <FormeComposee.h>
```

Graphe d'héritage de ExceptionArrayOutOfBoundFC:



Fonctions membres publiques

- `string getMessage () const` override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de `ExceptionFormeComposee`

- `string getMessage () const` override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de `ExceptionForme`

- `const char * what () const` noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

- `string message = " La case du tableau n'existe pas"`

Le message lié à l'exception.

6.4.1 Description détaillée

Permet de gérer l'exception en cas de dépassement de case d'un tableau.

6.4.2 Documentation des fonctions membres

6.4.2.1 `getMessage()`

```
string ExceptionArrayOutOfBoundFC::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de `ExceptionForme`.

6.4.3 Documentation des données membres

6.4.3.1 message

```
string ExceptionArrayOutOfBoundFC::message = " La case du tableau n'existe pas" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

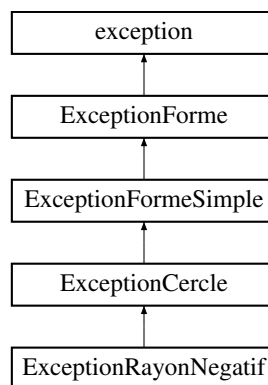
— [FormeComposee.h](#)

6.5 Référence de la classe ExceptionCercle

Permet de gérer l'ensemble des exceptions liées aux cercles.

```
#include <Cercle.h>
```

Graphe d'héritage de ExceptionCercle:



Fonctions membres publiques

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionFormeSimple](#)

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionForme](#)

— const char * [what](#) () const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— string `message` = " ExceptionCercle -"

Le message lié à l'exception.

6.5.1 Description détaillée

Permet de gérer l'ensemble des exceptions liées aux cercles.

6.5.2 Documentation des fonctions membres

6.5.2.1 getMessage()

```
string ExceptionCercle::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionForme](#).

Réimplémentée dans [ExceptionRayonNegatif](#).

6.5.3 Documentation des données membres

6.5.3.1 message

```
string ExceptionCercle::message = " ExceptionCercle -" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

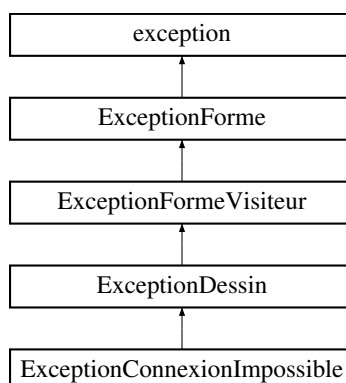
— [Cercle.h](#)

6.6 Référence de la classe ExceptionConnexionImpossible

Permet de gérer l'exception lors de l'impossibilité de se connecter au serveur.

```
#include <FormeDessiner.h>
```

Graphe d'héritage de ExceptionConnexionImpossible:



Fonctions membres publiques

- string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionDessin](#)

- string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionFormeVisiteur](#)

- string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionForme](#)

- const char * [what](#) () const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

- string [message](#) = " Il est impossible de se connecter au serveur"

Le message lié à l'exception.

6.6.1 Description détaillée

Permet de gérer l'exception lors de l'impossibilité de se connecter au serveur.

6.6.2 Documentation des fonctions membres

6.6.2.1 [getMessage\(\)](#)

```
string ExceptionConnexionImpossible::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionForme](#).

6.6.3 Documentation des données membres

6.6.3.1 message

```
string ExceptionConnexionImpossible::message = " Il est impossible de se connecter au serveur"  
[private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

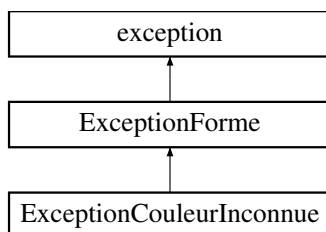
— [FormeDessiner.h](#)

6.7 Référence de la classe ExceptionCouleurInconnue

Permet de gérer l'erreur lors de la non-reconnaissance d'une couleur.

```
#include <Forme.h>
```

Graphe d'héritage de ExceptionCouleurInconnue:



Fonctions membres publiques

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionForme](#)

— const char * [what](#) () const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— string [message](#) = " La couleur est inconnue"

Le message lié à l'exception.

6.7.1 Description détaillée

Permet de gérer l'erreur lors de la non-reconnaissance d'une couleur.

6.7.2 Documentation des fonctions membres

6.7.2.1 getMessage()

```
string ExceptionCouleurInconnue::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionForme](#).

6.7.3 Documentation des données membres

6.7.3.1 message

```
string ExceptionCouleurInconnue::message = " La couleur est inconnue" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

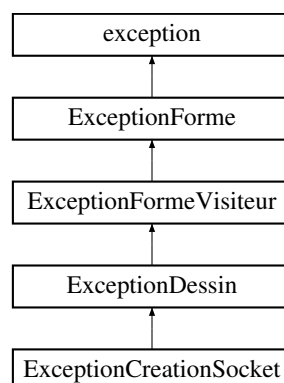
— [Forme.h](#)

6.8 Référence de la classe ExceptionCreationSocket

Permet de gérer l'exception lors de l'impossibilité de créer le socket.

```
#include <FormeDessiner.h>
```

Graphe d'héritage de ExceptionCreationSocket:



Fonctions membres publiques

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionDessin](#)

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionFormeVisiteur](#)

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionForme](#)

— const char * [what](#) () const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— string [message](#) = " Il est impossible de creer la socket"

Le message lié à l'exception.

6.8.1 Description détaillée

Permet de gérer l'exception lors de l'impossibilité de créer le socket.

6.8.2 Documentation des fonctions membres

6.8.2.1 [getMessage\(\)](#)

```
string ExceptionCreationSocket::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionForme](#).

6.8.3 Documentation des données membres

6.8.3.1 message

```
string ExceptionCreationSocket::message = " Il est impossible de creer la socket" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

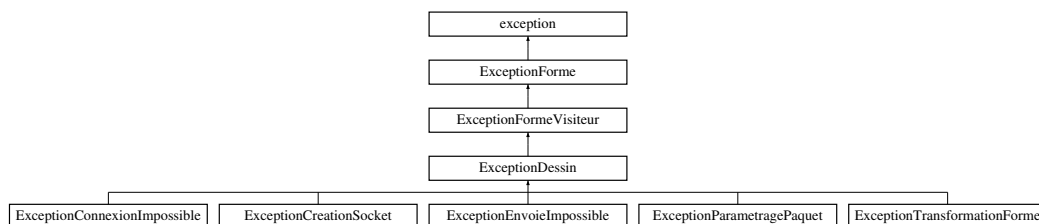
— [FormeDessiner.h](#)

6.9 Référence de la classe ExceptionDessin

Permet de gérer l'ensemble des exceptions liées aux visiteurs de forme dessin.

```
#include <FormeDessiner.h>
```

Graphe d'héritage de ExceptionDessin:



Fonctions membres publiques

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionFormeVisiteur](#)

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionForme](#)

— const char * [what](#) () const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— string `message` = " ExceptionDessin -"

Le message lié à l'exception.

6.9.1 Description détaillée

Permet de gérer l'ensemble des exceptions liées aux visiteurs de forme dessin.

6.9.2 Documentation des fonctions membres

6.9.2.1 `getMessage()`

```
string ExceptionDessin::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionForme](#).

Réimplémentée dans [ExceptionEnvoiImpossible](#), [ExceptionParametragePaquet](#), et [ExceptionTransformationForme](#).

6.9.3 Documentation des données membres

6.9.3.1 `message`

```
string ExceptionDessin::message = " ExceptionDessin -" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

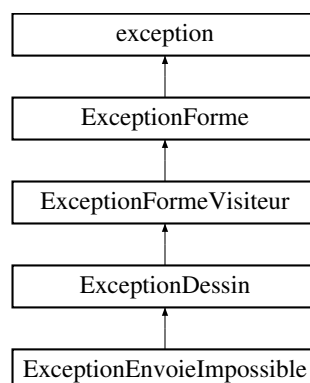
— [FormeDessiner.h](#)

6.10 Référence de la classe `ExceptionEnvoiImpossible`

Permet de gérer l'exception lors de l'impossibilité d'envoyer un message.

```
#include <FormeDessiner.h>
```

Graphe d'héritage de `ExceptionEnvoiImpossible`:



Fonctions membres publiques

- string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionFormeVisiteur](#)

- string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionForme](#)

- const char * [what](#) () const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

- string [message](#) = " Il est impossible d'envoyer un message"

Le message lié à l'exception.

6.10.1 Description détaillée

Permet de gérer l'exception lors de l'impossibilité d'envoyer un message.

6.10.2 Documentation des fonctions membres

6.10.2.1 [getMessage\(\)](#)

```
string ExceptionEnvoieImpossible::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionDessin](#).

6.10.3 Documentation des données membres

6.10.3.1 message

```
string ExceptionEnvoieImpossible::message = " Il est impossible d'envoyer un message" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

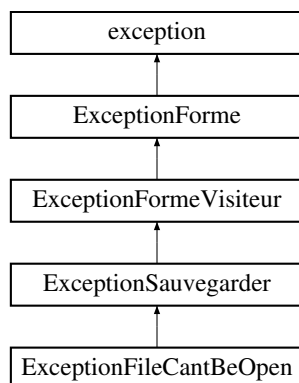
— [FormeDessiner.h](#)

6.11 Référence de la classe ExceptionFileCantBeOpen

Permet de gérer l'exception liée à l'impossibilité de créer un fichier.

```
#include <FormeSauvegarder.h>
```

Graphe d'héritage de ExceptionFileCantBeOpen:



Fonctions membres publiques

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionSauvegarder](#)

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionForme](#)

— const char * [what](#) () const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— string `message` = " Le fichier ne peut pas etre cree"

Le message lié à l'exception.

6.11.1 Description détaillée

Permet de gérer l'exception liée à l'impossibilité de créer un fichier.

6.11.2 Documentation des fonctions membres

6.11.2.1 getMessage()

```
string ExceptionFileCantBeOpen::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionForme](#).

6.11.3 Documentation des données membres

6.11.3.1 message

```
string ExceptionFileCantBeOpen::message = " Le fichier ne peut pas etre cree" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

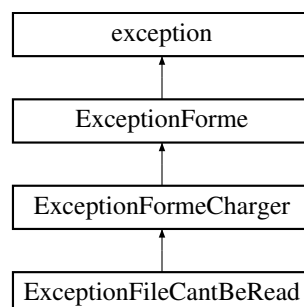
— [FormeSauvegarder.h](#)

6.12 Référence de la classe ExceptionFileCantBeRead

Permet de gérer l'exception liée à l'impossible de lire un fichier.

```
#include <FormeCharger.h>
```

Graphe d'héritage de ExceptionFileCantBeRead:



Fonctions membres publiques

— `string getMessage () const` override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de `ExceptionFormeCharger`

— `string getMessage () const` override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de `ExceptionForme`

— `const char * what () const` noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— `string message = " Le fichier ne peut pas être ouvert"`

Le message lié à l'exception.

6.12.1 Description détaillée

Permet de gérer l'exception liée à l'impossible de lire un fichier.

6.12.2 Documentation des fonctions membres

6.12.2.1 `getMessage()`

```
string ExceptionFileCantBeRead::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de `ExceptionForme`.

6.12.3 Documentation des données membres

6.12.3.1 message

```
string ExceptionFileCantBeRead::message = " Le fichier ne peut pas être ouvert" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

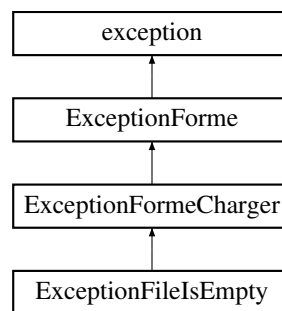
— [FormeCharger.h](#)

6.13 Référence de la classe ExceptionFileIsEmpty

Permet de gérer l'exception liée à un fichier vide.

```
#include <FormeCharger.h>
```

Graphe d'héritage de ExceptionFileIsEmpty:



Fonctions membres publiques

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionFormeCharger](#)

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionForme](#)

— const char * [what](#) () const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— string `message` = " Le fichier est vide"

Le message lié à l'exception.

6.13.1 Description détaillée

Permet de gérer l'exception liée à un fichier vide.

6.13.2 Documentation des fonctions membres

6.13.2.1 getMessage()

```
string ExceptionFileIsEmpty::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionForme](#).

6.13.3 Documentation des données membres

6.13.3.1 message

```
string ExceptionFileIsEmpty::message = " Le fichier est vide" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

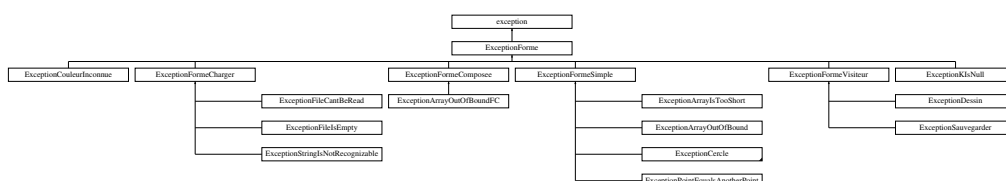
— [FormeCharger.h](#)

6.14 Référence de la classe ExceptionForme

Permet de gérer l'ensemble des exceptions liées aux formes.

```
#include <Forme.h>
```

Graphe d'héritage de ExceptionForme:



Fonctions membres publiques

- `const char * what () const` noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

- `virtual string getMessage () const`

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

- `string message = "ExceptionForme -"`

Le message lié à l'exception.

Amis

- `ostream & operator<< (ostream &s, const ExceptionForme &e)`

Permet l'utilisation du << pour une erreur.

6.14.1 Description détaillée

Permet de gérer l'ensemble des exceptions liées aux formes.

6.14.2 Documentation des fonctions membres

6.14.2.1 `getMessage()`

```
virtual string ExceptionForme::getMessage () const [inline], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée dans [ExceptionArrayIsTooShort](#), [ExceptionArrayOutOfBound](#), [ExceptionArrayOutOfBoundFC](#), [ExceptionCercle](#), [ExceptionConnexionImpossible](#), [ExceptionCouleurInconnue](#), [ExceptionCreationSocket](#), [ExceptionDessin](#), [ExceptionEnvoiImpossible](#), [ExceptionFileCantBeOpen](#), [ExceptionFileCantBeRead](#), [ExceptionFileIsEmpty](#), [ExceptionFormeCharger](#), [ExceptionFormeComposee](#), [ExceptionFormeSimple](#), [ExceptionFormeVisiteur](#), [ExceptionKIsNull](#), [ExceptionParametragePaquet](#), [ExceptionPointEqualsAnotherPoint](#), [ExceptionRayonNegatif](#), [ExceptionSauvegarder](#), [ExceptionStringsNotRecognizable](#), et [ExceptionTransformationForme](#).

6.14.2.2 what()

```
const char * ExceptionForme::what () const [inline], [override], [noexcept]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

6.14.3 Documentation des fonctions amies et associées

6.14.3.1 operator<<

```
ostream & operator<< (
    ostream & s,
    const ExceptionForme & e) [friend]
```

Permet l'utilisation du << pour une erreur.

Renvoie

L'ostream complété

6.14.4 Documentation des données membres

6.14.4.1 message

```
string ExceptionForme::message = "ExceptionForme -" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

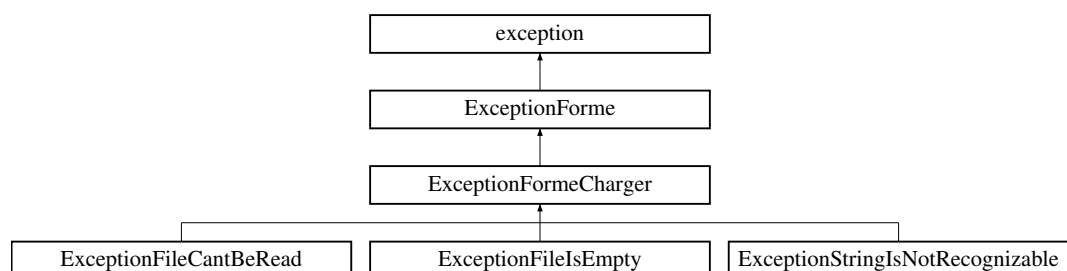
— [Forme.h](#)

6.15 Référence de la classe ExceptionFormeCharger

Permet de gérer l'ensemble des exceptions liées au chargement d'une forme.

```
#include <FormeCharger.h>
```

Graphe d'héritage de ExceptionFormeCharger:



Fonctions membres publiques

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionForme](#)

— const char * [what](#) () const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— string [message](#) = " ExceptionFormeCharge -"

Le message lié à l'exception.

6.15.1 Description détaillée

Permet de gérer l'ensemble des exceptions liées au chargement d'une forme.

6.15.2 Documentation des fonctions membres

6.15.2.1 [getMessage\(\)](#)

```
string ExceptionFormeCharger::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionForme](#).

Réimplémentée dans [ExceptionStringIsNotRecognizable](#).

6.15.3 Documentation des données membres

6.15.3.1 [message](#)

```
string ExceptionFormeCharger::message = " ExceptionFormeCharge -" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

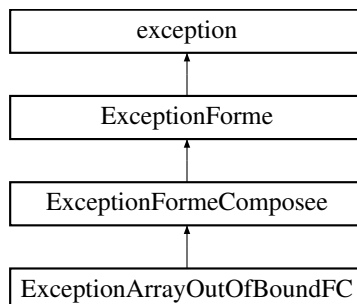
— [FormeCharger.h](#)

6.16 Référence de la classe ExceptionFormeComposee

Permet de gérer l'ensemble des exceptions liées aux formes composées.

```
#include <FormeComposee.h>
```

Graphe d'héritage de ExceptionFormeComposee:



Fonctions membres publiques

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques héritées de [ExceptionForme](#)

— const char * [what](#) () const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— string [message](#) = " ExceptionFormeComposee -"

Le message lié à l'exception.

6.16.1 Description détaillée

Permet de gérer l'ensemble des exceptions liées aux formes composées.

6.16.2 Documentation des fonctions membres

6.16.2.1 [getMessage\(\)](#)

```
string ExceptionFormeComposee::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionForme](#).

6.16.3 Documentation des données membres

6.16.3.1 message

```
string ExceptionFormeComposee::message = " ExceptionFormeComposee -" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

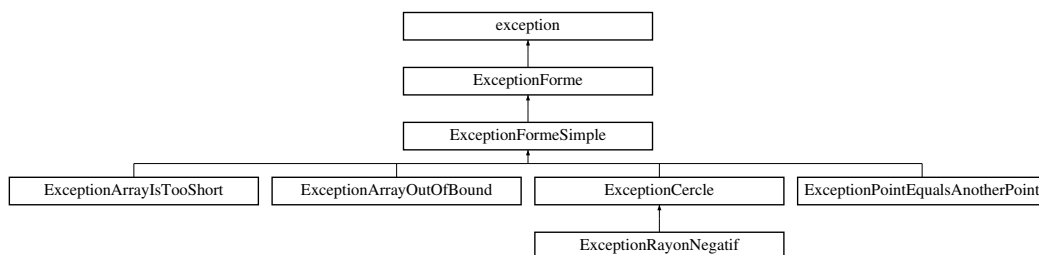
— [FormeComposee.h](#)

6.17 Référence de la classe ExceptionFormeSimple

Permet de gérer l'ensemble des exceptions liées aux formes simples.

```
#include <FormeSimple.h>
```

Graphe d'héritage de ExceptionFormeSimple:



Fonctions membres publiques

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques héritées de [ExceptionForme](#)

— const char * [what](#) () const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— string [message](#) = " ExceptionFormeSimple -"

Le message lié à l'exception.

6.17.1 Description détaillée

Permet de gérer l'ensemble des exceptions liées aux formes simples.

6.17.2 Documentation des fonctions membres

6.17.2.1 getMessage()

```
string ExceptionFormeSimple::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionForme](#).

Réimplémentée dans [ExceptionPointEqualsAnotherPoint](#), et [ExceptionRayonNegatif](#).

6.17.3 Documentation des données membres

6.17.3.1 message

```
string ExceptionFormeSimple::message = " ExceptionFormeSimple -" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

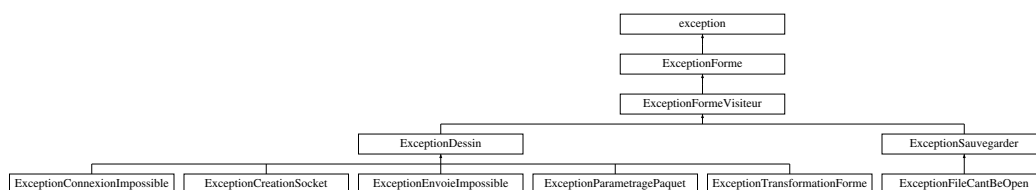
— [FormeSimple.h](#)

6.18 Référence de la classe ExceptionFormeVisiteur

Permet de gérer l'ensemble des exceptions liées aux visiteurs de formes.

```
#include <FormeVisiteur.h>
```

Graphe d'héritage de ExceptionFormeVisiteur:



Fonctions membres publiques

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionForme](#)

— const char * [what](#) () const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— string [message](#) = " ExceptionFormeVisiteur -"

Le message lié à l'exception.

6.18.1 Description détaillée

Permet de gérer l'ensemble des exceptions liées aux visiteurs de formes.

6.18.2 Documentation des fonctions membres

6.18.2.1 [getMessage\(\)](#)

```
string ExceptionFormeVisiteur::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionForme](#).

Réimplémentée dans [ExceptionParametragePaquet](#), [ExceptionSauvegarder](#), et [ExceptionTransformationForme](#).

6.18.3 Documentation des données membres

6.18.3.1 [message](#)

```
string ExceptionFormeVisiteur::message = " ExceptionFormeVisiteur -" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

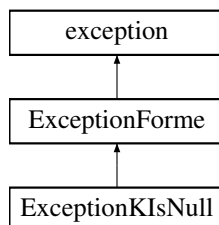
— [FormeVisiteur.h](#)

6.19 Référence de la classe ExceptionKIsNull

Permet de gérer l'erreur de l'utilisation d'un k nulle pour la rotation.

```
#include <Forme.h>
```

Graphe d'héritage de ExceptionKIsNull:



Fonctions membres publiques

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionForme](#)

— const char * [what](#) () const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— string [message](#) = " Le rapport d'homothetie k = 0 est interdit"

Le message lié à l'exception.

6.19.1 Description détaillée

Permet de gérer l'erreur de l'utilisation d'un k nulle pour la rotation.

6.19.2 Documentation des fonctions membres

6.19.2.1 [getMessage](#)()

```
string ExceptionKIsNull::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionForme](#).

6.19.3 Documentation des données membres

6.19.3.1 message

```
string ExceptionKIsNull::message = " Le rapport d'homothetie k = 0 est interdit" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

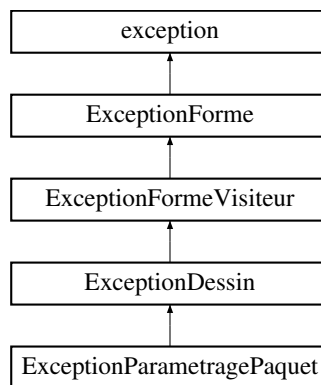
— [Forme.h](#)

6.20 Référence de la classe ExceptionParametragePaquet

Permet de gérer l'exception lors de l'impossibilité de paramétrer l'envoi des messages.

```
#include <FormeDessiner.h>
```

Graphe d'héritage de ExceptionParametragePaquet:



Fonctions membres publiques

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de [ExceptionForme](#)

— const char * [what](#) () const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— string [message](#) = " Il est impossible de parametrer les paquets"

Le message lié à l'exception.

6.20.1 Description détaillée

Permet de gérer l'exception lors de l'impossibilité de paramétrer l'envoi des messages.

6.20.2 Documentation des fonctions membres

6.20.2.1 getMessage()

```
string ExceptionParametragePaquet::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionDessin](#).

6.20.3 Documentation des données membres

6.20.3.1 message

```
string ExceptionParametragePaquet::message = " Il est impossible de paramettrer les paquets"  
[private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

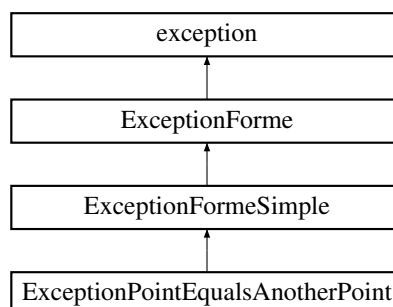
— [FormeDessiner.h](#)

6.21 Référence de la classe ExceptionPointEqualsAnotherPoint

Permet de gérer l'exception lorsque deux points sont identiques dans une forme simple.

```
#include <FormeSimple.h>
```

Graphe d'héritage de ExceptionPointEqualsAnotherPoint:



Fonctions membres publiques

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques héritées de [ExceptionForme](#)

— const char * [what](#) () const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— string [message](#) = " Deux des points sont de memes coordonnees"

Le message lié à l'exception.

6.21.1 Description détaillée

Permet de gérer l'exception lorsque deux points sont identiques dans une forme simple.

6.21.2 Documentation des fonctions membres

6.21.2.1 [getMessage\(\)](#)

```
string ExceptionPointEqualsAnotherPoint::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionFormeSimple](#).

6.21.3 Documentation des données membres

6.21.3.1 [message](#)

```
string ExceptionPointEqualsAnotherPoint::message = " Deux des points sont de memes coordonnees" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

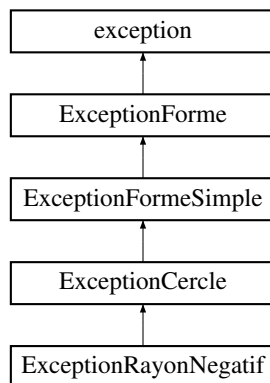
— [FormeSimple.h](#)

6.22 Référence de la classe ExceptionRayonNegatif

Permet de gérer l'exception lorsque le rayon est negatif.

```
#include <Cercle.h>
```

Graphe d'héritage de ExceptionRayonNegatif:



Fonctions membres publiques

— string `getMessage()` const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques hérités de `ExceptionForme`

— const char * `what()` const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— string `message` = " Le rayon doit etre superieur a 0"

Le message lié à l'exception.

6.22.1 Description détaillée

Permet de gérer l'exception lorsque le rayon est negatif.

6.22.2 Documentation des fonctions membres

6.22.2.1 getMessage()

```
string ExceptionRayonNegatif::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionCercle](#).

6.22.3 Documentation des données membres

6.22.3.1 message

```
string ExceptionRayonNegatif::message = " Le rayon doit etre superieur a 0" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

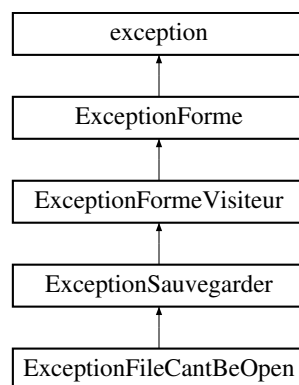
— [Cercle.h](#)

6.23 Référence de la classe ExceptionSauvegarder

Permet de gérer l'ensemble des exceptions liées aux visiteurs de formes sauvegarde.

```
#include <FormeSauvegarder.h>
```

Graphe d'héritage de ExceptionSauvegarder:



Fonctions membres publiques

— string [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques héritées de [ExceptionForme](#)

— `const char * what () const` noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— `string message = " ExceptionSauvegarder -"`

Le message lié à l'exception.

6.23.1 Description détaillée

Permet de gérer l'ensemble des exceptions liées aux visiteurs de formes sauvegarde.

6.23.2 Documentation des fonctions membres

6.23.2.1 `getMessage()`

```
string ExceptionSauvegarder::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionFormeVisiteur](#).

6.23.3 Documentation des données membres

6.23.3.1 `message`

```
string ExceptionSauvegarder::message = " ExceptionSauvegarder -" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

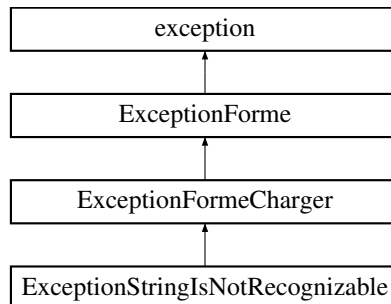
— [FormeSauvegarder.h](#)

6.24 Référence de la classe `ExceptionStringIsNotRecognizable`

Permet de gérer l'exception dans le cas d'une forme non reconnue.

```
#include <FormeCharger.h>
```

Graphe d'héritage de `ExceptionStringIsNotRecognizable`:



Fonctions membres publiques

— `string getMessage () const` override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques héritées de `ExceptionForme`

— `const char * what () const` noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— `string message = "Aucune forme ne peut être déduit d'une des chaînes de caractères du fichier"`

Le message lié à l'exception.

6.24.1 Description détaillée

Permet de gérer l'exception dans le cas d'une forme non reconnue.

6.24.2 Documentation des fonctions membres

6.24.2.1 `getMessage()`

```
string ExceptionStringIsNotRecognizable::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de `ExceptionFormeCharger`.

6.24.3 Documentation des données membres

6.24.3.1 message

```
string ExceptionStringIsNotRecognizable::message = " Aucune forme ne peut être déduit d'une  
des chaînes de caractères du fichier" [private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

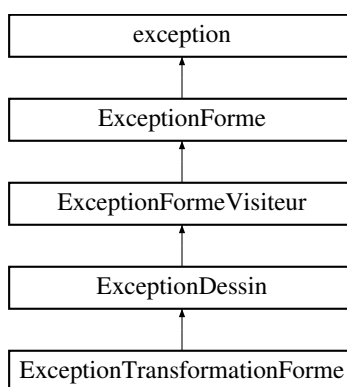
— [FormeCharger.h](#)

6.25 Référence de la classe ExceptionTransformationForme

Permet de gérer l'exception lors de l'impossibilité de transformer la forme.

```
#include <FormeDessiner.h>
```

Grappe d'héritage de ExceptionTransformationForme:



Fonctions membres publiques

— `string` [getMessage](#) () const override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Fonctions membres publiques héritées de [ExceptionForme](#)

— `const char *` [what](#) () const noexcept override

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Attributs privés

— `string` [message](#) = " Il est impossible de transformer la forme"

Le message lié à l'exception.

6.25.1 Description détaillée

Permet de gérer l'exception lors de l'impossibilité de transformer la forme.

6.25.2 Documentation des fonctions membres

6.25.2.1 getMessage()

```
string ExceptionTransformationForme::getMessage () const [inline], [override], [virtual]
```

Permet d'obtenir une chaîne de caractère lié à l'erreur.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [ExceptionDessin](#).

6.25.3 Documentation des données membres

6.25.3.1 message

```
string ExceptionTransformationForme::message = " Il est impossible de transformer la forme"  
[private]
```

Le message lié à l'exception.

La documentation de cette classe a été générée à partir du fichier suivant :

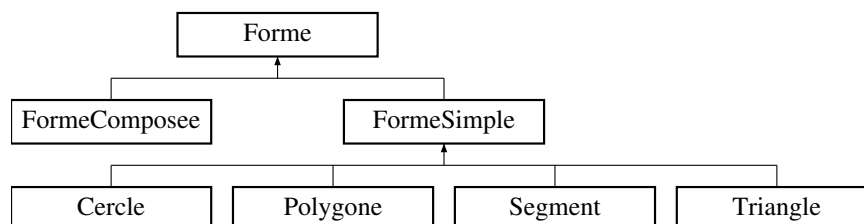
— [FormeDessiner.h](#)

6.26 Référence de la classe Forme

Permet de créer une forme.

```
#include <Forme.h>
```

Graphe d'héritage de Forme:



Fonctions membres publiques

- **Forme** (const **Couleur** &couleur=**Couleur::BLACK**)
Construit une forme à partir d'une couleur.
- virtual **Forme** * **clone** () const =0
Retourne un clone de la forme.
- virtual ~**Forme** ()
Permet la destruction de la forme.
- **Couleur** **getCouleur** () const
Retourne la couleur de la forme courante.
- **Forme** & **setCouleur** (const **Couleur** &couleur)
Permet de changer la couleur de la forme.
- virtual vector< **Vecteur2D** > **getPoints** () const =0
Permet d'obtenir l'ensemble des points de la forme.
- virtual double **getAire** () const =0
Permet d'obtenir l'aire d'une forme.
- virtual **Forme** & **translation** (const **Vecteur2D** &p)=0
Permet de réaliser une translation de la forme.
- virtual **Forme** & **homothetie** (const **Vecteur2D** &p, const double &k)=0
Permet de réaliser une homothétie de la forme.
- virtual **Forme** & **rotation** (const **Vecteur2D** &p, const double &r)=0
Permet de réaliser une rotation de la forme.
- virtual **Forme** & **transformationEcran** (const **Matrice22** &m, const **Vecteur2D** &ab)=0
Permet de réaliser une transformation de la forme pour un affichage sur écran.
- virtual string **toString** () const
Permet d'obtenir une chaîne de caractère indiquant les spécificités de la forme.
- **operator string** () const
Permet d'obtenir une chaîne de caractère indiquant les spécificités de la forme.
- virtual **Forme** & **accept** (**FormeVisiteur** &visiteur)=0
Permet l'introduction de méthode via le design pattern visitor.

Fonctions membres publiques statiques

- static `Forme * charger` (const string &path)

Permet de charger une forme à partir d'un fichier local.

Fonctions membres protégées statiques

- static void `verificationKIsNotNull` (const double &k)

Vérifie si la constante k pour l'homothétie n'est pas nulle.

Attributs privés

- `Couleur couleur`

La couleur de la forme.

Amis

- ostream & `operator<<` (ostream &s, const `Forme` &f)

Permet l'utilisation du << pour une forme lors de l'utilisation d'ostream.

6.26.1 Description détaillée

Permet de créer une forme.

6.26.2 Documentation des constructeurs et destructeur

6.26.2.1 `Forme()`

```
Forme::Forme (
    const Couleur & couleur = Couleur::BLACK) [explicit]
```

Construit une forme à partir d'une couleur.

Paramètres

in	<code>couleur</code>	La couleur souhaitée (à défaut noir)
----	----------------------	--------------------------------------

Renvoie

Une forme

6.26.2.2 `~Forme()`

```
Forme::~~Forme () [virtual], [default]
```

Permet la destruction de la forme.

6.26.3 Documentation des fonctions membres

6.26.3.1 `accept()`

```
virtual Forme & Forme::accept (  
    FormeVisiteur & visiteur) [pure virtual]
```

Permet l'introduction de méthode via le design pattern visitor.

Paramètres

in	<i>visiteur</i>	Un objet répondant au pattern visitor
----	-----------------	---------------------------------------

Renvoie

(*this) L'objet implicite

Implémenté dans [Cercle](#), [FormeComposee](#), [FormeSimple](#), [Polygone](#), [Segment](#), et [Triangle](#).

6.26.3.2 `charger()`

```
Forme * Forme::charger (  
    const string & path) [static]
```

Permet de charger une forme à partir d'un fichier local.

Paramètres

in	<i>path</i>	L'adresse du fichier souhaité
----	-------------	-------------------------------

Renvoie

L'objet du fichier

6.26.3.3 `clone()`

```
virtual Forme * Forme::clone () const [pure virtual]
```

Retourne un clone de la forme.

Renvoie

Un clone de la forme

Implémenté dans [Cercle](#), [FormeComposee](#), [FormeSimple](#), [Polygone](#), [Segment](#), et [Triangle](#).

6.26.3.4 `getAire()`

```
virtual double Forme::getAire () const [pure virtual]
```

Permet d'obtenir l'aire d'une forme.

Renvoie

L'aire de la forme

Implémenté dans [Cercle](#), [FormeComposee](#), [FormeSimple](#), [Polygone](#), [Segment](#), et [Triangle](#).

6.26.3.5 `getCouleur()`

```
Couleur Forme::getCouleur () const
```

Retourne la couleur de la forme courante.

Renvoie

couleur La couleur

6.26.3.6 `getPoints()`

```
virtual vector< Vecteur2D > Forme::getPoints () const [pure virtual]
```

Permet d'obtenir l'ensemble des points de la forme.

Renvoie

L'ensemble des points

Implémenté dans [Cercle](#), [FormeComposee](#), [Polygone](#), [Segment](#), et [Triangle](#).

6.26.3.7 `homothetie()`

```
virtual Forme & Forme::homothetie (  
    const Vecteur2D & p,  
    const double & k) [pure virtual]
```

Permet de réaliser une homothétie de la forme.

Renvoie

(*this) L'objet implicite

Implémenté dans [FormeComposee](#), et [FormeSimple](#).

6.26.3.8 operator string()

```
Forme::operator string () const [explicit]
```

Permet d'obtenir une chaîne de caractère indiquant les spécificités de la forme.

Renvoie

Une chaîne de caractère

6.26.3.9 rotation()

```
virtual Forme & Forme::rotation (  
    const Vecteur2D & p,  
    const double & r) [pure virtual]
```

Permet de réaliser une rotation de la forme.

Renvoie

(*this) L'objet implicite

Implémenté dans [FormeComposee](#), et [FormeSimple](#).

6.26.3.10 setCouleur()

```
Forme & Forme::setCouleur (  
    const Couleur & couleur)
```

Permet de changer la couleur de la forme.

Paramètres

in	couleur	La couleur
----	-------------------------	------------

Renvoie

(*this) L'objet implicite

6.26.3.11 toString()

```
string Forme::toString () const [virtual]
```

Permet d'obtenir une chaîne de caractère indiquant les spécificités de la forme.

Renvoie

Une chaîne de caractère

Réimplémentée dans [Cercle](#), [FormeComposee](#), [Polygone](#), [Segment](#), et [Triangle](#).

6.26.3.12 transformationEcran()

```
virtual Forme & Forme::transformationEcran (
    const Matrice2D & m,
    const Vecteur2D & ab) [pure virtual]
```

Permet de réaliser une transformation de la forme pour un affichage sur écran.

Paramètres

in	<i>m</i>	Une matrice
in	<i>ab</i>	Un vecteur

Renvoie

(*this) L'objet implicite

Implémenté dans [FormeComposee](#), et [FormeSimple](#).

6.26.3.13 translation()

```
virtual Forme & Forme::translation (
    const Vecteur2D & p) [pure virtual]
```

Permet de réaliser une translation de la forme.

Renvoie

(*this) L'objet implicite

Implémenté dans [FormeComposee](#), et [FormeSimple](#).

6.26.3.14 verificationKIsNotNull()

```
void Forme::verificationKIsNotNull (
    const double & k) [static], [protected]
```

Verifie si la constante k pour l'homothétie n'est pas nulle.

Paramètres

in	<i>k</i>	La constante
----	----------	--------------

Renvoie

void

6.26.4 Documentation des fonctions amies et associées

6.26.4.1 `operator<<`

```
ostream & operator<< (
    ostream & s,
    const Forme & f) [friend]
```

Permet l'utilisation du `<<` pour une forme lors de l'utilisation d'`ostream`.

Renvoie

L'`ostream` complété

6.26.5 Documentation des données membres

6.26.5.1 `couleur`

```
Couleur Forme::couleur [private]
```

La couleur de la forme.

La documentation de cette classe a été générée à partir des fichiers suivants :

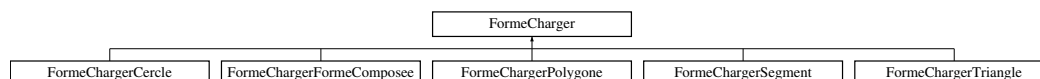
- [Forme.h](#)
- [Forme.cpp](#)

6.27 Référence de la classe `FormeCharger`

Permet de gérer le chargement d'une forme.

```
#include <FormeCharger.h>
```

Graphe d'héritage de `FormeCharger`:



Fonctions membres publiques

- [FormeCharger](#) ([FormeCharger](#) *suivant)

Permet la construction d'un maillon du design pattern chain of responsibility permettant le chargement d'une forme.

- `virtual ~FormeCharger ()`

Permet la destruction de l'objet.

- [Forme](#) * [chargerDepuisFichier](#) (const string &path) const

Permet de charger une forme à partir d'un fichier local.

- [Forme](#) * [chargerDepuisFlux](#) (const string &s, istream &flux) const

Permet de charger une forme à partir d'un flux et d'un string.

Fonctions membres publiques statiques

— static `FormeCharger * getOrigine ()`

Permet d'obtenir le maillon originel de la chaîne.

Fonctions membres protégées

— virtual `Forme * charger (const string &s, istringstream &flux) const =0`

Permet de charger une forme.

Fonctions membres privées

— `Forme * chargerDepuisFluxStart (istringstream &flux) const`

Permet la mise en place de la fonction chargerDepuisFlux.

Attributs privés

— `FormeCharger * suivant`

Le prochain maillon de la chaîne.

Attributs privés statiques

— static `FormeCharger * origine = nullptr`

L'origine de la chaîne.

6.27.1 Description détaillée

Permet de gérer le chargement d'une forme.

6.27.2 Documentation des constructeurs et destructeur

6.27.2.1 `FormeCharger()`

```
FormeCharger::FormeCharger (
    FormeCharger * suivant) [explicit]
```

Permet la construction d'un maillon du design pattern chain of responsibility permettant le chargement d'une forme.

Paramètres

in	suivant	Le prochain maillon
----	-------------------------	---------------------

6.27.2.2 `~FormeCharger()`

```
FormeCharger::~FormeCharger () [virtual]
```

Permet la destruction de l'objet.

6.27.3 Documentation des fonctions membres**6.27.3.1 `charger()`**

```
virtual Forme * FormeCharger::charger (  
    const string & s,  
    istream & flux) const [protected], [pure virtual]
```

Permet de charger une forme.

Paramètres

in	<i>s</i>	Le string étant la première ligne du flux en cours
in	<i>flux</i>	Le flux de string

Renvoie

L'objet du flux

Implémenté dans [FormeChargerCercle](#), [FormeChargerFormeComposee](#), [FormeChargerPolygone](#), [FormeChargerSegment](#), et [FormeChargerTriangle](#).

6.27.3.2 `chargerDepuisFichier()`

```
Forme * FormeCharger::chargerDepuisFichier (  
    const string & path) const
```

Permet de charger une forme à partir d'un fichier local.

Paramètres

in	<i>path</i>	L'adresse du fichier souhaité
----	-------------	-------------------------------

Renvoie

L'objet du fichier

6.27.3.3 chargerDepuisFlux()

```
Forme * FormeCharger::chargerDepuisFlux (
    const string & s,
    istream & flux) const
```

Permet de charger une forme à partir d'un flux et d'un string.

Paramètres

in	<i>s</i>	Le string étant la première ligne du flux en cours
in	<i>flux</i>	Le flux de string

Renvoie

L'objet du flux

6.27.3.4 chargerDepuisFluxStart()

```
Forme * FormeCharger::chargerDepuisFluxStart (
    istream & flux) const [private]
```

Permet la mise en place de la fonction chargerDepuisFlux.

Paramètres

in	<i>flux</i>	Le flux de string
----	-------------	-------------------

Renvoie

L'objet du flux

6.27.3.5 getOrigine()

```
FormeCharger * FormeCharger::getOrigine () [static]
```

Permet d'obtenir le maillon originel de la chaîne.

Renvoie

origine Le maillon originel

6.27.4 Documentation des données membres

6.27.4.1 origine

```
FormeCharger * FormeCharger::origine = nullptr [static], [private]
```

L'origine de la chaîne.

6.27.4.2 suivant

```
FormeCharger* FormeCharger::suivant [private]
```

Le prochain maillon de la chaîne.

La documentation de cette classe a été générée à partir des fichiers suivants :

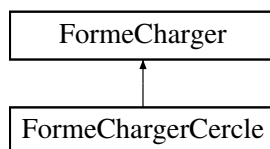
- [FormeCharger.h](#)
- [FormeCharger.cpp](#)

6.28 Référence de la classe `FormeChargerCercle`

Permet le chargement d'un cercle.

```
#include <FormeChargerCercle.h>
```

Graphe d'héritage de `FormeChargerCercle`:



Fonctions membres publiques

- [FormeChargerCercle](#) ([FormeCharger](#) *suivant)

Permet la construction d'un maillon du design pattern chain of responsibility permettant le chargement d'une forme, ici pour un cercle.

- [~FormeChargerCercle](#) () override

Permet la destruction de l'objet.

Fonctions membres publiques héritées de [FormeCharger](#)

- [FormeCharger](#) ([FormeCharger](#) *suivant)

Permet la construction d'un maillon du design pattern chain of responsibility permettant le chargement d'une forme.

- virtual [~FormeCharger](#) ()

Permet la destruction de l'objet.

- [Forme](#) * [chargerDepuisFichier](#) (const string &path) const

Permet de charger une forme à partir d'un fichier local.

- [Forme](#) * [chargerDepuisFlux](#) (const string &s, istream &flux) const

Permet de charger une forme à partir d'un flux et d'un string.

Fonctions membres protégées

- `Forme * charger` (const string &s, istringstream &flux) const override

Permet de charger un cercle.

Membres hérités additionnels

Fonctions membres publiques statiques hérités de `FormeCharger`

- static `FormeCharger * getOrigine` ()

Permet d'obtenir le maillon originel de la chaîne.

6.28.1 Description détaillée

Permet le chargement d'un cercle.

6.28.2 Documentation des constructeurs et destructeur

6.28.2.1 `FormeChargerCercle()`

```
FormeChargerCercle::FormeChargerCercle (
    FormeCharger * suivant) [explicit]
```

Permet la construction d'un maillon du design pattern chain of responsibility permettant le chargement d'une forme, ici pour un cercle.

Paramètres

in	<code>suivant</code>	Le prochain maillon
----	----------------------	---------------------

6.28.2.2 `~FormeChargerCercle()`

```
FormeChargerCercle::~~FormeChargerCercle () [override], [default]
```

Permet la destruction de l'objet.

6.28.3 Documentation des fonctions membres

6.28.3.1 `charger()`

```
Forme * FormeChargerCercle::charger (
    const string & s,
    istream & flux) const [override], [protected], [virtual]
```

Permet de charger un cercle.

Paramètres

in	<i>s</i>	Le string étant la première ligne du flux en cours
in	<i>flux</i>	Le flux de string

Renvoie

Un cercle

Implémente [FormeCharger](#).

La documentation de cette classe a été générée à partir des fichiers suivants :

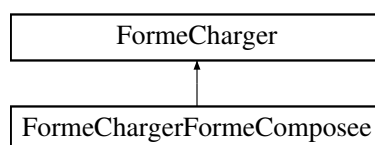
- [FormeChargerCercle.h](#)
- [FormeChargerCercle.cpp](#)

6.29 Référence de la classe `FormeChargerFormeComposee`

Permet le chargement d'une forme composée.

```
#include <FormeChargerFormeComposee.h>
```

Graphe d'héritage de `FormeChargerFormeComposee`:



Fonctions membres publiques

- [FormeChargerFormeComposee \(FormeCharger *suivant\)](#)

Permet la construction d'un maillon du design pattern chain of responsibility permettant le chargement d'une forme, ici pour une forme composée.

- [~FormeChargerFormeComposee \(\)](#) override

Permet la destruction de l'objet.

Fonctions membres publiques hérités de **FormeCharger**

- **FormeCharger** (**FormeCharger** *suivant)

Permet la construction d'un maillon du design pattern chain of responsibility permettant le chargement d'une forme.

- virtual ~**FormeCharger** ()

Permet la destruction de l'objet.

- **Forme** * chargerDepuisFichier (const string &path) const

Permet de charger une forme à partir d'un fichier local.

- **Forme** * chargerDepuisFlux (const string &s, istream &flux) const

Permet de charger une forme à partir d'un flux et d'un string.

Fonctions membres protégées

- **Forme** * charger (const string &s, istream &flux) const override

Permet de charger une forme composée.

Membres hérités additionnels

Fonctions membres publiques statiques hérités de **FormeCharger**

- static **FormeCharger** * getOrigine ()

Permet d'obtenir le maillon originel de la chaîne.

6.29.1 Description détaillée

Permet le chargement d'une forme composée.

6.29.2 Documentation des constructeurs et destructeur

6.29.2.1 **FormeChargerFormeComposee()**

```
FormeChargerFormeComposee::FormeChargerFormeComposee (
    FormeCharger * suivant) [explicit]
```

Permet la construction d'un maillon du design pattern chain of responsibility permettant le chargement d'une forme, ici pour une forme composée.

Paramètres

in	suivant	Le prochain maillon
----	-------------------------	---------------------

6.29.2.2 `~FormeChargerFormeComposee()`

```
FormeChargerFormeComposee::~FormeChargerFormeComposee () [override], [default]
```

Permet la destruction de l'objet.

6.29.3 Documentation des fonctions membres**6.29.3.1 `charger()`**

```
Forme * FormeChargerFormeComposee::charger (
    const string & s,
    istream & flux) const [override], [protected], [virtual]
```

Permet de charger une forme composée.

Paramètres

in	<i>s</i>	Le string étant la première ligne du flux en cours
in	<i>flux</i>	Le flux de string

Renvoie

Une forme composée

Implémente [FormeCharger](#).

La documentation de cette classe a été générée à partir des fichiers suivants :

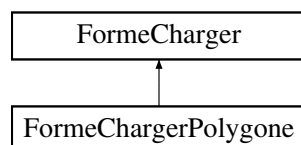
- [FormeChargerFormeComposee.h](#)
- [FormeChargerFormeComposee.cpp](#)

6.30 Référence de la classe `FormeChargerPolygone`

Permet le chargement d'un polygone.

```
#include <FormeChargerPolygone.h>
```

Graphique d'héritage de `FormeChargerPolygone`:



Fonctions membres publiques

- [FormeChargerPolygone](#) ([FormeCharger](#) *suivant)

Permet la construction d'un maillon du design pattern chain of responsibility permettant le chargement d'une forme, ici pour un polygone.

- [~FormeChargerPolygone](#) () override

Permet la destruction de l'objet.

Fonctions membres publiques hérités de [FormeCharger](#)

- [FormeCharger](#) ([FormeCharger](#) *suivant)

Permet la construction d'un maillon du design pattern chain of responsibility permettant le chargement d'une forme.

- virtual [~FormeCharger](#) ()

Permet la destruction de l'objet.

- [Forme](#) * [chargerDepuisFichier](#) (const string &path) const

Permet de charger une forme à partir d'un fichier local.

- [Forme](#) * [chargerDepuisFlux](#) (const string &s, istream &flux) const

Permet de charger une forme à partir d'un flux et d'un string.

Fonctions membres protégées

- [Forme](#) * [charger](#) (const string &s, istream &flux) const override

Permet de charger un polygone.

Membres hérités additionnels

Fonctions membres publiques statiques hérités de [FormeCharger](#)

- static [FormeCharger](#) * [getOrigine](#) ()

Permet d'obtenir le maillon originel de la chaîne.

6.30.1 Description détaillée

Permet le chargement d'un polygone.

6.30.2 Documentation des constructeurs et destructeur

6.30.2.1 `FormeChargerPolygone()`

```
FormeChargerPolygone::FormeChargerPolygone (
    FormeCharger * suivant) [explicit]
```

Permet la construction d'un maillon du design pattern chain of responsibility permettant le chargement d'une forme, ici pour un polygone.

Paramètres

in	suivant	Le prochain maillon
----	-------------------------	---------------------

6.30.2.2 `~FormeChargerPolygone()`

```
FormeChargerPolygone::~~FormeChargerPolygone () [override], [default]
```

Permet la destruction de l'objet.

6.30.3 Documentation des fonctions membres

6.30.3.1 `charger()`

```
Forme * FormeChargerPolygone::charger (
    const string & s,
    istream & flux) const [override], [protected], [virtual]
```

Permet de charger un polygone.

Paramètres

in	<i>s</i>	Le string étant la première ligne du flux en cours
in	<i>flux</i>	Le flux de string

Renvoie

Un polygone

Implémente [FormeCharger](#).

La documentation de cette classe a été générée à partir des fichiers suivants :

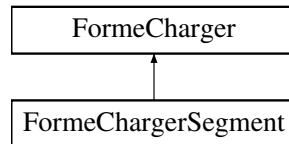
- [FormeChargerPolygone.h](#)
- [FormeChargerPolygone.cpp](#)

6.31 Référence de la classe `FormeChargerSegment`

Permet le chargement d'un segment.

```
#include <FormeChargerSegment.h>
```

Graphe d'héritage de `FormeChargerSegment`:



Fonctions membres publiques

- `FormeChargerSegment (FormeCharger *suivant)`

Permet la construction d'un maillon du design pattern chain of responsibility permettant le chargement d'une forme, ici pour un segment.

- `~FormeChargerSegment ()` override

Permet la destruction de l'objet.

Fonctions membres publiques héritées de `FormeCharger`

- `FormeCharger (FormeCharger *suivant)`

Permet la construction d'un maillon du design pattern chain of responsibility permettant le chargement d'une forme.

- `virtual ~FormeCharger ()`

Permet la destruction de l'objet.

- `Forme * chargerDepuisFichier (const string &path) const`

Permet de charger une forme à partir d'un fichier local.

- `Forme * chargerDepuisFlux (const string &s, istream &flux) const`

Permet de charger une forme à partir d'un flux et d'un string.

Fonctions membres protégées

- `Forme * charger (const string &s, istream &flux) const` override

Permet de charger un segment.

Membres hérités additionnels

Fonctions membres publiques statiques hérités de `FormeCharger`

— static `FormeCharger * getOrigine ()`

Permet d'obtenir le maillon originel de la chaîne.

6.31.1 Description détaillée

Permet le chargement d'un segment.

6.31.2 Documentation des constructeurs et destructeur

6.31.2.1 `FormeChargerSegment()`

```
FormeChargerSegment::FormeChargerSegment (
    FormeCharger * suivant) [explicit]
```

Permet la construction d'un maillon du design pattern chain of responsibility permettant le chargement d'une forme, ici pour un segment.

Paramètres

in	<code>suivant</code>	Le prochain maillon
----	----------------------	---------------------

6.31.2.2 `~FormeChargerSegment()`

```
FormeChargerSegment::~~FormeChargerSegment () [override], [default]
```

Permet la destruction de l'objet.

6.31.3 Documentation des fonctions membres

6.31.3.1 `charger()`

```
Forme * FormeChargerSegment::charger (
    const string & s,
    istringstream & flux) const [override], [protected], [virtual]
```

Permet de charger un segment.

Paramètres

in	<code>s</code>	Le string étant la première ligne du flux en cours
----	----------------	--

in	flux	Le flux de string
----	------	-------------------

Renvoie

Un segment

Implémente [FormeCharger](#).

La documentation de cette classe a été générée à partir des fichiers suivants :

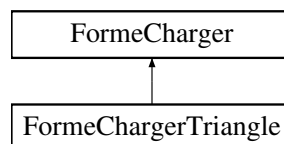
- [FormeChargerSegment.h](#)
- [FormeChargerSegment.cpp](#)

6.32 Référence de la classe FormeChargerTriangle

Permet le chargement d'un triangle.

```
#include <FormeChargerTriangle.h>
```

Graphe d'héritage de FormeChargerTriangle:



Fonctions membres publiques

- [FormeChargerTriangle](#) ([FormeCharger](#) *suivant)

Permet la construction d'un maillon du design pattern chain of responsibility permettant le chargement d'une forme, ici pour un triangle.

- [~FormeChargerTriangle](#) () override

Permet la destruction de l'objet.

Fonctions membres publiques hérités de [FormeCharger](#)

- [FormeCharger](#) ([FormeCharger](#) *suivant)

Permet la construction d'un maillon du design pattern chain of responsibility permettant le chargement d'une forme.

- virtual [~FormeCharger](#) ()

Permet la destruction de l'objet.

- [Forme](#) * [chargerDepuisFichier](#) (const string &path) const

Permet de charger une forme à partir d'un fichier local.

- [Forme](#) * [chargerDepuisFlux](#) (const string &s, istream &flux) const

Permet de charger une forme à partir d'un flux et d'un string.

Fonctions membres protégées

— `Forme * charger` (const string &s, istringstream &flux) const override

Permet de charger un triangle.

Membres hérités additionnels

Fonctions membres publiques statiques hérités de `FormeCharger`

— static `FormeCharger * getOrigine` ()

Permet d'obtenir le maillon originel de la chaîne.

6.32.1 Description détaillée

Permet le chargement d'un triangle.

6.32.2 Documentation des constructeurs et destructeur

6.32.2.1 `FormeChargerTriangle()`

```
FormeChargerTriangle::FormeChargerTriangle (  
    FormeCharger * suivant) [explicit]
```

Permet la construction d'un maillon du design pattern chain of responsibility permettant le chargement d'une forme, ici pour un triangle.

Paramètres

in	<code>suivant</code>	Le prochain maillon
----	----------------------	---------------------

6.32.2.2 `~FormeChargerTriangle()`

```
FormeChargerTriangle::~~FormeChargerTriangle () [override], [default]
```

Permet la destruction de l'objet.

6.32.3 Documentation des fonctions membres

6.32.3.1 charger()

```
Forme * FormeChargerTriangle::charger (
    const string & s,
    istream & flux) const [override], [protected], [virtual]
```

Permet de charger un triangle.

Paramètres

in	<i>s</i>	Le string étant la première ligne du flux en cours
in	<i>flux</i>	Le flux de string

Renvoie

Un triangle

Implémente [FormeCharger](#).

La documentation de cette classe a été générée à partir des fichiers suivants :

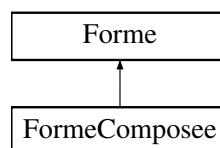
- [FormeChargerTriangle.h](#)
- [FormeChargerTriangle.cpp](#)

6.33 Référence de la classe FormeComposee

Permet de gérer une forme composée.

```
#include <FormeComposee.h>
```

Graphe d'héritage de FormeComposee:



Fonctions membres publiques

- `FormeComposee` (const `Couleur` &couleur)

Construit une forme composée à partir d'une couleur.

- `FormeComposee` (const vector< `Forme` * > &v=vector< `Forme` * >(), const `Couleur` &couleur=`Couleur::BLACK`)

Construit une forme composée à partir d'une couleur et d'un vecteur de forme.

- `FormeComposee` (const `FormeComposee` &f)

Construit une forme composée à partir d'une autre forme composée.

- `FormeComposee` & `operator=` (const `FormeComposee` &f)

Permet la copie des champs de `f` dans l'objet implicite.

- `~FormeComposee` () override

Permet la destruction de la forme composée.

- `FormeComposee` * `clone` () const override

Retourne un clone de la forme composée.

- vector< `Vecteur2D` > `getPoints` () const override

Permet d'obtenir l'ensemble des points de la forme.

- vector< `Forme` * > `getFormes` () const

Retourne l'ensemble des formes de la forme composée.

- `FormeComposee` & `setFormes` (const vector< `Forme` * > &vf)

Attribue un ensemble de forme à la forme composée.

- `Forme` * `getFormeAt` (int i) const

Retourne une forme à l'indice `i` parmi les formes de la forme composée.

- `FormeComposee` & `addForme` (const `Forme` &f)

Permet d'ajouter une forme à la forme composée.

- `FormeComposee` & `addFormeAt` (const `Forme` &f, int i)

Permet d'ajouter une forme à la forme composée à la position `i`.

- `FormeComposee` & `removeFormeAt` (int i)

Permet de retirer une forme à la position `i` du vecteur de la forme composée.

- double `getAire` () const override

Retourne l'aire de la forme composée.

- `FormeComposee` & `translation` (const `Vecteur2D` &p) override

Permet de réaliser une translation de la forme composée.

- [FormeComposee](#) & [homothetie](#) (const [Vecteur2D](#) &p, const double &k) override

Permet de réaliser une homothétie de la forme composée.

- [FormeComposee](#) & [rotation](#) (const [Vecteur2D](#) &p, const double &r) override

Permet de réaliser une rotation de la forme composée.

- [FormeComposee](#) & [transformationEcran](#) (const [Matrice22](#) &m, const [Vecteur2D](#) &ab) override

Permet de réaliser une transformation de la forme simple pour un affichage sur écran.

- string [toString](#) () const override

Permet d'obtenir une chaîne de caractère indiquant les spécificités de la forme composée.

- [FormeComposee](#) & [operator+](#) (const [Forme](#) &f)

Permet d'ajouter une forme à la forme composée.

- [FormeComposee](#) & [operator-](#) (int i)

Retourne l'aire de la forme composée.

- [Forme](#) * [operator\[\]](#) (int i) const

Retourne une forme à l'indice i parmi les formes de la forme composée.

- [FormeComposee](#) & [accept](#) ([FormeVisiteur](#) &visiteur) override

Permet l'introduction de méthode via le design pattern visitor.

Fonctions membres publiques hérités de [Forme](#)

- [Forme](#) (const [Couleur](#) &couleur=[Couleur::BLACK](#))

Construit une forme à partir d'une couleur.

- virtual [~Forme](#) ()

Permet la destruction de la forme.

- [Couleur](#) [getCouleur](#) () const

Retourne la couleur de la forme courante.

- [Forme](#) & [setCouleur](#) (const [Couleur](#) &couleur)

Permet de changer la couleur de la forme.

- [operator string](#) () const

Permet d'obtenir une chaîne de caractère indiquant les spécificités de la forme.

Fonctions membres privées

- void `copy` (const `FormeComposee` &*f*)

*Permet de copier une forme *f* pour l'objet implicite.*

- void `destroy` ()

Permet la destruction des champs de l'objet implicite.

Fonctions membres privées statiques

- static void `verificationAccesVector` (const vector< `Forme` * > &*v*, int *i*)

*Verifie si un entier *i* permet l'accès à un élément d'un vecteur *v*.*

- static void `verificationAccesVectorAddForme` (const vector< `Forme` * > &*v*, int *i*)

*Verifie si un entier *i* permet l'accès à un élément d'un vecteur *v* lors de l'ajout d'une forme.*

Attributs privés

- vector< `Forme` * > *v*

L'ensemble des formes.

Membres hérités additionnels

Fonctions membres publiques statiques hérités de `Forme`

- static `Forme` * `charger` (const string &*path*)

Permet de charger une forme à partir d'un fichier local.

Fonctions membres protégées statiques hérités de `Forme`

- static void `verificationKIsNotNull` (const double &*k*)

*Verifie si la constante *k* pour l'homothétie n'est pas nulle.*

6.33.1 Description détaillée

Permet de gérer une forme composée.

6.33.2 Documentation des constructeurs et destructeur

6.33.2.1 `FormeComposee()` [1/3]

```
FormeComposee::FormeComposee (
    const Couleur & couleur) [explicit]
```

Construit une forme composée à partir d'une couleur.

Paramètres

in	<code>couleur</code>	La couleur souhaitée
----	----------------------	----------------------

Renvoie

Une forme composée

6.33.2.2 `FormeComposee()` [2/3]

```
FormeComposee::FormeComposee (
    const vector< Forme * > & v = vector<Forme *>(),
    const Couleur & couleur = Couleur::BLACK) [explicit]
```

Construit une forme composée à partir d'une couleur et d'un vecteur de forme.

Paramètres

in	<code>v</code>	Le vecteur de formes
in	<code>couleur</code>	La couleur souhaitée (à défaut noir)

Renvoie

Une forme composée

Chaque forme ne doit pas avoir de parent, et chaque forme ne doit pas être un antécédent de la forme composée.

6.33.2.3 `FormeComposee()` [3/3]

```
FormeComposee::FormeComposee (
    const FormeComposee & f)
```

Construit une forme composée à partir d'une autre forme composée.

Paramètres

in	<code>f</code>	Une forme composée
----	----------------	--------------------

Renvoie

Une forme composée

6.33.2.4 `~FormeComposee()`

```
FormeComposee::~~FormeComposee () [override]
```

Permet la destruction de la forme composée.

6.33.3 Documentation des fonctions membres

6.33.3.1 `accept()`

```
FormeComposee & FormeComposee::accept (
    FormeVisiteur & visiteur) [override], [virtual]
```

Permet l'introduction de méthode via le design pattern visitor.

Paramètres

in	<i>visiteur</i>	Un objet répondant au pattern visitor
----	-----------------	---------------------------------------

Renvoie

(*this) L'objet implicite

Implémente [Forme](#).

6.33.3.2 `addForme()`

```
FormeComposee & FormeComposee::addForme (
    const Forme & f)
```

Permet d'ajouter une forme à la forme composée.

Paramètres

in	<i>f</i>	La forme
----	----------	----------

Renvoie

(*this) L'objet implicite

La forme ne doit pas avoir de parent, et la forme ne doit pas être un antécédent de la forme composée.

6.33.3.3 addFormeAt()

```
FormeComposee & FormeComposee::addFormeAt (
    const Forme & f,
    int i)
```

Permet d'ajouter une forme à la forme composée à la position i.

Paramètres

in	<i>f</i>	La forme
in	<i>i</i>	L'indice

Renvoie

(*this) L'objet implicite

La forme ne doit pas avoir de parent, et la forme ne doit pas être un antécédent de la forme composée.

La position doit respecter la taille du vecteur

6.33.3.4 clone()

```
FormeComposee * FormeComposee::clone () const [override], [virtual]
```

Retourne un clone de la forme composée.

Renvoie

Un clone de la forme composée

Implémente [Forme](#).

6.33.3.5 copy()

```
void FormeComposee::copy (
    const FormeComposee & f) [private]
```

Permet de copier une forme f pour l'objet implicite.

Paramètres

in	<i>f</i>	Une forme composée
----	----------	--------------------

Renvoie

void

6.33.3.6 `destroy()`

```
void FormeComposee::destroy () [private]
```

Permet la destruction des champs de l'objet implicite.

Renvoie

`void`

6.33.3.7 `getAire()`

```
double FormeComposee::getAire () const [override], [virtual]
```

Retourne l'aire de la forme composée.

Renvoie

L'aire de la forme composée

Implémente [Forme](#).

6.33.3.8 `getFormeAt()`

```
Forme * FormeComposee::getFormeAt (
    int i) const
```

Retourne une forme à l'indice `i` parmi les formes de la forme composée.

Paramètres

<code>in</code>	<code>i</code>	Un indice
-----------------	----------------	-----------

Renvoie

Un pointeur de forme

La position doit respecter la taille du vecteur

6.33.3.9 `getFormes()`

```
vector< Forme * > FormeComposee::getFormes () const
```

Retourne l'ensemble des formes de la forme composée.

Renvoie

L'ensemble des formes

6.33.3.10 getPoints()

```
vector< Vecteur2D > FormeComposee::getPoints () const [override], [virtual]
```

Permet d'obtenir l'ensemble des points de la forme.

Renvoie

L'ensemble des points

Implémente [Forme](#).

6.33.3.11 homothetie()

```
FormeComposee & FormeComposee::homothetie (
    const Vecteur2D & p,
    const double & k) [override], [virtual]
```

Permet de réaliser une homothétie de la forme composée.

Renvoie

(*this) L'objet implicite

Implémente [Forme](#).

6.33.3.12 operator+()

```
FormeComposee & FormeComposee::operator+ (
    const Forme & f)
```

Permet d'ajouter une forme à la forme composée.

Paramètres

in	<i>f</i>	La forme
----	----------	----------

Renvoie

(*this) L'objet implicite

La forme ne doit pas avoir de parent, et la forme ne doit pas être un antécédent de la forme composée.

6.33.3.13 operator-()

```
FormeComposee & FormeComposee::operator- (
    int i)
```

Retourne l'aire de la forme composée.

Renvoie

L'aire de la forme composée

6.33.3.14 `operator=()`

```
FormeComposee & FormeComposee::operator= (
    const FormeComposee & f)
```

Permet la copie des champs de `f` dans l'objet implicite.

Paramètres

in	<i>f</i>	Une forme composée
----	----------	--------------------

Renvoie

(`*this`) l'objet implicite

6.33.3.15 `operator[]()`

```
Forme * FormeComposee::operator[] (
    int i) const
```

Retourne une forme à l'indice `i` parmi les formes de la forme composée.

Paramètres

in	<i>i</i>	Un indice
----	----------	-----------

Renvoie

Un pointeur de forme

La position doit respecter la taille du vecteur

6.33.3.16 `removeFormeAt()`

```
FormeComposee & FormeComposee::removeFormeAt (
    int i)
```

Permet de retirer une forme à la position `i` du vecteur de la forme composée.

Paramètres

in	<i>i</i>	La position
----	----------	-------------

Renvoie

(`*this`) L'objet implicite

La position doit respecter la taille du vecteur

6.33.3.17 rotation()

```
FormeComposee & FormeComposee::rotation (
    const Vecteur2D & p,
    const double & r) [override], [virtual]
```

Permet de réaliser une rotation de la forme composée.

Renvoie

(*this) L'objet implicite

Implémente [Forme](#).

6.33.3.18 setFormes()

```
FormeComposee & FormeComposee::setFormes (
    const vector< Forme * > & vf)
```

Attribue un ensemble de forme à la forme composée.

Paramètres

in	vf	Un vecteur de forme
----	----	---------------------

Renvoie

(*this) l'objet implicite

Chaque forme ne doit pas avoir de parent, et chaque forme ne doit pas être un antécédent de la forme composée.

6.33.3.19 toString()

```
string FormeComposee::toString () const [override], [virtual]
```

Permet d'obtenir une chaîne de caractère indiquant les spécificités de la forme composée.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [Forme](#).

6.33.3.20 `transformationEcran()`

```
FormeComposee & FormeComposee::transformationEcran (
    const Matrice22 & m,
    const Vecteur2D & ab) [override], [virtual]
```

Permet de réaliser une transformation de la forme simple pour un affichage sur écran.

Paramètres

in	<i>m</i>	Une matrice
in	<i>ab</i>	Un vecteur

Renvoie

(*this) L'objet implicite

Implémente [Forme](#).

6.33.3.21 `translation()`

```
FormeComposee & FormeComposee::translation (
    const Vecteur2D & p) [override], [virtual]
```

Permet de réaliser une translation de la forme composée.

Renvoie

(*this) L'objet implicite

Implémente [Forme](#).

6.33.3.22 `verificationAccesVector()`

```
void FormeComposee::verificationAccesVector (
    const vector< Forme * > & v,
    int i) [static], [private]
```

Verifie si un entier *i* permet l'accès à un élément d'un vecteur *v*.

Paramètres

in	<i>v</i>	Un vector
in	<i>i</i>	Un entier

Renvoie

void

6.33.3.23 verificationAccesVectorAddForme()

```
void FormeComposee::verificationAccesVectorAddForme (
    const vector< Forme * > & v,
    int i) [static], [private]
```

Verifie si un entier `i` permet l'accès à un élément d'un vecteur `v` lors de l'ajout d'une forme.

Paramètres

in	v	Un vector
in	<code>i</code>	Un entier

Renvoie

void

6.33.4 Documentation des données membres

6.33.4.1 v

```
vector<Forme *> FormeComposee::v [private]
```

L'ensemble des formes.

La documentation de cette classe a été générée à partir des fichiers suivants :

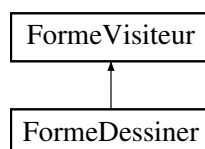
- [FormeComposee.h](#)
- [FormeComposee.cpp](#)

6.34 Référence de la classe FormeDessiner

Permet l'implémentation du pattern design visiteur pour les formes dans le cadre d'un dessin.

```
#include <FormeDessiner.h>
```

Graphe d'héritage de FormeDessiner:



Fonctions membres publiques— **FormeDessiner** ()

Permet la construction du Visiteur permettant le dessin d'un objet.

— **~FormeDessiner** () override

Permet la destruction de l'objet.

— **FormeDessiner** & **visit** (const **Segment** &f) override

Permet l'introduction de méthode pour un segment via le design pattern visitor, ici le dessin.

— **FormeDessiner** & **visit** (const **Cercle** &f) override

Permet l'introduction de méthode pour un cercle via le design pattern visitor, ici le dessin.

— **FormeDessiner** & **visit** (const **Triangle** &f) override

Permet l'introduction de méthode pour un triangle via le design pattern visitor, ici le dessin.

— **FormeDessiner** & **visit** (const **Polygone** &f) override

Permet l'introduction de méthode pour un polygone via le design pattern visitor, ici le dessin.

— **FormeDessiner** & **visit** (const **FormeComposee** &f) override

Permet l'introduction de méthode pour une forme composée via le design pattern visitor, ici le dessin.

Fonctions membres publiques hérités de **FormeVisiteur**— virtual **~FormeVisiteur** ()=default

Permet la destruction du visiteur de forme simple.

Fonctions membres privées— **FormeDessiner** & **traiterFormePourRequete** (const **Forme** &f)

Permet l'envoi d'une forme pour la dessiner.

— **FormeDessiner** & **preTransformationPassageEcran** (**Forme** &f)

Permet le prétraitement d'une forme afin que celle-ci soit adaptée pour le dessin.

— **FormeDessiner** & **envoyerRequete** (const string &requete)

Permet l'envoi d'un message au serveur.

Attributs privés— SOCKET **sock**

Socket.

— SOCKADDR_IN `sockaddr` {}

Paramétrage de la socket.

Attributs privés statiques

— static const string `adress` = "127.0.0.1"

Adresse IP du serveur.

— static constexpr int `port` = 4567

Port du serveur.

6.34.1 Description détaillée

Permet l'implémentation du pattern design visiteur pour les formes dans le cadre d'un dessin.

6.34.2 Documentation des constructeurs et destructeur

6.34.2.1 FormeDessiner()

```
FormeDessiner::FormeDessiner () [explicit]
```

Permet la construction du Visiteur permettant le dessin d'un objet.

6.34.2.2 ~FormeDessiner()

```
FormeDessiner::~FormeDessiner () [override]
```

Permet la destruction de l'objet.

6.34.3 Documentation des fonctions membres

6.34.3.1 envoyerRequete()

```
FormeDessiner & FormeDessiner::envoyerRequete (
    const string & requete) [private]
```

Permet l'envoi d'un message au serveur.

Paramètres

in	<i>requete</i>	Une chaîne de caractère
----	----------------	-------------------------

Renvoie

(*this) L'objet implicite

6.34.3.2 `preTransformationPassageEcran()`

```
FormeDessiner & FormeDessiner::preTransformationPassageEcran (  
    Forme & f) [private]
```

Permet le prétraitement d'une forme afin que celle-ci soit adaptée pour le dessin.

Paramètres

in	f	Une forme
----	---	-----------

Renvoie

(*this) L'objet implicite

6.34.3.3 `traiterFormePourRequete()`

```
FormeDessiner & FormeDessiner::traiterFormePourRequete (  
    const Forme & f) [private]
```

Permet l'envoi d'une forme pour la dessiner.

Paramètres

in	f	Une forme
----	---	-----------

Renvoie

(*this) L'objet implicite

6.34.3.4 `visit()` [1/5]

```
FormeDessiner & FormeDessiner::visit (  
    const Cercle & f) [override], [virtual]
```

Permet l'introduction de méthode pour un cercle via le design pattern visitor, ici le dessin.

Paramètres

in	f	Un cercle
----	---	-----------

Renvoie

(*this) L'objet implicite

Implémente [FormeVisiteur](#).

6.34.3.5 visit() [2/5]

```
FormeDessiner & FormeDessiner::visit (
    const FormeComposee & f) [override], [virtual]
```

Permet l'introduction de méthode pour une forme composée via le design pattern visitor, ici le dessin.

Paramètres

in	f	Une forme composée
----	---	--------------------

Renvoie

(*this) L'objet implicite

Implémente [FormeVisiteur](#).

6.34.3.6 visit() [3/5]

```
FormeDessiner & FormeDessiner::visit (
    const Polygone & f) [override], [virtual]
```

Permet l'introduction de méthode pour un polygone via le design pattern visitor, ici le dessin.

Paramètres

in	f	Un polygone
----	---	-------------

Renvoie

(*this) L'objet implicite

Implémente [FormeVisiteur](#).

6.34.3.7 visit() [4/5]

```
FormeDessiner & FormeDessiner::visit (
    const Segment & f) [override], [virtual]
```

Permet l'introduction de méthode pour un segment via le design pattern visitor, ici le dessin.

Paramètres

in	f	Un segment
----	---	------------

Renvoie

(*this) L'objet implicite

Implémente [FormeVisiteur](#).

6.34.3.8 `visit()` [5/5]

```
FormeDessiner & FormeDessiner::visit (  
    const Triangle & f) [override], [virtual]
```

Permet l'introduction de méthode pour un triangle via le design pattern visitor, ici le dessin.

Paramètres

in	f	Un triangle
----	---	-------------

Renvoie

(*this) L'objet implicite

Implémente [FormeVisiteur](#).

6.34.4 Documentation des données membres

6.34.4.1 `adress`

```
const string FormeDessiner::adress = "127.0.0.1" [static], [private]
```

Adresse IP du serveur.

6.34.4.2 `port`

```
int FormeDessiner::port = 4567 [static], [constexpr], [private]
```

Port du serveur.

6.34.4.3 `sock`

```
SOCKET FormeDessiner::sock [private]
```

Socket.

6.34.4.4 `sockaddr`

```
SOCKADDR_IN FormeDessiner::sockaddr {} [private]
```

Paramétrage de la socket.

La documentation de cette classe a été générée à partir des fichiers suivants :

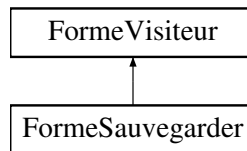
- [FormeDessiner.h](#)
- [FormeDessiner.cpp](#)

6.35 Référence de la classe **FormeSauvegarder**

Permet l'implémentation du pattern design visiteur pour les formes dans le cadre d'une sauvegarde.

```
#include <FormeSauvegarder.h>
```

Graphe d'héritage de **FormeSauvegarder**:



Fonctions membres publiques

- **FormeSauvegarder** (string **path**)

Permet la construction du Visiteur permettant la sauvegarde d'un objet à partir d'un nom de fichier.

- **~FormeSauvegarder** () override

Permet la destruction de l'objet.

- **FormeSauvegarder** & **visit** (const **Segment** &f) override

Permet l'introduction de méthode pour un segment via le design pattern visitor, ici la sauvegarde.

- **FormeSauvegarder** & **visit** (const **Cercle** &f) override

Permet l'introduction de méthode pour un cercle via le design pattern visitor, ici la sauvegarde.

- **FormeSauvegarder** & **visit** (const **Triangle** &f) override

Permet l'introduction de méthode pour un triangle via le design pattern visitor, ici la sauvegarde.

- **FormeSauvegarder** & **visit** (const **Polygone** &f) override

Permet l'introduction de méthode pour un polygone via le design pattern visitor, ici la sauvegarde.

- **FormeSauvegarder** & **visit** (const **FormeComposee** &f) override

Permet l'introduction de méthode pour une forme composée via le design pattern visitor, ici la sauvegarde.

Fonctions membres publiques hérités de **FormeVisiteur**

- virtual **~FormeVisiteur** ()=default

Permet la destruction du visiteur de forme simple.

Fonctions membres privées

- **FormeSauvegarder** & **sauvegarderForme** (const string &formeToString)

Permet la sauvegarde d'une forme à partir de sa chaîne de caractère.

Attributs privés

— `const string path`

Le path du fichier où sera enregistrée la forme.

6.35.1 Description détaillée

Permet l'implémentation du pattern design visiteur pour les formes dans le cadre d'une sauvegarde.

6.35.2 Documentation des constructeurs et destructeur

6.35.2.1 `FormeSauvegarder()`

```
FormeSauvegarder::FormeSauvegarder (  
    string path) [explicit]
```

Permet la construction du Visiteur permettant la sauvegarde d'un objet à partir d'un nom de fichier.

Paramètres

in	<code>path</code>	Le nom du fichier
----	-------------------	-------------------

6.35.2.2 `~FormeSauvegarder()`

```
FormeSauvegarder::~~FormeSauvegarder () [override], [default]
```

Permet la destruction de l'objet.

6.35.3 Documentation des fonctions membres

6.35.3.1 `sauvegarderForme()`

```
FormeSauvegarder & FormeSauvegarder::sauvegarderForme (  
    const string & formeToString) [private]
```

Permet la sauvegarde d'une forme à partir de sa chaîne de caractère.

Paramètres

in	<code>formeToString</code>	La chaîne de caractère de la forme
----	----------------------------	------------------------------------

Renvoie

`(*this)` L'objet implicite

6.35.3.2 visit() [1/5]

```
FormeSauvegarder & FormeSauvegarder::visit (
    const Cercle & f) [override], [virtual]
```

Permet l'introduction de méthode pour un cercle via le design pattern visitor, ici la sauvegarde.

Paramètres

in	f	Un cercle
----	---	-----------

Renvoie

(*this) L'objet implicite

Implémente [FormeVisiteur](#).

6.35.3.3 visit() [2/5]

```
FormeSauvegarder & FormeSauvegarder::visit (
    const FormeComposee & f) [override], [virtual]
```

Permet l'introduction de méthode pour une forme composée via le design pattern visitor, ici la sauvegarde.

Paramètres

in	f	Une forme composée
----	---	--------------------

Renvoie

(*this) L'objet implicite

Implémente [FormeVisiteur](#).

6.35.3.4 visit() [3/5]

```
FormeSauvegarder & FormeSauvegarder::visit (
    const Polygone & f) [override], [virtual]
```

Permet l'introduction de méthode pour un polygone via le design pattern visitor, ici la sauvegarde.

Paramètres

in	f	Un polygone
----	---	-------------

Renvoie

(*this) L'objet implicite

Implémente [FormeVisiteur](#).

6.35.3.5 `visit()` [4/5]

```
FormeSauvegarder & FormeSauvegarder::visit (  
    const Segment & f) [override], [virtual]
```

Permet l'introduction de méthode pour un segment via le design pattern visitor, ici la sauvegarde.

Paramètres

in	f	Un segment
----	---	------------

Renvoie

(*this) L'objet implicite

Implémente [FormeVisiteur](#).

6.35.3.6 `visit()` [5/5]

```
FormeSauvegarder & FormeSauvegarder::visit (  
    const Triangle & f) [override], [virtual]
```

Permet l'introduction de méthode pour un triangle via le design pattern visitor, ici la sauvegarde.

Paramètres

in	f	Un triangle
----	---	-------------

Renvoie

(*this) L'objet implicite

Implémente [FormeVisiteur](#).

6.35.4 Documentation des données membres

6.35.4.1 `path`

```
const string FormeSauvegarder::path [private]
```

Le path du fichier où sera enregistrée la forme.

La documentation de cette classe a été générée à partir des fichiers suivants :

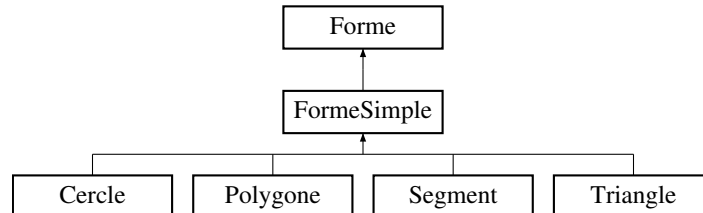
- [FormeSauvegarder.h](#)
- [FormeSauvegarder.cpp](#)

6.36 Référence de la classe FormeSimple

Permet de gérer une forme simple.

```
#include <FormeSimple.h>
```

Graphe d'héritage de FormeSimple:



Fonctions membres publiques

— `FormeSimple (const Couleur &couleur=Couleur::BLACK)`

Construit une forme simple à partir d'une couleur.

— `FormeSimple * clone () const override=0`

Retourne un clone de la forme simple.

— `~FormeSimple () override`

Permet la destruction de la forme simple.

— `virtual FormeSimple & setPoints (const vector< Vecteur2D > &v)=0`

Permet de modifier l'ensemble des points de la forme simple.

— `double getAire () const override=0`

Permet d'obtenir l'aire d'une forme simple.

— `FormeSimple & translation (const Vecteur2D &p) override`

Permet de réaliser une translation de la forme simple.

— `FormeSimple & homothetie (const Vecteur2D &p, const double &k) override`

Permet de réaliser une homothétie de la forme simple.

— `FormeSimple & rotation (const Vecteur2D &p, const double &r) override`

Permet de réaliser une rotation de la forme simple.

— `FormeSimple & transformationEcran (const Matrice22 &m, const Vecteur2D &ab) override`

Permet de réaliser une transformation de la forme simple pour un affichage sur écran.

— `FormeSimple & accept (FormeVisiteur &visiteur) override=0`

Permet l'introduction de méthode via le design pattern visitor.

Fonctions membres publiques hérités de `Forme`

- `Forme` (const `Couleur` &couleur=`Couleur::BLACK`)
Construit une forme à partir d'une couleur.
- virtual `~Forme` ()
Permet la destruction de la forme.
- `Couleur` `getCouleur` () const
Retourne la couleur de la forme courante.
- `Forme` & `setCouleur` (const `Couleur` &couleur)
Permet de changer la couleur de la forme.
- virtual vector< `Vecteur2D` > `getPoints` () const =0
Permet d'obtenir l'ensemble des points de la forme.
- virtual string `toString` () const
Permet d'obtenir une chaîne de caractère indiquant les spécificités de la forme.
- `operator string` () const
Permet d'obtenir une chaîne de caractère indiquant les spécificités de la forme.

Fonctions membres protégées statiques

- static void `verificationNonEgaliteVecteur2D` (const `Vecteur2D` &a, const `Vecteur2D` &b)
Verifie si un vecteur a n'est pas égal à un vecteur b .
- static void `verificationTailleVector` (const vector< `Vecteur2D` > &v, int i)
Verifie si un entier i correspond à la taille d'un vector v .
- static void `verificationAccesVector` (const vector< `Vecteur2D` > &v, int i)
Verifie si un entier i permet l'accès à un élément d'un vecteur v .
- static void `verificationAccesVectorAddPoint` (const vector< `Vecteur2D` > &v, int i)
Verifie si un entier i permet l'accès à un élément d'un vecteur v lors de l'ajout d'un point.

Fonctions membres protégées statiques hérités de `Forme`

- static void `verificationKIsNotNull` (const double &k)
Verifie si la constante k pour l'homothétie n'est pas nulle.

Membres hérités additionnels

Fonctions membres publiques statiques hérités de [Forme](#)

— static [Forme](#) * [charger](#) (const string &path)

Permet de charger une forme à partir d'un fichier local.

6.36.1 Description détaillée

Permet de gérer une forme simple.

6.36.2 Documentation des constructeurs et destructeur

6.36.2.1 [FormeSimple\(\)](#)

```
FormeSimple::FormeSimple (
    const Couleur & couleur = Couleur::BLACK) [explicit]
```

Construit une forme simple à partir d'une couleur.

Paramètres

in	couleur	La couleur souhaitée (à défaut noir)
----	-------------------------	--------------------------------------

Renvoie

Une forme simple

6.36.2.2 [~FormeSimple\(\)](#)

```
FormeSimple::~~FormeSimple () [override], [default]
```

Permet la destruction de la forme simple.

6.36.3 Documentation des fonctions membres

6.36.3.1 `accept()`

```
FormeSimple & FormeSimple::accept (
    FormeVisiteur & visiteur) [override], [pure virtual]
```

Permet l'introduction de méthode via le design pattern visitor.

Paramètres

in	<i>visiteur</i>	Un objet répondant au pattern visitor
----	-----------------	---------------------------------------

Renvoie

(*this) L'objet implicite

Implémente [Forme](#).

Implémenté dans [Cercle](#), [Polygone](#), [Segment](#), et [Triangle](#).

6.36.3.2 `clone()`

```
FormeSimple * FormeSimple::clone () const [override], [pure virtual]
```

Retourne un clone de la forme simple.

Renvoie

Un clone de la forme simple

Implémente [Forme](#).

Implémenté dans [Cercle](#), [Polygone](#), [Segment](#), et [Triangle](#).

6.36.3.3 `getAire()`

```
double FormeSimple::getAire () const [override], [pure virtual]
```

Permet d'obtenir l'aire d'une forme simple.

Renvoie

L'aire de la forme simple

Implémente [Forme](#).

Implémenté dans [Cercle](#), [Polygone](#), [Segment](#), et [Triangle](#).

6.36.3.4 homothetie()

```
FormeSimple & FormeSimple::homothetie (
    const Vecteur2D & p,
    const double & k) [override], [virtual]
```

Permet de réaliser une homothétie de la forme simple.

Renvoie

(*this) L'objet implicite

Implémente [Forme](#).

6.36.3.5 rotation()

```
FormeSimple & FormeSimple::rotation (
    const Vecteur2D & p,
    const double & r) [override], [virtual]
```

Permet de réaliser une rotation de la forme simple.

Renvoie

(*this) L'objet implicite

Implémente [Forme](#).

6.36.3.6 setPoints()

```
virtual FormeSimple & FormeSimple::setPoints (
    const vector< Vecteur2D > & v) [pure virtual]
```

Permet de modifier l'ensemble des points de la forme simple.

Renvoie

(*this) L'objet implicite

Implémenté dans [Cercle](#), [Polygone](#), [Segment](#), et [Triangle](#).

6.36.3.7 transformationEcran()

```
FormeSimple & FormeSimple::transformationEcran (
    const Matrice22 & m,
    const Vecteur2D & ab) [override], [virtual]
```

Permet de réaliser une transformation de la forme simple pour un affichage sur écran.

Paramètres

in	<i>m</i>	Une matrice
in	<i>ab</i>	Un vecteur

Renvoie

(*this) L'objet implicite

Implémente [Forme](#).

6.36.3.8 translation()

```
FormeSimple & FormeSimple::translation (
    const Vecteur2D & p) [override], [virtual]
```

Permet de réaliser une translation de la forme simple.

Renvoie

(*this) L'objet implicite

Implémente [Forme](#).

6.36.3.9 verificationAccesVector()

```
void FormeSimple::verificationAccesVector (
    const vector< Vecteur2D > & v,
    int i) [static], [protected]
```

Verifie si un entier *i* permet l'accès à un élément d'un vecteur *v*.

Paramètres

in	<i>v</i>	Un vector
in	<i>i</i>	Un entier

Renvoie

void

6.36.3.10 verificationAccesVectorAddPoint()

```
void FormeSimple::verificationAccesVectorAddPoint (
    const vector< Vecteur2D > & v,
    int i) [static], [protected]
```

Verifie si un entier *i* permet l'accès à un élément d'un vecteur *v* lors de l'ajout d'un point.

Paramètres

in	<i>v</i>	Un vector
in	<i>i</i>	Un entier

Renvoie

void

6.36.3.11 verificationNonEgaliteVecteur2D()

```
void FormeSimple::verificationNonEgaliteVecteur2D (
    const Vecteur2D & a,
    const Vecteur2D & b) [static], [protected]
```

Verifie si un vecteur *a* n'est pas égal à un vecteur *b*.

Paramètres

in	<i>a</i>	Un vecteur
in	<i>b</i>	Un vecteur

Renvoie

void

6.36.3.12 verificationTailleVector()

```
void FormeSimple::verificationTailleVector (
    const vector< Vecteur2D > & v,
    int i) [static], [protected]
```

Verifie si un entier *i* correspond à la taille d'un vector *v*.

Paramètres

in	<i>v</i>	Un vector
in	<i>i</i>	Un entier

Renvoie

void

La documentation de cette classe a été générée à partir des fichiers suivants :

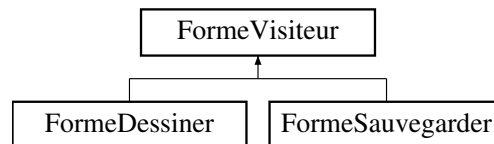
- [FormeSimple.h](#)
- [FormeSimple.cpp](#)

6.37 Référence de la classe FormeVisiteur

Permet l'implémentation du pattern design visiteur pour les formes.

```
#include <FormeVisiteur.h>
```

Graphe d'héritage de FormeVisiteur:



Fonctions membres publiques

— virtual `~FormeVisiteur()`=default

Permet la destruction du visiteur de forme simple.

— virtual `FormeVisiteur & visit (const Segment &f)=0`

Permet l'introduction de méthode pour un segment via le design pattern visitor.

— virtual `FormeVisiteur & visit (const Cercle &f)=0`

Permet l'introduction de méthode pour un cercle via le design pattern visitor.

— virtual `FormeVisiteur & visit (const Triangle &f)=0`

Permet l'introduction de méthode pour un triangle via le design pattern visitor.

— virtual `FormeVisiteur & visit (const Polygone &f)=0`

Permet l'introduction de méthode pour un polygone via le design pattern visitor.

— virtual `FormeVisiteur & visit (const FormeComposee &f)=0`

Permet l'introduction de méthode pour une forme composée via le design pattern visitor.

6.37.1 Description détaillée

Permet l'implémentation du pattern design visiteur pour les formes.

6.37.2 Documentation des constructeurs et destructeur

6.37.2.1 `~FormeVisiteur()`

```
virtual FormeVisiteur::~~FormeVisiteur () [virtual], [default]
```

Permet la destruction du visiteur de forme simple.

6.37.3 Documentation des fonctions membres

6.37.3.1 visit() [1/5]

```
virtual FormeVisiteur & FormeVisiteur::visit (  
    const Cercle & f) [pure virtual]
```

Permet l'introduction de méthode pour un cercle via le design pattern visitor.

Paramètres

in	f	Un cercle
----	---	-----------

Renvoie

(*this) L'objet implicite

Implémenté dans [FormeDessiner](#), et [FormeSauvegarder](#).

6.37.3.2 visit() [2/5]

```
virtual FormeVisiteur & FormeVisiteur::visit (  
    const FormeComposee & f) [pure virtual]
```

Permet l'introduction de méthode pour une forme composée via le design pattern visitor.

Paramètres

in	f	Une forme composée
----	---	--------------------

Renvoie

(*this) L'objet implicite

Implémenté dans [FormeDessiner](#), et [FormeSauvegarder](#).

6.37.3.3 visit() [3/5]

```
virtual FormeVisiteur & FormeVisiteur::visit (  
    const Polygone & f) [pure virtual]
```

Permet l'introduction de méthode pour un polygone via le design pattern visitor.

Paramètres

in	f	Un polygone
----	---	-------------

Renvoie

(*this) L'objet implicite

Implémenté dans [FormeDessiner](#), et [FormeSauvegarder](#).

6.37.3.4 visit() [4/5]

```
virtual FormeVisiteur & FormeVisiteur::visit (  
    const Segment & f) [pure virtual]
```

Permet l'introduction de méthode pour un segment via le design pattern visitor.

Paramètres

in	f	Un segment
----	---	------------

Renvoie

(*this) L'objet implicite

Implémenté dans [FormeDessiner](#), et [FormeSauvegarder](#).

6.37.3.5 visit() [5/5]

```
virtual FormeVisiteur & FormeVisiteur::visit (  
    const Triangle & f) [pure virtual]
```

Permet l'introduction de méthode pour un triangle via le design pattern visitor.

Paramètres

in	f	Un triangle
----	---	-------------

Renvoie

(*this) L'objet implicite

Implémenté dans [FormeDessiner](#), et [FormeSauvegarder](#).

La documentation de cette classe a été générée à partir du fichier suivant :

— [FormeVisiteur.h](#)

6.38 Référence de la classe Matrice22

Permet de gérer des matrices de taille 2 * 2.

```
#include <Matrice22.h>
```

Fonctions membres publiques

- [Matrice22](#) (const [Vecteur2D](#) &ligneHaut, const [Vecteur2D](#) &ligneBas)

Construit une matrice à partir d'un vecteur `ligneHaut` et d'un vecteur `ligneBas`.

- [Matrice22](#) (const double &a11=1, const double &a12=0, const double &a21=0, const double &a22=1)

Construit une matrice à partir d'un double `a11`, d'un double `a12`, d'un double `a21`, et d'un double `a22`.

- [operator string](#) () const

Permet d'obtenir une chaîne de caractère indiquant les spécificités de la matrice.

- [Vecteur2D operator*](#) (const [Vecteur2D](#) &v) const

Realise la multiplication d'une matrice par rapport à un vecteur.

Fonctions membres publiques statiques

- static [Matrice22 creeRotation](#) (const double &alpha)

Construit une matrice utile à la réalisation d'une rotation.

Attributs publics

- [Vecteur2D ligneHaut](#)

Les deux vecteurs représentatifs de la matrice.

- [Vecteur2D ligneBas](#)

6.38.1 Description détaillée

Permet de gérer des matrices de taille 2×2 .

6.38.2 Documentation des constructeurs et destructeur

6.38.2.1 [Matrice22\(\)](#) [1/2]

```
Matrice22::Matrice22 (
    const Vecteur2D & ligneHaut,
    const Vecteur2D & ligneBas) [inline]
```

Construit une matrice à partir d'un vecteur `ligneHaut` et d'un vecteur `ligneBas`.

Paramètres

in	ligneHaut	Un vecteur 2D
----	---------------------------	---------------

in	<i>ligneBas</i>	Un vecteur 2D
----	-----------------	---------------

Renvoie

Une matrice22

6.38.2.2 Matrice22() [2/2]

```
Matrice22::Matrice22 (
    const double & a11 = 1,
    const double & a12 = 0,
    const double & a21 = 0,
    const double & a22 = 1) [inline], [explicit]
```

Construit une matrice à partir d'un double a11, d'un double a12, d'un double a21, et d'un double a22.

Paramètres

in	<i>a11</i>	Un double (à défaut 1)
in	<i>a12</i>	Un double (à défaut 1)
in	<i>a21</i>	Un double (à défaut 1)
in	<i>a22</i>	Un double (à défaut 1)

Renvoie

Une matrice22

6.38.3 Documentation des fonctions membres

6.38.3.1 creeRotation()

```
Matrice22 Matrice22::creeRotation (
    const double & alpha) [inline], [static]
```

Construit une matrice utile à la réalisation d'une rotation.

Paramètres

in	<i>alpha</i>	Un double correspondant à un angle radian
----	--------------	---

Renvoie

Une matrice22

6.38.3.2 operator string()

```
Matrice22::operator string () const [inline], [explicit]
```

Permet d'obtenir une chaîne de caractère indiquant les spécificités de la matrice.

Renvoie

Une chaîne de caractère

6.38.3.3 operator*()

```
Vecteur2D Matrice22::operator* (  
    const Vecteur2D & v) const [inline]
```

Realise la multiplication d'une matrice par rapport à un vecteur.

Paramètres

in	v	Un vecteur2D
----	---	--------------

Renvoie

Un vecteur2D

6.38.4 Documentation des données membres

6.38.4.1 ligneBas

```
Vecteur2D Matrice22::ligneBas
```

6.38.4.2 ligneHaut

```
Vecteur2D Matrice22::ligneHaut
```

Les deux vecteurs représentatifs de la matrice.

La documentation de cette classe a été générée à partir du fichier suivant :

— [Matrice22.h](#)

6.39 Référence de la classe MaWinsock

Permet la gestion de la socket serveur.

```
#include <MaWinsock.h>
```

Fonctions membres publiques

— `~MaWinsock ()`

Permet la destruction de la socket.

Fonctions membres publiques statiques

— `static MaWinsock * getInstance ()`

Permet l'obtention de l'instance, et la crée si elle n'existe pas.

Fonctions membres privées

— `MaWinsock ()`

Permet la création de l'instance.

Attributs privés

— `WSADATA wsaData`

Attributs privés statiques

— `static MaWinsock * instanceUnique = nullptr`

L'instance unique (singleton).

6.39.1 Description détaillée

Permet la gestion de la socket serveur.

6.39.2 Documentation des constructeurs et destructeur

6.39.2.1 MaWinsock()

```
MaWinsock::MaWinsock () [explicit], [private]
```

Permet la création de l'instance.

6.39.2.2 ~MaWinsock()

```
MaWinsock::~~MaWinsock ()
```

Permet la destruction de la socket.

6.39.3 Documentation des fonctions membres

6.39.3.1 getInstance()

```
MaWinsock * MaWinsock::getInstance () [static]
```

Permet l'obtention de l'instance, et la crée si elle n'existe pas.

Renvoie

L'instance Winsock

6.39.4 Documentation des données membres

6.39.4.1 instanceUnique

```
MaWinsock * MaWinsock::instanceUnique = nullptr [static], [private]
```

L'instance unique (singleton).

6.39.4.2 wsaData

```
WSADATA MaWinsock::wsaData [private]
```

La documentation de cette classe a été générée à partir des fichiers suivants :

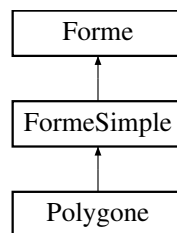
- [MaWinsock.h](#)
- [MaWinsock.cpp](#)

6.40 Référence de la classe Polygone

Permet de gérer du polygone.

```
#include <Polygone.h>
```

Graphe d'héritage de Polygone:



Fonctions membres publiques

- **Polygone** (const **Couleur** &couleur)

*Construit un **Polygone** à partir d'une couleur.*

- **Polygone** (const vector< **Vecteur2D** > &v={}, const **Couleur** &couleur=**Couleur::BLACK**)

*Construit un **Polygone** à partir d'une couleur et d'un vecteur de points.*

- **Polygone** * **clone** () const override

Retourne un clone du polygone.

- ~**Polygone** () override

Permet la destruction du polygone.

- vector< **Vecteur2D** > **getPoints** () const override

Permet d'obtenir l'ensemble des points du polygone.

- **Polygone** & **setPoints** (const vector< **Vecteur2D** > &v) override

Permet de modifier l'ensemble des points du polygone.

- **Vecteur2D** **getPointAt** (int i) const

Permet d'obtenir un point en fonction de sa position dans le vecteur du polygone.

- **Polygone** & **addPoint** (const **Vecteur2D** &p)

Permet d'ajouter un point à la fin du vecteur du polygone.

- **Polygone** & **addPointAt** (const **Vecteur2D** &p, int i)

Permet d'ajouter un point à la position i du vecteur du polygone.

- **Polygone** & **removePoint** (const **Vecteur2D** &p)

Permet de retirer un point du polygone.

- **Polygone** & **removePointAt** (int i)

Permet de retirer un point à la position i du vecteur du polygone.

- **Polygone** & **modifyPoint** (const **Vecteur2D** &p, const **Vecteur2D** &q)

Permet de modifier un point du polygone.

- **Polygone** & **modifyPointAt** (int i, const **Vecteur2D** &q)

Permet de modifier un point à la position i du vecteur du polygone.

- **Polygone** & **switchPoint** (const **Vecteur2D** &p, const **Vecteur2D** &q)

Permet d'inverser la position de deux points.

- **Polygone** & **switchPointAt** (int i, int j)

Permet d'inverser la position de deux points en fonction de leurs indices.

- double `getAire ()` const override

Retourne l'aire du polygone.

- string `toString ()` const override

Permet d'obtenir une chaîne de caractère indiquant les spécificités du polygone.

- `Polygone & operator+` (const `Vecteur2D` &p)

Permet d'ajouter un point au vecteur du polygone.

- `Polygone & operator-` (const `Vecteur2D` &p)

Permet de retirer un point au vecteur du polygone.

- `Vecteur2D operator[]` (int i) const

Permet d'obtenir un point en fonction de sa position dans le vecteur.

- `Polygone & accept (FormeVisiteur &visiteur)` override

Permet l'introduction de méthode via le design pattern visitor.

Fonctions membres publiques héritées de `FormeSimple`

- `FormeSimple` (const `Couleur` &couleur=`Couleur::BLACK`)

Construit une forme simple à partir d'une couleur.

- `~FormeSimple ()` override

Permet la destruction de la forme simple.

- `FormeSimple & translation` (const `Vecteur2D` &p) override

Permet de réaliser une translation de la forme simple.

- `FormeSimple & homothetie` (const `Vecteur2D` &p, const double &k) override

Permet de réaliser une homothétie de la forme simple.

- `FormeSimple & rotation` (const `Vecteur2D` &p, const double &r) override

Permet de réaliser une rotation de la forme simple.

- `FormeSimple & transformationEcran` (const `Matrice22` &m, const `Vecteur2D` &ab) override

Permet de réaliser une transformation de la forme simple pour un affichage sur écran.

Fonctions membres publiques héritées de `Forme`

- `Forme` (const `Couleur` &couleur=`Couleur::BLACK`)

Construit une forme à partir d'une couleur.

— virtual `~Forme()`

Permet la destruction de la forme.

— `Couleur` `getCouleur()` const

Retourne la couleur de la forme courante.

— `Forme` & `setCouleur` (const `Couleur` &`couleur`)

Permet de changer la couleur de la forme.

— `operator string` () const

Permet d'obtenir une chaîne de caractère indiquant les spécificités de la forme.

Fonctions membres privées statiques

— static void `verificationParametreVector` (const vector< `Vecteur2D` > &`v`)

Verifie dans un vector l'existence ou non de deux vecteurs identiques.

— static void `verificationVectorEtVecteur2D` (const vector< `Vecteur2D` > &`v`, const `Vecteur2D` &`p`)

Verifie dans un vector de `Vecteur2D` l'existence ou non d'un vecteur `p`.

Attributs privés

— vector< `Vecteur2D` > `v`

L'ensemble des points du polygone.

Membres hérités additionnels

Fonctions membres publiques statiques hérités de `Forme`

— static `Forme` * `charger` (const string &`path`)

Permet de charger une forme à partir d'un fichier local.

Fonctions membres protégées statiques hérités de `FormeSimple`

— static void `verificationNonEgaliteVecteur2D` (const `Vecteur2D` &`a`, const `Vecteur2D` &`b`)

Verifie si un vecteur `a` n'est pas égal à un vecteur `b`.

— static void `verificationTailleVector` (const vector< `Vecteur2D` > &`v`, int `i`)

Verifie si un entier `i` correspond à la taille d'un vector `v`.

— static void `verificationAccesVector` (const vector< `Vecteur2D` > &`v`, int `i`)

Verifie si un entier i permet l'accès à un élément d'un vecteur v .

— static void `verificationAccesVectorAddPoint` (const vector< `Vecteur2D` > & v , int i)

Verifie si un entier i permet l'accès à un élément d'un vecteur v lors de l'ajout d'un point.

Fonctions membres protégées statiques hérités de `Forme`

— static void `verificationKIsNotNull` (const double & k)

Verifie si la constante k pour l'homothétie n'est pas nulle.

6.40.1 Description détaillée

Permet de gérer du polygone.

6.40.2 Documentation des constructeurs et destructeur

6.40.2.1 `Polygone()` [1/2]

```
Polygone::Polygone (
    const Couleur & couleur) [explicit]
```

Construit un `Polygone` à partir d'une couleur.

Paramètres

in	<code>couleur</code>	La couleur souhaitée
----	----------------------	----------------------

Renvoie

Un polygone

6.40.2.2 `Polygone()` [2/2]

```
Polygone::Polygone (
    const vector< Vecteur2D > & v = {},
    const Couleur & couleur = Couleur::BLACK) [explicit]
```

Construit un `Polygone` à partir d'une couleur et d'un vecteur de points.

Paramètres

in	v	Le vecteur de points
in	<code>couleur</code>	La couleur souhaitée (à défaut noir)

Renvoie

Un polygone

Un point ne peut pas apparaître deux fois

6.40.2.3 ~Polygone()

```
Polygone::~~Polygone () [override], [default]
```

Permet la destruction du polygone.

6.40.3 Documentation des fonctions membres

6.40.3.1 accept()

```
Polygone & Polygone::accept (
    FormeVisiteur & visiteur) [override], [virtual]
```

Permet l'introduction de méthode via le design pattern visitor.

Paramètres

in	<i>visiteur</i>	Un objet répondant au pattern visitor
----	-----------------	---------------------------------------

Renvoie

(*this) L'objet implicite

Implémente [FormeSimple](#).

6.40.3.2 addPoint()

```
Polygone & Polygone::addPoint (
    const Vecteur2D & p)
```

Permet d'ajouter un point à la fin du vecteur du polygone.

Paramètres

in	<i>p</i>	Le point
----	----------	----------

Renvoie

(*this) L'objet implicite

Un point ne peut pas apparaître deux fois

6.40.3.3 addPointAt()

```
Polygone & Polygone::addPointAt (
    const Vecteur2D & p,
    int i)
```

Permet d'ajouter un point à la position i du vecteur du polygone.

Paramètres

in	p	Le point
in	i	La position

Renvoie

(*this) L'objet implicite

La position doit respecter la taille du vecteur

Un point ne peut pas apparaître deux fois

6.40.3.4 clone()

```
Polygone * Polygone::clone () const [override], [virtual]
```

Retourne un clone du polygone.

Renvoie

Un clone du polygone

Implémente [FormeSimple](#).

6.40.3.5 getAire()

```
double Polygone::getAire () const [override], [virtual]
```

Retourne l'aire du polygone.

Renvoie

L'aire du polygone

Implémente [FormeSimple](#).

6.40.3.6 getPointAt()

```
Vecteur2D Polygone::getPointAt (
    int i) const
```

Permet d'obtenir un point en fonction de sa position dans le vecteur du polygone.

Paramètres

in	<i>i</i>	Une position
----	----------	--------------

Renvoie

Le point à la position *i*

6.40.3.7 getPoints()

```
vector< Vecteur2D > Polygone::getPoints () const [override], [virtual]
```

Permet d'obtenir l'ensemble des points du polygone.

Renvoie

L'ensemble des points

Implémente [Forme](#).

6.40.3.8 modifyPoint()

```
Polygone & Polygone::modifyPoint (
    const Vecteur2D & p,
    const Vecteur2D & q)
```

Permet de modifier un point du polygone.

Paramètres

in	<i>p</i>	Le point à modifier
in	<i>q</i>	Le nouveau point

Renvoie

(*this) L'objet implicite

Un point ne peut pas apparaître deux fois

6.40.3.9 modifyPointAt()

```
Polygone & Polygone::modifyPointAt (
    int i,
    const Vecteur2D & q)
```

Permet de modifier un point à la position i du vecteur du polygone.

Paramètres

in	<i>i</i>	La position
in	<i>q</i>	Le nouveau point

Renvoie

(*this) L'objet implicite

Un point ne peut pas apparaître deux fois

6.40.3.10 operator+()

```
Polygone & Polygone::operator+ (
    const Vecteur2D & p)
```

Permet d'ajouter un point au vecteur du polygone.

Paramètres

in	<i>p</i>	Le point
----	----------	----------

Renvoie

(*this) L'objet implicite

Un point ne peut pas apparaître deux fois

6.40.3.11 operator-()

```
Polygone & Polygone::operator- (
    const Vecteur2D & p)
```

Permet de retirer un point au vecteur du polygone.

Paramètres

in	<i>p</i>	Le point
----	----------	----------

Renvoie

(*this) L'objet implicite

6.40.3.12 operator[]()

```
Vecteur2D Polygone::operator[] (
    int i) const
```

Permet d'obtenir un point en fonction de sa position dans le vecteur.

Paramètres

in	<i>i</i>	Une position
----	----------	--------------

Renvoie

Le point à la position *i*

6.40.3.13 removePoint()

```
Polygone & Polygone::removePoint (
    const Vecteur2D & p)
```

Permet de retirer un point du polygone.

Paramètres

in	<i>p</i>	Le point
----	----------	----------

Renvoie

(*this) L'objet implicite

6.40.3.14 removePointAt()

```
Polygone & Polygone::removePointAt (
    int i)
```

Permet de retirer un point à la position *i* du vecteur du polygone.

Paramètres

in	<i>i</i>	La position
----	----------	-------------

Renvoie

(*this) L'objet implicite

La position doit respecter la taille du vecteur

6.40.3.15 setPoints()

```
Polygone & Polygone::setPoints (
    const vector< Vecteur2D > & v) [override], [virtual]
```

Permet de modifier l'ensemble des points du polygone.

Paramètres

in	v	Un vecteur de points
----	-------------------	----------------------

Renvoie

(*this) L'objet implicite

Un point ne peut pas apparaître deux fois

Implémente [FormeSimple](#).

6.40.3.16 switchPoint()

```
Polygone & Polygone::switchPoint (
    const Vecteur2D & p,
    const Vecteur2D & q)
```

Permet d'inverser la position de deux points.

Paramètres

in	p	Un point du polygone
in	q	Un point du polygone

Renvoie

(*this) L'objet implicite

6.40.3.17 switchPointAt()

```
Polygone & Polygone::switchPointAt (
    int i,
    int j)
```

Permet d'inverser la position de deux points en fonction de leurs indices.

Paramètres

in	i	La position d'un point
in	j	La position d'un point

Renvoie

(*this) L'objet implicite

La position doit respecter la taille du vecteur

6.40.3.18 toString()

```
string Polygone::toString () const [override], [virtual]
```

Permet d'obtenir une chaîne de caractère indiquant les spécificités du polygone.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [Forme](#).

6.40.3.19 verificationParametreVector()

```
void Polygone::verificationParametreVector (  
    const vector< Vecteur2D > & v) [static], [private]
```

Verifie dans un vector l'existence ou non de deux vecteurs identiques.

Paramètres

in	v	Un vector de Vecteur2D
----	-------------------	--

Renvoie

void

6.40.3.20 verificationVectorEtVecteur2D()

```
void Polygone::verificationVectorEtVecteur2D (  
    const vector< Vecteur2D > & v,  
    const Vecteur2D & p) [static], [private]
```

Verifie dans un vector de [Vecteur2D](#) l'existence ou non d'un vecteur p.

Paramètres

in	v	Un vector de Vecteur2D
in	p	Un vecteur2D

Renvoie

void

6.40.4 Documentation des données membres

6.40.4.1 v

```
vector<Vecteur2D> Polygone::v [private]
```

L'ensemble des points du polygone.

La documentation de cette classe a été générée à partir des fichiers suivants :

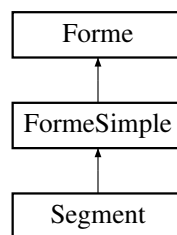
- [Polygone.h](#)
- [Polygone.cpp](#)

6.41 Référence de la classe Segment

Permet de créer un segment.

```
#include <Segment.h>
```

Graphe d'héritage de Segment:



Fonctions membres publiques

- [Segment](#) (const [Vecteur2D](#) &p1, const [Vecteur2D](#) &p2, const [Couleur](#) &couleur=[Couleur::BLACK](#))

Construit un segment à partir d'une couleur, d'un sommet $p1$ et d'un sommet $p2$.

- [Segment](#) (const double &x1=0, const double &y1=0, const double &x2=1, const double &y2=1, const [Couleur](#) &couleur=[Couleur::BLACK](#))

Construit un segment à partir d'une couleur, de coordonnées $x1$ et $y1$ désignant un sommet et de coordonnées $x2$ et $y2$ désignant un autre sommet.

- [Segment](#) * [clone](#) () const override

Retourne un clone du segment.

- [~Segment](#) () override

Permet la destruction du segment.

- [Vecteur2D](#) [getp1](#) () const

Retourne le premier sommet.

- `Vecteur2D getp2 ()` const
Retourne le deuxième sommet.
- `Segment & setp1 (const Vecteur2D &p1)`
Change le premier sommet du segment par un autre sommet $p1$.
- `Segment & setp1 (const double &x1=0, const double &y1=0)`
Change le premier sommet du segment par les coordonnées $x1$ et $y1$.
- `Segment & setp2 (const Vecteur2D &p2)`
Change le deuxième sommet du segment par un autre sommet $p2$.
- `Segment & setp2 (const double &x2=1, const double &y2=1)`
Change le deuxième sommet du segment par les coordonnées $x2$ et $y2$.
- `vector< Vecteur2D > getPoints ()` const override
Permet d'obtenir l'ensemble des points de la forme.
- `Segment & setPoints (const vector< Vecteur2D > &v)` override
Permet de modifier l'ensemble des points du segment.
- `double getAire ()` const override
Retourne l'aire du segment.
- `string toString ()` const override
Permet d'obtenir une chaîne de caractère indiquant les spécificités du segment.
- `Segment & accept (FormeVisiteur &visiteur)` override
Permet l'introduction de méthode via le design pattern visitor.

Fonctions membres publiques hérités de `FormeSimple`

- `FormeSimple (const Couleur &couleur=Couleur::BLACK)`
Construit une forme simple à partir d'une couleur.
- `~FormeSimple ()` override
Permet la destruction de la forme simple.
- `FormeSimple & translation (const Vecteur2D &p)` override
Permet de réaliser une translation de la forme simple.
- `FormeSimple & homothetie (const Vecteur2D &p, const double &k)` override
Permet de réaliser une homothétie de la forme simple.
- `FormeSimple & rotation (const Vecteur2D &p, const double &r)` override

Permet de réaliser une rotation de la forme simple.

- `FormeSimple` & `transformationEcran` (const `Matrice22` &m, const `Vecteur2D` &ab) override

Permet de réaliser une transformation de la forme simple pour un affichage sur écran.

Fonctions membres publiques hérités de `Forme`

- `Forme` (const `Couleur` &couleur=`Couleur::BLACK`)

Construit une forme à partir d'une couleur.

- virtual `~Forme` ()

Permet la destruction de la forme.

- `Couleur` `getCouleur` () const

Retourne la couleur de la forme courante.

- `Forme` & `setCouleur` (const `Couleur` &couleur)

Permet de changer la couleur de la forme.

- `operator string` () const

Permet d'obtenir une chaîne de caractère indiquant les spécificités de la forme.

Attributs privés

- `Vecteur2D` p [2]

L'ensemble des points du `Segment`.

Membres hérités additionnels

Fonctions membres publiques statiques hérités de `Forme`

- static `Forme` * `charger` (const string &path)

Permet de charger une forme à partir d'un fichier local.

Fonctions membres protégées statiques hérités de `FormeSimple`

- static void `verificationNonEgaliteVecteur2D` (const `Vecteur2D` &a, const `Vecteur2D` &b)

*Verifie si un vecteur *a* n'est pas égal à un vecteur *b*.*

- static void `verificationTailleVector` (const vector< `Vecteur2D` > &v, int i)

*Verifie si un entier *i* correspond à la taille d'un vector *v*.*

— static void `verificationAccesVector` (const vector< `Vecteur2D` > &v, int i)

Verifie si un entier i permet l'accès à un élément d'un vecteur v .

— static void `verificationAccesVectorAddPoint` (const vector< `Vecteur2D` > &v, int i)

Verifie si un entier i permet l'accès à un élément d'un vecteur v lors de l'ajout d'un point.

Fonctions membres protégées statiques hérités de `Forme`

— static void `verificationKIsNotNull` (const double &k)

Verifie si la constante k pour l'homothétie n'est pas nulle.

6.41.1 Description détaillée

Permet de créer un segment.

6.41.2 Documentation des constructeurs et destructeur

6.41.2.1 `Segment()` [1/2]

```
Segment::Segment (
    const Vecteur2D & p1,
    const Vecteur2D & p2,
    const Couleur & couleur = Couleur::BLACK) [explicit]
```

Construit un segment à partir d'une couleur, d'un sommet `p1` et d'un sommet `p2`.

Paramètres

in	<code>p1</code>	Un sommet
in	<code>p2</code>	Un sommet
in	<code>couleur</code>	La couleur souhaitée (à défaut noir)

Renvoie

Un segment

`p1` et `p2` ne peuvent pas être égaux

6.41.2.2 Segment() [2/2]

```
Segment::Segment (
    const double & x1 = 0,
    const double & y1 = 0,
    const double & x2 = 1,
    const double & y2 = 1,
    const Couleur & couleur = Couleur::BLACK) [explicit]
```

Construit un segment à partir d'une couleur, de coordonnées *x1* et *y1* désignant un sommet et de coordonnées *x2* et *y2* désignant un autre sommet.

Paramètres

in	<i>x1</i>	Coordonnée x du premier sommet (à défaut 0)
in	<i>y1</i>	Coordonnée y du premier sommet (à défaut 0)
in	<i>x2</i>	Coordonnée x du deuxième sommet (à défaut 1)
in	<i>y2</i>	Coordonnée y du deuxième sommet (à défaut 1)
in	<i>couleur</i>	La couleur souhaitée (à défaut noir)

Renvoie

Un segment

x1 et *x2* ne peuvent pas être égaux en même temps que *y1* et *y2*

6.41.2.3 ~Segment()

```
Segment::~~Segment () [override], [default]
```

Permet la destruction du segment.

6.41.3 Documentation des fonctions membres

6.41.3.1 accept()

```
Segment & Segment::accept (
    FormeVisiteur & visiteur) [override], [virtual]
```

Permet l'introduction de méthode via le design pattern visitor.

Paramètres

in	<i>visiteur</i>	Un objet répondant au pattern visitor
----	-----------------	---------------------------------------

Renvoie

(*this) L'objet implicite

Implémente [FormeSimple](#).

6.41.3.2 clone()

```
Segment * Segment::clone () const [override], [virtual]
```

Retourne un clone du segment.

Renvoie

Un clone du segment

Implémente [FormeSimple](#).

6.41.3.3 getAire()

```
double Segment::getAire () const [override], [virtual]
```

Retourne l'aire du segment.

Renvoie

L'aire du segment

Implémente [FormeSimple](#).

6.41.3.4 getp1()

```
Vecteur2D Segment::getp1 () const
```

Retourne le premier sommet.

Renvoie

p1 Le premier sommet

6.41.3.5 getp2()

```
Vecteur2D Segment::getp2 () const
```

Retourne le deuxième sommet.

Renvoie

p2 Le deuxième sommet

6.41.3.6 getPoints()

```
vector< Vecteur2D > Segment::getPoints () const [override], [virtual]
```

Permet d'obtenir l'ensemble des points de la forme.

Renvoie

L'ensemble des points

Implémente [Forme](#).

6.41.3.7 setp1() [1/2]

```
Segment & Segment::setp1 (
    const double & x1 = 0,
    const double & y1 = 0)
```

Change le premier sommet du segment par les coordonnées x1 et y1.

Renvoie

*this L'objet implicite

Paramètres

in	x1	Coordonnée x du nouveau premier sommet du segment (à défaut 0)
in	y1	Coordonnée y du nouveau premier sommet du segment (à défaut 0)

x1 et x2 ne peuvent pas être égaux en même temps que y1 et y2

6.41.3.8 setp1() [2/2]

```
Segment & Segment::setp1 (
    const Vecteur2D & p1)
```

Change le premier sommet du segment par un autre sommet p1.

Paramètres

in	p1	Le nouveau premier sommet du segment
----	----	--------------------------------------

Renvoie

*this L'objet implicite

p1 et p2 ne peuvent pas être égaux

6.41.3.9 setp2() [1/2]

```
Segment & Segment::setp2 (
    const double & x2 = 1,
    const double & y2 = 1)
```

Change le deuxième sommet du segment par les coordonnées x2 et y2.

Paramètres

in	x2	Coordonnée x du nouveau premier sommet du segment (à défaut 1)
in	y2	Coordonnée y du nouveau premier sommet du segment (à défaut 1)

Renvoie

*this L'objet implicite

x2 et x2 ne peuvent pas être égaux en même temps que y2 et y2

6.41.3.10 setp2() [2/2]

```
Segment & Segment::setp2 (  
    const Vecteur2D & p2)
```

Change le deuxième sommet du segment par un autre sommet p2.

Paramètres

in	p2	Le nouveau deuxième sommet du segment
----	----	---------------------------------------

Renvoie

*this L'objet implicite

p1 et p2 ne peuvent pas être égaux

6.41.3.11 setPoints()

```
Segment & Segment::setPoints (  
    const vector< Vecteur2D > & v) [override], [virtual]
```

Permet de modifier l'ensemble des points du segment.

Paramètres

in	v	Un vecteur de points
----	---	----------------------

Renvoie

(*this) L'objet implicite

Implémente [FormeSimple](#).

6.41.3.12 toString()

```
string Segment::toString () const [override], [virtual]
```

Permet d'obtenir une chaîne de caractère indiquant les spécificités du segment.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [Forme](#).

6.41.4 Documentation des données membres

6.41.4.1 p

`Vecteur2D` `Segment::p[2]` [private]

L'ensemble des points du `Segment`.

La documentation de cette classe a été générée à partir des fichiers suivants :

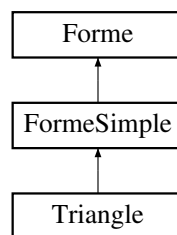
- [Segment.h](#)
- [Segment.cpp](#)

6.42 Référence de la classe Triangle

Permet de gérer un triangle.

```
#include <Triangle.h>
```

Graphe d'héritage de Triangle:



Fonctions membres publiques

- `Triangle` (const `Vecteur2D` &p1, const `Vecteur2D` &p2, const `Vecteur2D` &p3, const `Couleur` &couleur=`Couleur::BLACK`)

Construit un triangle à partir d'une couleur, d'un sommet p1, d'un sommet p2 et d'un sommet p3.

- `Triangle` (const double &x1=0, const double &y1=0, const double &x2=1, const double &y2=1, const double &x3=0, const double &y3=1, const `Couleur` &couleur=`Couleur::BLACK`)

Construit un triangle à partir d'une couleur, de coordonnées x1 et y1 désignant un sommet, de coordonnées x2 et y2 désignant un autre sommet, et de coordonnées x3 et y3 désignant un troisième sommet.

- `Triangle * clone` () const override

Retourne un clone du triangle.

- `~Triangle` () override

Permet la destruction du cercle.

- `Vecteur2D getp1` () const

Retourne le premier sommet.

- `Vecteur2D getp2 ()` const
Retourne le deuxième sommet.
- `Vecteur2D getp3 ()` const
Retourne le troisième sommet.
- `Triangle & setp1 (const Vecteur2D &p1)`
Change le premier sommet du triangle par un autre sommet $p1$.
- `Triangle & setp1 (const double &x1=0, const double &y1=0)`
Change le premier sommet du triangle par les coordonnées $x1$ et $y1$.
- `Triangle & setp2 (const Vecteur2D &p2)`
Change le deuxième sommet du triangle par un autre sommet $p2$.
- `Triangle & setp2 (const double &x2=1, const double &y2=1)`
Change le deuxième sommet du triangle par les coordonnées $x2$ et $y2$.
- `Triangle & setp3 (const Vecteur2D &p3)`
Change le troisième sommet du triangle par un autre sommet $p3$.
- `Triangle & setp3 (const double &x3=0, const double &y3=1)`
Change le troisième sommet du triangle par les coordonnées $x3$ et $y3$.
- `vector< Vecteur2D > getPoints ()` const override
Permet d'obtenir l'ensemble des points du triangle.
- `Triangle & setPoints (const vector< Vecteur2D > &v)` override
Permet de modifier l'ensemble des points du triangle.
- `double getAire ()` const override
Retourne l'aire du triangle.
- `string toString ()` const override
Permet d'obtenir une chaîne de caractère indiquant les spécificités du triangle.
- `Triangle & accept (FormeVisiteur &visiteur)` override
Permet l'introduction de méthode via le design pattern visitor.

Fonctions membres publiques héritées de `FormeSimple`

- `FormeSimple (const Couleur &couleur=Couleur::BLACK)`
Construit une forme simple à partir d'une couleur.
- `~FormeSimple ()` override

Permet la destruction de la forme simple.

- `FormeSimple` & `translation` (const `Vecteur2D` &p) override

Permet de réaliser une translation de la forme simple.

- `FormeSimple` & `homothetie` (const `Vecteur2D` &p, const double &k) override

Permet de réaliser une homothétie de la forme simple.

- `FormeSimple` & `rotation` (const `Vecteur2D` &p, const double &r) override

Permet de réaliser une rotation de la forme simple.

- `FormeSimple` & `transformationEcran` (const `Matrice22` &m, const `Vecteur2D` &ab) override

Permet de réaliser une transformation de la forme simple pour un affichage sur écran.

Fonctions membres publiques hérités de `Forme`

- `Forme` (const `Couleur` &couleur=`Couleur::BLACK`)

Construit une forme à partir d'une couleur.

- virtual `~Forme` ()

Permet la destruction de la forme.

- `Couleur` `getCouleur` () const

Retourne la couleur de la forme courante.

- `Forme` & `setCouleur` (const `Couleur` &couleur)

Permet de changer la couleur de la forme.

- `operator string` () const

Permet d'obtenir une chaîne de caractère indiquant les spécificités de la forme.

Attributs privés

- `Vecteur2D` p [3]

L'ensemble des points du triangle.

Membres hérités additionnels

Fonctions membres publiques statiques hérités de `Forme`

- static `Forme` * `charger` (const string &path)

Permet de charger une forme à partir d'un fichier local.

Fonctions membres protégées statiques hérités de **FormeSimple**

- static void **verificationNonEgaliteVecteur2D** (const **Vecteur2D** &a, const **Vecteur2D** &b)

Verifie si un vecteur a n'est pas égal à un vecteur b .

- static void **verificationTailleVector** (const vector< **Vecteur2D** > &v, int i)

Verifie si un entier i correspond à la taille d'un vector v .

- static void **verificationAccesVector** (const vector< **Vecteur2D** > &v, int i)

Verifie si un entier i permet l'accès à un élément d'un vecteur v .

- static void **verificationAccesVectorAddPoint** (const vector< **Vecteur2D** > &v, int i)

Verifie si un entier i permet l'accès à un élément d'un vecteur v lors de l'ajout d'un point.

Fonctions membres protégées statiques hérités de **Forme**

- static void **verificationKIsNotNull** (const double &k)

Verifie si la constante k pour l'homothétie n'est pas nulle.

6.42.1 Description détaillée

Permet de gérer un triangle.

6.42.2 Documentation des constructeurs et destructeur

6.42.2.1 Triangle() [1/2]

```
Triangle::Triangle (
    const Vecteur2D & p1,
    const Vecteur2D & p2,
    const Vecteur2D & p3,
    const Couleur & couleur = Couleur::BLACK) [explicit]
```

Construit un triangle à partir d'une couleur, d'un sommet $p1$, d'un sommet $p2$ et d'un sommet $p3$.

Paramètres

in	$p1$	Un sommet
in	$p2$	Un sommet
in	$p3$	Un sommet
in	$couleur$	La couleur souhaitée (à défaut noir)

Renvoie

Un triangle

Aucun des sommets ne peut avoir les mêmes coordonnées

6.42.2.2 Triangle() [2/2]

```
Triangle::Triangle (
    const double & x1 = 0,
    const double & y1 = 0,
    const double & x2 = 1,
    const double & y2 = 1,
    const double & x3 = 0,
    const double & y3 = 1,
    const Couleur & couleur = Couleur::BLACK) [explicit]
```

Construit un triangle à partir d'une couleur, de coordonnées `x1` et `y1` désignant un sommet, de coordonnées `x2` et `y2` désignant un autre sommet, et de coordonnées `x3` et `y3` désignant un troisième sommet.

Paramètres

in	<code>x1</code>	Coordonnée x du premier sommet (à défaut 0)
in	<code>y1</code>	Coordonnée y du premier sommet (à défaut 0)
in	<code>x2</code>	Coordonnée x du deuxième sommet (à défaut 1)
in	<code>y2</code>	Coordonnée y du deuxième sommet (à défaut 1)
in	<code>x3</code>	Coordonnée x du troisième sommet (à défaut 0)
in	<code>y3</code>	Coordonnée y du troisième sommet (à défaut 1)
in	<code>couleur</code>	La couleur souhaitée (à défaut noir)

Renvoie

Un triangle

Aucun des sommets ne peut avoir les mêmes coordonnées

6.42.2.3 ~Triangle()

```
Triangle::~~Triangle () [override], [default]
```

Permet la destruction du cercle.

6.42.3 Documentation des fonctions membres

6.42.3.1 accept()

```
Triangle & Triangle::accept (
    FormeVisiteur & visiteur) [override], [virtual]
```

Permet l'introduction de méthode via le design pattern visitor.

Paramètres

in	<code>visiteur</code>	Un objet répondant au pattern visitor
----	-----------------------	---------------------------------------

Renvoie

(*this) L'objet implicite

Implémente [FormeSimple](#).

6.42.3.2 clone()

```
Triangle * Triangle::clone () const [override], [virtual]
```

Retourne un clone du triangle.

Renvoie

Un clone du triangle

Implémente [FormeSimple](#).

6.42.3.3 getAire()

```
double Triangle::getAire () const [override], [virtual]
```

Retourne l'aire du triangle.

Renvoie

L'aire du triangle

Implémente [FormeSimple](#).

6.42.3.4 getp1()

```
Vecteur2D Triangle::getp1 () const
```

Retourne le premier sommet.

Renvoie

p1 Le premier sommet

6.42.3.5 getp2()

```
Vecteur2D Triangle::getp2 () const
```

Retourne le deuxième sommet.

Renvoie

p2 Le deuxième sommet

6.42.3.6 getp3()

```
Vecteur2D Triangle::getp3 () const
```

Retourne le troisième sommet.

Renvoie

p3 Le troisième sommet

6.42.3.7 `getPoints()`

```
vector< Vecteur2D > Triangle::getPoints () const [override], [virtual]
```

Permet d'obtenir l'ensemble des points du triangle.

Renvoie

L'ensemble des points

Implémente [Forme](#).

6.42.3.8 `setp1()` [1/2]

```
Triangle & Triangle::setp1 (  
    const double & x1 = 0,  
    const double & y1 = 0)
```

Change le premier sommet du triangle par les coordonnées `x1` et `y1`.

Paramètres

in	<i>x1</i>	Coordonnée x du nouveau premier sommet du triangle (à défaut 0)
in	<i>y1</i>	Coordonnée y du nouveau premier sommet du triangle (à défaut 0)

Renvoie

`*this` L'objet implicite

Aucun des sommets ne peut avoir les mêmes coordonnées

6.42.3.9 `setp1()` [2/2]

```
Triangle & Triangle::setp1 (  
    const Vecteur2D & p1)
```

Change le premier sommet du triangle par un autre sommet `p1`.

Paramètres

in	<i>p1</i>	Le nouveau premier sommet du triangle
----	-----------	---------------------------------------

Renvoie

`*this` L'objet implicite

Aucun des sommets ne peut avoir les mêmes coordonnées

6.42.3.10 setp2() [1/2]

```
Triangle & Triangle::setp2 (
    const double & x2 = 1,
    const double & y2 = 1)
```

Change le deuxième sommet du triangle par les coordonnées $x2$ et $y2$.

Paramètres

in	$x2$	Coordonnée x du nouveau deuxième sommet du triangle (à défaut 1)
in	$y2$	Coordonnée y du nouveau deuxième sommet du triangle (à défaut 1)

Renvoie

`*this` L'objet implicite

Aucun des sommets ne peut avoir les mêmes coordonnées

6.42.3.11 setp2() [2/2]

```
Triangle & Triangle::setp2 (
    const Vecteur2D & p2)
```

Change le deuxième sommet du triangle par un autre sommet $p2$.

Paramètres

in	$p2$	Le nouveau premier sommet du triangle
----	------	---------------------------------------

Renvoie

`*this` L'objet implicite

Aucun des sommets ne peut avoir les mêmes coordonnées

6.42.3.12 setp3() [1/2]

```
Triangle & Triangle::setp3 (
    const double & x3 = 0,
    const double & y3 = 1)
```

Change le troisième sommet du triangle par les coordonnées $x3$ et $y3$.

Paramètres

in	$x3$	Coordonnée x du nouveau troisième sommet du triangle (à défaut 0)
in	$y3$	Coordonnée y du nouveau troisième sommet du triangle (à défaut 1)

Renvoie

`*this` L'objet implicite

Aucun des sommets ne peut avoir les mêmes coordonnées

6.42.3.13 setp3() [2/2]

```
Triangle & Triangle::setp3 (  
    const Vecteur2D & p3)
```

Change le troisième sommet du triangle par un autre sommet p3.

Paramètres

in	p3	Le nouveau deuxième sommet du triangle
----	----	--

Renvoie

*this L'objet implicite

Aucun des sommets ne peut avoir les mêmes coordonnées

6.42.3.14 setPoints()

```
Triangle & Triangle::setPoints (  
    const vector< Vecteur2D > & v) [override], [virtual]
```

Permet de modifier l'ensemble des points du triangle.

Paramètres

in	v	Un vecteur de points
----	---	----------------------

Renvoie

(*this) L'objet implicite

Implémente [FormeSimple](#).

6.42.3.15 toString()

```
string Triangle::toString () const [override], [virtual]
```

Permet d'obtenir une chaîne de caractère indiquant les spécificités du triangle.

Renvoie

Une chaîne de caractère

Réimplémentée à partir de [Forme](#).

6.42.4 Documentation des données membres

6.42.4.1 p

`Vecteur2D Triangle::p[3] [private]`

L'ensemble des points du triangle.

La documentation de cette classe a été générée à partir des fichiers suivants :

- [Triangle.h](#)
- [Triangle.cpp](#)

6.43 Référence de la classe Vecteur2D

Permet de gérer un vecteur.

```
#include <Vecteur2D.h>
```

Fonctions membres publiques

- [Vecteur2D](#) (const double &x=0, const double &y=0)

Construit un vecteur 2D à partir d'une abscisse x et d'une ordonnée y .

- [Vecteur2D operator+](#) (const [Vecteur2D](#) &v) const

Realise l'addition de deux vecteurs.

- [Vecteur2D operator-](#) () const

Realise l'inverse du vecteur implicite.

- [Vecteur2D operator-](#) (const [Vecteur2D](#) &v) const

Realise la soustraction de deux vecteurs.

- const [Vecteur2D](#) & [operator+=](#) (const [Vecteur2D](#) &v)

Realise l'addition de deux vecteurs.

- [Vecteur2D operator*](#) (const double &k) const

Realise le produit d'un vecteur en fonction d'une constante.

- double [operator*](#) (const [Vecteur2D](#) &v) const

Realise le produit vectoriel de deux vecteurs.

- double [operator|](#) (const [Vecteur2D](#) &v) const

Realise le determinant de deux vecteurs.

- double [norme2](#) () const

Realise la norme au carré d'un vecteur.

— double `norme` () const

Realise la norme d'un vecteur.

— bool `operator==` (const `Vecteur2D` &v) const

Verifie si deux vecteurs sont égaux.

— `operator string` () const

Permet d'obtenir une chaîne de caractère indiquant les spécificités du vecteur.

Attributs publics

— double `x`

L'ensemble des points du segment.

— double `y`

6.43.1 Description détaillée

Permet de gérer un vecteur.

6.43.2 Documentation des constructeurs et destructeur

6.43.2.1 Vecteur2D()

```
Vecteur2D::Vecteur2D (
    const double & x = 0,
    const double & y = 0) [inline], [explicit]
```

Construit un vecteur 2D à partir d'une abscisse `x` et d'une ordonnée `y`.

Paramètres

in	<code>x</code>	L'abscisse
in	<code>y</code>	L'ordonnée

Renvoie

Un vecteur 2D

6.43.3 Documentation des fonctions membres

6.43.3.1 norme()

```
double Vecteur2D::norme () const [inline]
```

Realise la norme d'un vecteur.

Renvoie

La norme

6.43.3.2 norme2()

```
double Vecteur2D::norme2 () const [inline]
```

Realise la norme au carré d'un vecteur.

Renvoie

La norme au carre

6.43.3.3 operator string()

```
Vecteur2D::operator string () const [inline], [explicit]
```

Permet d'obtenir une chaîne de caractère indiquant les spécificités du vecteur.

Renvoie

Une chaîne de caractère

6.43.3.4 operator*() [1/2]

```
Vecteur2D Vecteur2D::operator* (  
    const double & k) const [inline]
```

Realise le produit d'un vecteur en fonction d'une constante.

Paramètres

in	k	Une constante
----	-----	---------------

Renvoie

Un vecteur 2D

6.43.3.5 operator*() [2/2]

```
double Vecteur2D::operator* (
    const Vecteur2D & v) const [inline]
```

Realise le produit vectoriel de deux vecteurs.

Paramètres

in	v	Un vecteur
----	---	------------

Renvoie

Un nombre

6.43.3.6 operator+()

```
Vecteur2D Vecteur2D::operator+ (
    const Vecteur2D & v) const [inline]
```

Realise l'addition de deux vecteurs.

Paramètres

in	v	Un vecteur
----	---	------------

Renvoie

Un vecteur 2D

6.43.3.7 operator+=()

```
const Vecteur2D & Vecteur2D::operator+= (
    const Vecteur2D & v) [inline]
```

Realise l'addition de deux vecteurs.

Paramètres

in	v	Un vecteur
----	---	------------

Renvoie

Un vecteur 2D

6.43.3.8 operator-() [1/2]

```
Vecteur2D Vecteur2D::operator- () const [inline]
```

Realise l'inverse du vecteur implicite.

Renvoie

Un vecteur 2D

6.43.3.9 operator-() [2/2]

```
Vecteur2D Vecteur2D::operator- (  
    const Vecteur2D & v) const [inline]
```

Realise la soustraction de deux vecteurs.

Paramètres

in	v	Un vecteur
----	---	------------

Renvoie

Un vecteur 2D

6.43.3.10 operator==()

```
bool Vecteur2D::operator== (  
    const Vecteur2D & v) const [inline]
```

Verifie si deux vecteurs sont égaux.

Renvoie

Un booléen

6.43.3.11 operator" |())

```
double Vecteur2D::operator| (  
    const Vecteur2D & v) const [inline]
```

Realise le determinant de deux vecteurs.

Paramètres

in	v	Un vecteur
----	---	------------

Renvoie

Le determinant

6.43.4 Documentation des données membres

6.43.4.1 x

```
double Vecteur2D::x
```

L'ensemble des points du segment.

6.43.4.2 y

```
double Vecteur2D::y
```

La documentation de cette classe a été générée à partir du fichier suivant :

— [Vecteur2D.h](#)

Chapitre 7

Documentation des fichiers

7.1 Référence du fichier Cercle.h

```
#include "FormeSimple.h"
```

Classes

— class [ExceptionCercle](#)

Permet de gérer l'ensemble des exceptions liées aux cercles.

— class [ExceptionRayonNegatif](#)

Permet de gérer l'exception lorsque le rayon est negatif.

— class [Cercle](#)

Permet de gérer un cercle.

Macros

— #define [M_PI](#) 3.14159265358979323846

7.1.1 Documentation des macros

7.1.1.1 M_PI

```
#define M_PI 3.14159265358979323846
```

7.2 Cercle.h

[Aller à la documentation de ce fichier.](#)

```

00001 #ifndef PPIL_CERCLE_H
00002 #define PPIL_CERCLE_H
00003 #include "FormeSimple.h"
00004
00005 #ifndef M_PI
00006     #define M_PI 3.14159265358979323846
00007 #endif
00008
00012 class ExceptionCercle : public ExceptionFormeSimple {
00016     string message = " ExceptionCercle -";
00017 public:
00022     string getMessage() const override {
00023         return ExceptionFormeSimple::getMessage() + message;
00024     }
00025 };
00026
00030 class ExceptionRayonNegatif : public ExceptionCercle {
00034     string message = " Le rayon doit etre superieur a 0";
00035 public:
00040     string getMessage() const override {
00041         return ExceptionCercle::getMessage() + message;
00042     }
00043 };
00044
00048 class Cercle : public FormeSimple {
00052     Vecteur2D c;
00056     double r;
00061     static void verificationRayon(const double & r);
00062 public:
00071     explicit Cercle(const Vecteur2D & c, const double & r = 1, const Couleur & couleur =
Couleur::BLACK);
00081     explicit Cercle(const double & xc = 0, const double & yc = 0, const double & r = 1, const Couleur
& couleur = Couleur::BLACK);
00086     Cercle * clone() const override;
00090     ~Cercle() override;
00095     Vecteur2D getc() const;
00100     double getr() const;
00107     Cercle & setc(const Vecteur2D & c);
00115     Cercle & setc(const double & xc = 0, const double & yc = 0);
00122     Cercle & setr(const double & r);
00127     vector<Vecteur2D> getPoints() const override;
00133     Cercle & setPoints(const vector<Vecteur2D> & v) override;
00138     double getAire() const override;
00143     string toString() const override;
00149     Cercle & accept(FormeVisiteur & visiteur) override;
00150 };
00151
00152 #endif

```

7.3 Référence du fichier Forme.h

```

#include <string>
#include <vector>
#include "Matrice22.h"
#include "Vecteur2D.h"

```

Classes

— class [ExceptionForme](#)

Permet de gérer l'ensemble des exceptions liées aux formes.

— class [ExceptionCouleurInconnue](#)

Permet de gérer l'erreur lors de la non-reconnaissance d'une couleur.

— class `ExceptionKIsNull`

Permet de gérer l'erreur de l'utilisation d'un `k` nulle pour la rotation.

— class `Forme`

Permet de créer une forme.

Énumérations

— enum class `Couleur` {
 `BLACK` , `BLUE` , `RED` , `GREEN` ,
 `YELLOW` , `CYAN` }

Permet de caractériser les couleurs de nos formes.

Fonctions

— string `couleurToString` (const `Couleur` &couleur)

Retourne la chaîne de caractères associée à la couleur.

— `Couleur` `stringToCouleur` (const string &s)

Retourne la couleur associée à la chaîne de caractères.

7.3.1 Documentation du type de l'énumération

7.3.1.1 Couleur

```
enum class Couleur [strong]
```

Permet de caractériser les couleurs de nos formes.

Valeurs énumérées

BLACK	
BLUE	
RED	
GREEN	
YELLOW	
CYAN	

7.3.2 Documentation des fonctions

7.3.2.1 couleurToString()

```
string couleurToString (
    const Couleur & couleur) [inline]
```

Retourne la chaîne de caractères associée à la couleur.

Paramètres

in	<i>couleur</i>	Une couleur
----	----------------	-------------

Renvoie

Une chaîne de caractères

7.3.2.2 stringToCouleur()

```
Couleur stringToCouleur (
    const string & s) [inline]
```

Retourne la couleur associée à la chaîne de caractères.

Paramètres

in	<i>s</i>	Une chaîne de caractères
----	----------	--------------------------

Renvoie

Une couleur

Si la chaîne est invalide, alors une erreur est lancée

7.4 Forme.h

[Aller à la documentation de ce fichier.](#)

```
00001 #ifndef PPIL_FORME_H
00002 #define PPIL_FORME_H
00003 #include <string>
00004 #include <vector>
00005
00006 #include "Matrice22.h"
00007 #include "Vecteur2D.h"
00008
00009 using namespace std;
00010
00011 class FormeVisiteur;
00012
00016 enum class Couleur {BLACK, BLUE, RED, GREEN, YELLOW, CYAN};
00017
00023 inline string couleurToString(const Couleur & couleur) {
00024     switch (couleur) {
00025         case Couleur::BLACK:
00026             return "Black";
00027         case Couleur::BLUE:
00028             return "Blue";
```

```

00029         case Couleur::RED:
00030             return "Red";
00031         case Couleur::GREEN:
00032             return "Green";
00033         case Couleur::YELLOW:
00034             return "Yellow";
00035         case Couleur::CYAN:
00036             return "Cyan";
00037     }
00038     return "";
00039 }
00040
00044 class ExceptionForme : public exception {
00048     string message = "ExceptionForme -";
00049 public:
00054     const char * what() const noexcept override {
00055         return getMessage().c_str();
00056     }
00061     virtual string getMessage() const {
00062         return message;
00063     }
00068     friend ostream & operator<<(ostream & s, const ExceptionForme & e) {
00069         return s << e.getMessage();
00070     }
00071 };
00072
00076 class ExceptionCouleurInconnue : public ExceptionForme {
00080     string message = " La couleur est inconnue";
00081 public:
00086     string getMessage() const override {
00087         return ExceptionForme::getMessage() + message;
00088     }
00089 };
00090
00097 inline Couleur stringToCouleur(const string & s) {
00098     if (s == "Black") {
00099         return Couleur::BLACK;
00100     }
00101     if (s == "Blue") {
00102         return Couleur::BLUE;
00103     }
00104     if (s == "Red") {
00105         return Couleur::RED;
00106     }
00107     if (s == "Green") {
00108         return Couleur::GREEN;
00109     }
00110     if (s == "Yellow") {
00111         return Couleur::YELLOW;
00112     }
00113     if (s == "Cyan") {
00114         return Couleur::CYAN;
00115     }
00116     throw ExceptionForme();
00117 }
00118
00122 class ExceptionKIsNull : public ExceptionForme {
00126     string message = " Le rapport d'homothetie k = 0 est interdit";
00127 public:
00132     string getMessage() const override {
00133         return ExceptionForme::getMessage() + message;
00134     }
00135 };
00136
00140 class Forme {
00144     Couleur couleur;
00145 protected:
00151     static void verificationKIsNotNull(const double & k);
00152 public:
00158     explicit Forme(const Couleur & couleur = Couleur::BLACK);
00163     virtual Forme * clone() const = 0;
00167     virtual ~Forme();
00172     Couleur getCouleur() const;
00178     Forme & setCouleur(const Couleur & couleur);
00183     virtual vector<Vecteur2D> getPoints() const = 0;
00188     virtual double getAire() const = 0;
00193     virtual Forme & translation(const Vecteur2D & p) = 0;
00198     virtual Forme & homothetie(const Vecteur2D & p, const double & k) = 0;
00203     virtual Forme & rotation(const Vecteur2D & p, const double & r) = 0;
00210     virtual Forme & transformationEcran(const Matrice22 & m, const Vecteur2D & ab) = 0;
00216     static Forme * charger(const string & path);
00221     virtual string toString() const;
00226     explicit operator string() const;
00231     friend ostream & operator<<(ostream & s, const Forme & f);
00237     virtual Forme & accept(FormeVisiteur & visiteur) = 0;
00238 };
00239

```

```
00240 #endif
```

7.5 Référence du fichier FormeCharger.h

```
#include "Forme.h"
```

Classes

— class [ExceptionFormeCharger](#)

Permet de gérer l'ensemble des exceptions liées au chargement d'une forme.

— class [ExceptionFileCantBeRead](#)

Permet de gérer l'exception liée à l'impossible de lire un fichier.

— class [ExceptionFileIsEmpty](#)

Permet de gérer l'exception liée à un fichier vide.

— class [ExceptionStringIsNotRecognizable](#)

Permet de gérer l'exception dans le cas d'une forme non reconnue.

— class [FormeCharger](#)

Permet de gérer le chargement d'une forme.

7.6 FormeCharger.h

[Aller à la documentation de ce fichier.](#)

```
00001 #ifndef PPIL_FORMECHAINOFRESPONSABILITY_H
00002 #define PPIL_FORMECHAINOFRESPONSABILITY_H
00003 #include "Forme.h"
00004
00008 class ExceptionFormeCharger : public ExceptionForme {
00012     string message = " ExceptionFormeCharge -";
00013 public:
00018     string getMessage() const override {
00019         return ExceptionForme::getMessage() + message;
00020     }
00021 };
00022
00026 class ExceptionFileCantBeRead : public ExceptionFormeCharger {
00030     string message = " Le fichier ne peut pas être ouvert";
00031 public:
00036     string getMessage() const override {
00037         return ExceptionFormeCharger::getMessage() + message;
00038     }
00039 };
00040
00044 class ExceptionFileIsEmpty : public ExceptionFormeCharger {
00048     string message = " Le fichier est vide";
00049 public:
00054     string getMessage() const override {
00055         return ExceptionFormeCharger::getMessage() + message;
00056     }
00057 };
00058
00062 class ExceptionStringIsNotRecognizable : public ExceptionFormeCharger {
```



```

00066     string message = " Aucune forme ne peut être déduit d'une des chaînes de caractères du fichier";
00067 public:
00072     string getMessage() const override {
00073         return ExceptionFormeCharger::getMessage() + message;
00074     }
00075 };
00076
00080 class FormeCharger {
00084     static FormeCharger * origine;
00088     FormeCharger * suivant;
00094     Forme * chargerDepuisFluxStart(istream & flux) const;
00095 protected:
00102     virtual Forme * charger(const string & s, istream & flux) const = 0;
00103 public:
00108     explicit FormeCharger(FormeCharger * suivant);
00112     virtual ~FormeCharger();
00117     static FormeCharger * getOrigine();
00123     Forme * chargerDepuisFichier(const string & path) const;
00130     Forme * chargerDepuisFlux(const string & s, istream & flux) const;
00131 };
00132
00133 #endif

```

7.7 Référence du fichier `FormeChargerCercle.h`

```
#include "FormeCharger.h"
```

Classes

— class `FormeChargerCercle`

Permet le chargement d'un cercle.

7.8 `FormeChargerCercle.h`

[Aller à la documentation de ce fichier.](#)

```

00001 #ifndef PPIL_FORMECHARGERCERCLE_H
00002 #define PPIL_FORMECHARGERCERCLE_H
00003 #include "FormeCharger.h"
00004
00008 class FormeChargerCercle : public FormeCharger {
00009 protected:
00016     Forme * charger(const string & s, istream & flux) const override;
00017 public:
00022     explicit FormeChargerCercle(FormeCharger * suivant);
00026     ~FormeChargerCercle() override;
00027 };
00028
00029 #endif

```

7.9 Référence du fichier `FormeChargerFormeComposee.h`

```
#include "FormeCharger.h"
```

Classes

— class [FormeChargerFormeComposee](#)

Permet le chargement d'une forme composée.

7.10 FormeChargerFormeComposee.h

[Aller à la documentation de ce fichier.](#)

```
00001 #ifndef PPIL_FORMECHARGERFORMECOMPOSEE_H
00002 #define PPIL_FORMECHARGERFORMECOMPOSEE_H
00003 #include "FormeCharger.h"
00004
00008 class FormeChargerFormeComposee : public FormeCharger {
00009 protected:
00016     Forme * charger(const string & s, istream & flux) const override;
00017 public:
00022     explicit FormeChargerFormeComposee(FormeCharger * suivant);
00026     ~FormeChargerFormeComposee() override;
00027 };
00028
00029 #endif
```

7.11 Référence du fichier FormeChargerPolygone.h

```
#include "FormeCharger.h"
```

Classes

— class [FormeChargerPolygone](#)

Permet le chargement d'un polygone.

7.12 FormeChargerPolygone.h

[Aller à la documentation de ce fichier.](#)

```
00001 #ifndef PPIL_FORMECHARGERPOLYGONE_H
00002 #define PPIL_FORMECHARGERPOLYGONE_H
00003 #include "FormeCharger.h"
00004
00008 class FormeChargerPolygone : public FormeCharger {
00009 protected:
00016     Forme * charger(const string & s, istream & flux) const override;
00017 public:
00022     explicit FormeChargerPolygone(FormeCharger * suivant);
00026     ~FormeChargerPolygone() override;
00027 };
00028
00029 #endif
```

7.13 Référence du fichier FormeChargerSegment.h

```
#include "FormeCharger.h"
```

Classes

— class [FormeChargerSegment](#)

Permet le chargement d'un segment.

7.14 FormeChargerSegment.h

[Aller à la documentation de ce fichier.](#)

```
00001 #ifndef PPIL_FORMECHARGERSEGMENT_H
00002 #define PPIL_FORMECHARGERSEGMENT_H
00003 #include "FormeCharger.h"
00004
00008 class FormeChargerSegment : public FormeCharger {
00009 protected:
00016     Forme * charger(const string & s, istringstream & flux) const override;
00017 public:
00022     explicit FormeChargerSegment(FormeCharger * suivant);
00026     ~FormeChargerSegment() override;
00027 };
00028
00029 #endif
```

7.15 Référence du fichier FormeChargerTriangle.h

```
#include "FormeCharger.h"
```

Classes

— class [FormeChargerTriangle](#)

Permet le chargement d'un triangle.

7.16 FormeChargerTriangle.h

[Aller à la documentation de ce fichier.](#)

```
00001 #ifndef PPIL_FORMECHARGERTRIANGLE_H
00002 #define PPIL_FORMECHARGERTRIANGLE_H
00003 #include "FormeCharger.h"
00004
00008 class FormeChargerTriangle : public FormeCharger {
00009 protected:
00016     Forme * charger(const string & s, istringstream & flux) const override;
00017 public:
00022     explicit FormeChargerTriangle(FormeCharger * suivant);
00026     ~FormeChargerTriangle() override;
00027 };
00028
00029 #endif
```

7.17 Référence du fichier FormeComposee.h

```
#include "Forme.h"
```

Classes

— class [ExceptionFormeComposee](#)

Permet de gérer l'ensemble des exceptions liées aux formes composées.

— class [ExceptionArrayOutOfBoundFC](#)

Permet de gérer l'exception en cas de dépassement de case d'un tableau.

— class [FormeComposee](#)

Permet de gérer une forme composée.

7.18 FormeComposee.h

[Aller à la documentation de ce fichier.](#)

```

00001 #ifndef PPIL_FORMECOMPOSEE_H
00002 #define PPIL_FORMECOMPOSEE_H
00003 #include "Forme.h"
00004
00008 class ExceptionFormeComposee : public ExceptionForme {
00012     string message = " ExceptionFormeComposee -";
00013 public:
00018     string getMessage() const override {
00019         return ExceptionForme::getMessage() + message;
00020     }
00021 };
00022
00026 class ExceptionArrayOutOfBoundFC : public ExceptionFormeComposee {
00030     string message = " La case du tableau n'existe pas";
00031 public:
00036     string getMessage() const override {
00037         return ExceptionFormeComposee::getMessage() + message;
00038     }
00039 };
00040
00044 class FormeComposee : public Forme {
00048     vector<Forme *> v;
00054     void copy(const FormeComposee & f);
00059     void destroy();
00066     static void verificationAccesVector(const vector<Forme *> & v, int i);
00073     static void verificationAccesVectorAddForme(const vector<Forme *> & v, int i);
00074 public:
00080     explicit FormeComposee(const Couleur & couleur);
00088     explicit FormeComposee(const vector<Forme *> & v = vector<Forme *>(), const Couleur & couleur =
Couleur::BLACK);
00094     FormeComposee(const FormeComposee & f);
00100     FormeComposee & operator=(const FormeComposee & f);
00104     ~FormeComposee() override;
00109     FormeComposee * clone() const override;
00110     /*
00111      * @brief Permet d'obtenir l'ensemble des points de la forme composée
00112      * @return L'ensemble des points
00113      */
00114     vector<Vecteur2D> getPoints() const override;
00119     vector<Forme *> getFormes() const;
00126     FormeComposee & setFormes(const vector<Forme *> & vf);
00133     Forme * getFormeAt(int i) const;
00140     FormeComposee & addForme(const Forme & f);
00149     FormeComposee & addFormeAt(const Forme & f, int i);
00156     FormeComposee & removeFormeAt(int i);
00161     double getAire() const override;
00166     FormeComposee & translation(const Vecteur2D & p) override;
00171     FormeComposee & homothetie(const Vecteur2D & p, const double & k) override;
00176     FormeComposee & rotation(const Vecteur2D & p, const double & r) override;
00183     FormeComposee & transformationEcran(const Matrice22 & m, const Vecteur2D & ab) override;
00188     string toString() const override;
00195     FormeComposee & operator+(const Forme & f);
00200     FormeComposee & operator-(int i);
00207     Forme * operator[](int i) const;
00213     FormeComposee & accept(FormeVisiteur & visiteur) override;
00214 };
00215
00216 #endif

```

7.19 Référence du fichier FormeDessiner.h

```
#include "FormeVisiteur.h"
#include <winsock2.h>
```

Classes

— class [ExceptionDessin](#)

Permet de gérer l'ensemble des exceptions liées aux visiteurs de forme dessin.

— class [ExceptionTransformationForme](#)

Permet de gérer l'exception lors de l'impossibilité de transformer la forme.

— class [ExceptionCreationSocket](#)

Permet de gérer l'exception lors de l'impossibilité de créer le socket.

— class [ExceptionEnvoiImpossible](#)

Permet de gérer l'exception lors de l'impossibilité d'envoyer un message.

— class [ExceptionParametragePaquet](#)

Permet de gérer l'exception lors de l'impossibilité de paramétrer l'envoi des messages.

— class [ExceptionConnexionImpossible](#)

Permet de gérer l'exception lors de l'impossibilité de se connecter au serveur.

— class [FormeDessiner](#)

Permet l'implémentation du pattern design visiteur pour les formes dans le cadre d'un dessin.

7.20 FormeDessiner.h

[Aller à la documentation de ce fichier.](#)

```
00001 #ifndef PPIL_FORMEDESSIN_H
00002 #define PPIL_FORMEDESSIN_H
00003 #include "FormeVisiteur.h"
00004 #include <winsock2.h>
00005
00009 class ExceptionDessin : public ExceptionFormeVisiteur {
00013     string message = " ExceptionDessin -";
00014 public:
00019     string getMessage() const override {
00020         return ExceptionFormeVisiteur::getMessage() + message;
00021     }
00022 };
00023
00027 class ExceptionTransformationForme : public ExceptionDessin {
00031     string message = " Il est impossible de transformer la forme";
00032 public:
00037     string getMessage() const override {
00038         return ExceptionDessin::getMessage() + message;
00039     }
00040 };
00041
00045 class ExceptionCreationSocket : public ExceptionDessin {
```

```

00049     string message = " Il est impossible de creer la socket";
00050 public:
00055     string getMessage() const override {
00056         return ExceptionDessin::getMessage() + message;
00057     }
00058 };
00059
00063 class ExceptionEnvoieImpossible : public ExceptionDessin {
00067     string message = " Il est impossible d'envoyer un message";
00068 public:
00073     string getMessage() const override {
00074         return ExceptionDessin::getMessage() + message;
00075     }
00076 };
00077
00081 class ExceptionParametragePaquet : public ExceptionDessin {
00085     string message = " Il est impossible de paramettrer les paquets";
00086 public:
00091     string getMessage() const override {
00092         return ExceptionDessin::getMessage() + message;
00093     }
00094 };
00095
00099 class ExceptionConnexionImpossible : public ExceptionDessin {
00103     string message = " Il est impossible de se connecter au serveur";
00104 public:
00109     string getMessage() const override {
00110         return ExceptionDessin::getMessage() + message;
00111     }
00112 };
00113
00117 class FormeDessiner : public FormeVisiteur {
00121     static const string adress;
00125     static constexpr int port = 4567;
00129     SOCKET sock;
00133     SOCKADDR_IN sockaddr{};
00139     FormeDessiner & traiterFormePourRequete(const Forme & f);
00145     FormeDessiner & preTransformationPassageEcran(Forme & f);
00151     FormeDessiner & envoyerRequete(const string & requete);
00152 public:
00156     explicit FormeDessiner();
00160     ~FormeDessiner() override;
00166     FormeDessiner & visit(const Segment & f) override;
00172     FormeDessiner & visit(const Cercle & f) override;
00178     FormeDessiner & visit(const Triangle & f) override;
00184     FormeDessiner & visit(const Polygone & f) override;
00190     FormeDessiner & visit(const FormeComposee & f) override;
00191 };
00192
00193 #endif

```

7.21 Référence du fichier FormeSauvegarder.h

```
#include "FormeVisiteur.h"
```

Classes

— class [ExceptionSauvegarder](#)

Permet de gérer l'ensemble des exceptions liées aux visiteurs de formes sauvegarde.

— class [ExceptionFileCantBeOpen](#)

Permet de gérer l'exception liée à l'impossibilité de créer un fichier.

— class [FormeSauvegarder](#)

Permet l'implémentation du pattern design visiteur pour les formes dans le cadre d'une sauvegarde.

7.22 FormeSauvegarder.h

[Aller à la documentation de ce fichier.](#)

```

00001 #ifndef PPIL_DESSINER_H
00002 #define PPIL_DESSINER_H
00003 #include "FormeVisiteur.h"
00004
00008 class ExceptionSauvegarder : public ExceptionFormeVisiteur {
00012     string message = " ExceptionSauvegarder -";
00013 public:
00018     string getMessage() const override {
00019         return ExceptionFormeVisiteur::getMessage() + message;
00020     }
00021 };
00022
00026 class ExceptionFileCantBeOpen : public ExceptionSauvegarder {
00030     string message = " Le fichier ne peut pas etre cree";
00031 public:
00036     string getMessage() const override {
00037         return ExceptionSauvegarder::getMessage() + message;
00038     }
00039 };
00040
00044 class FormeSauvegarder : public FormeVisiteur {
00048     const string path;
00054     FormeSauvegarder & sauvegarderForme(const string & formeToString);
00055 public:
00060     explicit FormeSauvegarder(string path);
00064     ~FormeSauvegarder() override;
00070     FormeSauvegarder & visit(const Segment & f) override;
00076     FormeSauvegarder & visit(const Cercle & f) override;
00082     FormeSauvegarder & visit(const Triangle & f) override;
00088     FormeSauvegarder & visit(const Polygone & f) override;
00094     FormeSauvegarder & visit(const FormeComposee & f) override;
00095 };
00096
00097 #endif //PPIL_DESSINER_H

```

7.23 Référence du fichier FormeSimple.h

```
#include "Forme.h"
```

Classes

— class [ExceptionFormeSimple](#)

Permet de gérer l'ensemble des exceptions liées aux formes simples.

— class [ExceptionPointEqualsAnotherPoint](#)

Permet de gérer l'exception lorsque deux points sont identiques dans une forme simple.

— class [ExceptionArrayIsTooShort](#)

Permet de gérer l'exception lorsqu'on essaye d'appliquer un ensemble de points pour une forme tandis que cet ensemble n'est pas de bonne taille.

— class [ExceptionArrayOutOfBounds](#)

Permet de gérer l'exception en cas de dépassement de case d'un tableau.

— class [FormeSimple](#)

Permet de gérer une forme simple.

7.24 FormeSimple.h

[Aller à la documentation de ce fichier.](#)

```

00001 #ifndef PPIL_FORMESIMPLE_H
00002 #define PPIL_FORMESIMPLE_H
00003 #include "Forme.h"
00004
00008 class ExceptionFormeSimple : public ExceptionForme {
00012     string message = " ExceptionFormeSimple -";
00013 public:
00018     string getMessage() const override {
00019         return ExceptionForme::getMessage() + message;
00020     }
00021 };
00022
00026 class ExceptionPointEqualsAnotherPoint : public ExceptionFormeSimple {
00030     string message = " Deux des points sont de memes coordonnees";
00031 public:
00036     string getMessage() const override {
00037         return ExceptionFormeSimple::getMessage() + message;
00038     }
00039 };
00040
00044 class ExceptionArrayIsTooShort : public ExceptionFormeSimple {
00048     string message = " Le tableau ne possede pas suffisamment de points pour couvrir la forme";
00049 public:
00054     string getMessage() const override {
00055         return ExceptionFormeSimple::getMessage() + message;
00056     }
00057 };
00058
00062 class ExceptionArrayOutOfBounds : public ExceptionFormeSimple {
00066     string message = " La case du tableau n'existe pas";
00067 public:
00072     string getMessage() const override {
00073         return ExceptionFormeSimple::getMessage() + message;
00074     }
00075 };
00076
00080 class FormeSimple : public Forme {
00081 protected:
00088     static void verificationNonEgaliteVecteur2D(const Vecteur2D & a, const Vecteur2D & b);
00095     static void verificationTailleVector(const vector<Vecteur2D> & v, int i);
00102     static void verificationAccesVector(const vector<Vecteur2D> & v, int i);
00109     static void verificationAccesVectorAddPoint(const vector<Vecteur2D> & v, int i);
00110 public:
00116     explicit FormeSimple(const Couleur & couleur = Couleur::BLACK);
00121     FormeSimple * clone() const override = 0;
00125     ~FormeSimple() override;
00130     virtual FormeSimple & setPoints(const vector<Vecteur2D> & v) = 0;
00135     double getAire() const override = 0;
00140     FormeSimple & translation(const Vecteur2D & p) override;
00145     FormeSimple & homothetie(const Vecteur2D & p, const double & k) override;
00150     FormeSimple & rotation(const Vecteur2D & p, const double & r) override;
00157     FormeSimple & transformationEcran(const Matrice22 & m, const Vecteur2D & ab) override;
00163     FormeSimple & accept(FormeVisiteur & visiteur) override = 0;
00164 };
00165
00166 #endif

```

7.25 Référence du fichier FormeVisiteur.h

```

#include "FormeComposee.h"
#include "Cercle.h"
#include "Polygone.h"
#include "Segment.h"
#include "Triangle.h"

```

Classes

— class [ExceptionFormeVisiteur](#)

Permet de gérer l'ensemble des exceptions liées aux visiteurs de formes.

— class [FormeVisiteur](#)

Permet l'implémentation du pattern design visiteur pour les formes.

7.26 FormeVisiteur.h

[Aller à la documentation de ce fichier.](#)

```
00001 #ifndef PPIL_FORMEVISITEUR_H
00002 #define PPIL_FORMEVISITEUR_H
00003 #include "FormeComposee.h"
00004 #include "Cercle.h"
00005 #include "Polygone.h"
00006 #include "Segment.h"
00007 #include "Triangle.h"
00008
00012 class ExceptionFormeVisiteur : public ExceptionForme {
00016     string message = " ExceptionFormeVisiteur -";
00017 public:
00022     string getMessage() const override {
00023         return ExceptionForme::getMessage() + message;
00024     }
00025 };
00026
00030 class FormeVisiteur {
00031 public:
00035     virtual ~FormeVisiteur() = default;
00041     virtual FormeVisiteur & visit(const Segment & f) = 0;
00047     virtual FormeVisiteur & visit(const Cercle & f) = 0;
00053     virtual FormeVisiteur & visit(const Triangle & f) = 0;
00059     virtual FormeVisiteur & visit(const Polygone & f) = 0;
00065     virtual FormeVisiteur & visit(const FormeComposee & f) = 0;
00066 };
00067
00068 #endif
```

7.27 Référence du fichier Matrice22.h

```
#include "Vecteur2D.h"
```

Classes

— class [Matrice22](#)

*Permet de gérer des matrices de taille 2 * 2.*

Fonctions

— ostream & [operator<<](#) (ostream &o, const [Matrice22](#) &mat)

Permet l'utilisation du << pour une matrice lors de l'utilisation d'ostream.

7.27.1 Documentation des fonctions

7.27.1.1 operator<<()

```
ostream & operator<< (
    ostream & o,
    const Matrice22 & mat) [inline]
```

Permet l'utilisation du << pour une matrice lors de l'utilisation d'ostream.

Renvoie

L'ostream complété

7.28 Matrice22.h

[Aller à la documentation de ce fichier.](#)

```
00001 #ifndef PPIL_MATRICE22_H
00002 #define PPIL_MATRICE22_H
00003 #include "Vecteur2D.h"
00004
00008 class Matrice22 {
00009 public:
00013     Vecteur2D ligneHaut, ligneBas;
00020     Matrice22(const Vecteur2D & ligneHaut, const Vecteur2D & ligneBas) : ligneHaut(ligneHaut),
    ligneBas(ligneBas) {}
00029     explicit Matrice22(const double & a11 = 1, const double & a12 = 0, const double & a21 = 0, const
    double & a22 = 1) : Matrice22(Vecteur2D(a11, a12), Vecteur2D(a21, a22)) {}
00035     static Matrice22 creeRotation(const double & alpha) {
00036         const double calpha = cos(alpha);
00037         const double salpha = sin(alpha);
00038         return Matrice22(calpha, -salpha, salpha, calpha);
00039     }
00044     explicit operator string() const {
00045         ostream o;
00046         o << ligneHaut << endl;
00047         o << ligneBas;
00048         return o.str();
00049     }
00055     Vecteur2D operator *(const Vecteur2D & v) const {
00056         return Vecteur2D(ligneHaut*v, ligneBas*v);
00057     }
00058 };
00059
00064 inline ostream & operator <<(ostream & o, const Matrice22 & mat) {
00065     return o << string(mat);
00066 }
00067
00068 #endif
```

7.29 Référence du fichier MaWinsock.h

```
#include <winsock2.h>
```

Classes

— class [MaWinsock](#)

Permet la gestion de la socket serveur.

7.30 MaWinsock.h

[Aller à la documentation de ce fichier.](#)

```
00001 #ifndef PPIL_MAWINSOCK_H
00002 #define PPIL_MAWINSOCK_H
00003 #include <winsock2.h>
00004
00008 class MaWinsock final {
00009     WSADATA wsaData;
00013     explicit MaWinsock();
00017     static MaWinsock * instanceUnique;
00018 public:
00023     static MaWinsock * getInstance();
00027     ~MaWinsock();
00028 };
00029
00030 #endif
```

7.31 Référence du fichier Polygone.h

```
#include "FormeSimple.h"
```

Classes

— class [Polygone](#)

Permet de gérer du polygone.

7.32 Polygone.h

[Aller à la documentation de ce fichier.](#)

```
00001 #ifndef PPIL_POLYGON_H
00002 #define PPIL_POLYGON_H
00003 #include "FormeSimple.h"
00004
00008 class Polygone : public FormeSimple {
00012     vector<Vecteur2D> v;
00018     void static verificationParametreVector(const vector<Vecteur2D> & v);
00025     void static verificationVectorEtVecteur2D(const vector<Vecteur2D> & v, const Vecteur2D & p);
00026 public:
00032     explicit Polygone(const Couleur & couleur);
00040     explicit Polygone(const vector<Vecteur2D> & v = {}, const Couleur & couleur = Couleur::BLACK);
00045     Polygone * clone() const override;
00049     ~Polygone() override;
00054     vector<Vecteur2D> getPoints() const override;
00061     Polygone & setPoints(const vector<Vecteur2D> & v) override;
00067     Vecteur2D getPointAt(int i) const;
00074     Polygone & addPoint(const Vecteur2D & p);
00083     Polygone & addPointAt(const Vecteur2D & p, int i);
00089     Polygone & removePoint(const Vecteur2D & p);
00096     Polygone & removePointAt(int i);
00104     Polygone & modifyPoint(const Vecteur2D & p, const Vecteur2D & q);
00112     Polygone & modifyPointAt(int i, const Vecteur2D & q);
00119     Polygone & switchPoint(const Vecteur2D & p, const Vecteur2D & q);
00127     Polygone & switchPointAt(int i, int j);
00132     double getAire() const override;
00137     string toString() const override;
00144     Polygone & operator+(const Vecteur2D & p);
00150     Polygone & operator-(const Vecteur2D & p);
00156     Vecteur2D operator[](int i) const;
00162     Polygone & accept(FormeVisiteur & visiteur) override;
00163 };
00164
00165 #endif
```

7.33 Référence du fichier Segment.h

```
#include "FormeSimple.h"
```

Classes

— class [Segment](#)

Permet de créer un segment.

7.34 Segment.h

[Aller à la documentation de ce fichier.](#)

```
00001 #ifndef PPIL_SEGMENT_H
00002 #define PPIL_SEGMENT_H
00003 #include "FormeSimple.h"
00004
00008 class Segment : public FormeSimple {
00012     Vecteur2D p[2];
00013 public:
00022     explicit Segment(const Vecteur2D & p1, const Vecteur2D & p2, const Couleur & couleur =
Couleur::BLACK);
00033     explicit Segment(const double & x1 = 0, const double & y1 = 0, const double & x2 = 1, const double
& y2 = 1, const Couleur & couleur = Couleur::BLACK);
00038     Segment * clone() const override;
00042     ~Segment() override;
00047     Vecteur2D getp1() const;
00052     Vecteur2D getp2() const;
00059     Segment & setp1(const Vecteur2D & p1);
00067     Segment & setp1(const double & x1 = 0, const double & y1 = 0);
00074     Segment & setp2(const Vecteur2D & p2);
00082     Segment & setp2(const double & x2 = 1, const double & y2 = 1);
00083     /*
00084      * @brief Permet d'obtenir l'ensemble des points du segment
00085      * @return L'ensemble des points
00086      */
00087     vector<Vecteur2D> getPoints() const override;
00093     Segment & setPoints(const vector<Vecteur2D> & v) override;
00098     double getAire() const override;
00103     string toString() const override;
00109     Segment & accept(FormeVisiteur & visiteur) override;
00110 };
00111
00112 #endif
```

7.35 Référence du fichier Triangle.h

```
#include "FormeSimple.h"
```

Classes

— class [Triangle](#)

Permet de gérer un triangle.

7.36 Triangle.h

[Aller à la documentation de ce fichier.](#)

```

00001 #ifndef PPIL_TRIANGLE_H
00002 #define PPIL_TRIANGLE_H
00003 #include "FormeSimple.h"
00004
00008 class Triangle : public FormeSimple {
00012     Vecteur2D p[3];
00013 public:
00023     explicit Triangle(const Vecteur2D & p1, const Vecteur2D & p2, const Vecteur2D & p3, const Couleur
& couleur = Couleur::BLACK);
00036     explicit Triangle(const double & x1 = 0, const double & y1 = 0, const double & x2 = 1, const
double & y2 = 1, const double & x3 = 0, const double & y3 = 1, const Couleur & couleur =
Couleur::BLACK);
00041     Triangle * clone() const override;
00045     ~Triangle() override;
00050     Vecteur2D getp1() const;
00055     Vecteur2D getp2() const;
00060     Vecteur2D getp3() const;
00067     Triangle & setp1(const Vecteur2D & p1);
00075     Triangle & setp1(const double & x1 = 0, const double & y1 = 0);
00082     Triangle & setp2(const Vecteur2D & p2);
00090     Triangle & setp2(const double & x2 = 1, const double & y2 = 1);
00097     Triangle & setp3(const Vecteur2D & p3);
00105     Triangle & setp3(const double & x3 = 0, const double & y3 = 1);
00110     vector<Vecteur2D> getPoints() const override;
00116     Triangle & setPoints(const vector<Vecteur2D> & v) override;
00121     double getAire() const override;
00126     string toString() const override;
00132     Triangle & accept(FormeVisiteur & visiteur) override;
00133 };
00134
00135 #endif

```

7.37 Référence du fichier Vecteur2D.h

```

#include <string>
#include <sstream>
#include <cmath>

```

Classes

— class [Vecteur2D](#)

Permet de gérer un vecteur.

Fonctions

— ostream & [operator<<](#) (ostream &o, const [Vecteur2D](#) &v)

Permet l'utilisation du << pour un vecteur lors de l'utilisation d'ostream.

7.37.1 Documentation des fonctions

7.37.1.1 operator<<()

```
ostream & operator<< (
    ostream & o,
    const Vecteur2D & v) [inline]
```

Permet l'utilisation du << pour un vecteur lors de l'utilisation d'ostream.

Renvoie

L'ostream complété

7.38 Vecteur2D.h

[Aller à la documentation de ce fichier.](#)

```
00001 #ifndef PPIL_VECTEUR2D_H
00002 #define PPIL_VECTEUR2D_H
00003 #include <string>
00004 #include <sstream>
00005 #include <cmath>
00006 using namespace std;
00007
00011 class Vecteur2D {
00012 public:
00016     double x, y;
00023     explicit Vecteur2D(const double &x = 0, const double &y = 0): x(x), y(y) {}
00029     Vecteur2D operator+(const Vecteur2D& v) const {
00030         return Vecteur2D(x + v.x, y + v.y);
00031     }
00036     Vecteur2D operator-() const {
00037         return Vecteur2D(-x, -y);
00038     }
00044     Vecteur2D operator-(const Vecteur2D& v) const {
00045         return *this + -v;
00046     }
00052     const Vecteur2D & operator+=(const Vecteur2D& v) {
00053         x += v.x;
00054         y += v.y;
00055         return *this;
00056     }
00062     Vecteur2D operator*(const double& k) const {
00063         return Vecteur2D(k * x, k * y);
00064     }
00070     double operator*(const Vecteur2D & v) const {
00071         return x * v.x + y * v.y;
00072     }
00078     double operator|(const Vecteur2D & v) const {
00079         return abs(x * v.y - y * v.x);
00080     }
00085     double norme2() const {
00086         return *this * *this;
00087     }
00092     double norme() const {
00093         return sqrt(norme2());
00094     }
00099     bool operator==(const Vecteur2D & v) const {
00100         return x == v.x && y == v.y;
00101     }
00106     explicit operator string() const {
00107         ostringstream o;
00108         o << x << " " << y;
00109         return o.str();
00110     }
00111 };
00112
00117 inline ostream & operator <<(ostream& o, const Vecteur2D& v) {
00118     o << string(v);
00119     return o;
00120 }
00121
00122 #endif
```

7.39 Référence du fichier Cercle.cpp

```
#include "../include/Cercle.h"  
#include "../include/FormeVisiteur.h"
```

7.40 Référence du fichier Forme.cpp

```
#include "../include/Forme.h"  
#include "../include/FormeCharger.h"  
#include "../include/FormeChargerCercle.h"  
#include "../include/FormeChargerFormeComposee.h"  
#include "../include/FormeChargerPolygone.h"  
#include "../include/FormeChargerSegment.h"  
#include "../include/FormeChargerTriangle.h"
```

Fonctions

— ostream & [operator<<](#) (ostream &s, const [Forme](#) &f)

7.40.1 Documentation des fonctions

7.40.1.1 [operator<<\(\)](#)

```
ostream & operator<< (  
    ostream & s,  
    const Forme & f)
```

Renvoie

L'ostream complété

7.41 Référence du fichier FormeCharger.cpp

```
#include <fstream>  
#include "../include/FormeCharger.h"
```

7.42 Référence du fichier FormeChargerCercle.cpp

```
#include "../include/FormeChargerCercle.h"  
#include "../include/Cercle.h"
```

7.43 Référence du fichier `FormeChargerFormeComposee.cpp`

```
#include "../include/FormeChargerFormeComposee.h"  
#include "../include/FormeComposee.h"
```

7.44 Référence du fichier `FormeChargerPolygone.cpp`

```
#include "../include/FormeChargerPolygone.h"  
#include "../include/Polygone.h"
```

7.45 Référence du fichier `FormeChargerSegment.cpp`

```
#include "../include/FormeChargerSegment.h"  
#include "../include/Segment.h"
```

7.46 Référence du fichier `FormeChargerTriangle.cpp`

```
#include "../include/FormeChargerTriangle.h"  
#include "../include/Triangle.h"
```

7.47 Référence du fichier `FormeComposee.cpp`

```
#include "../include/FormeComposee.h"  
#include "../include/FormeVisiteur.h"
```

7.48 Référence du fichier `FormeDessiner.cpp`

```
#include "../include/FormeDessiner.h"  
#include "../include/MaWinsock.h"  
#include "../include/Matrice22.h"
```

7.49 Référence du fichier `FormeSauvegarder.cpp`

```
#include <fstream>  
#include <utility>  
#include "../include/FormeSauvegarder.h"
```


7.50 Référence du fichier FormeSimple.cpp

```
#include "../include/FormeSimple.h"  
#include "../include/Matrice22.h"
```

7.51 Référence du fichier FormeVisiteur.cpp

```
#include "../include/FormeVisiteur.h"
```

7.52 Référence du fichier Main.cpp

```
#include <iostream>  
#include <string>  
#include <unistd.h>  
#include "cmath"  
#include "../include/FormeSauvegarder.h"  
#include "../include/FormeDessiner.h"
```

Macros

```
— #define M_PI_2 1.57079632679489661923  
— #define M_PI_4 0.78539816339744830961
```

Fonctions

```
— int main ()
```

7.52.1 Documentation des macros

7.52.1.1 M_PI_2

```
#define M_PI_2 1.57079632679489661923
```

7.52.1.2 M_PI_4

```
#define M_PI_4 0.78539816339744830961
```

7.52.2 Documentation des fonctions

7.52.2.1 main()

```
int main ()  
TEST SEGMENT///  
TEST CERCLE///  
TEST TRIANGLE///  
TEST POLYGONE///  
TEST FORME COMPOSEE///
```

7.53 Référence du fichier Matrice22.cpp

```
#include "../include/Matrice22.h"
```

7.54 Référence du fichier MaWinsock.cpp

```
#include "../include/MaWinsock.h"  
#include <exception>
```

7.55 Référence du fichier Polygone.cpp

```
#include <algorithm>  
#include "../include/Polygone.h"  
#include "../include/FormeVisiteur.h"
```

7.56 Référence du fichier Segment.cpp

```
#include "../include/Segment.h"  
#include "../include/FormeVisiteur.h"
```

7.57 Référence du fichier Triangle.cpp

```
#include "../include/Triangle.h"  
#include "../include/FormeVisiteur.h"
```

7.58 Référence du fichier Vecteur2D.cpp

```
#include "../include/Vecteur2D.h"
```